
目录

序章：问题与约定	1
第一章 矩阵作为对应的坐标影子	4
1.1 条目、端点与权重	4
1.2 矩阵作用是沿目标纤维聚合	5
1.3 矩阵乘法是中间端点的消去	6
1.4 为什么需要事件空间	7
1.5 生成规则与矩阵展开	8
1.6 坐标影子的定义	9
1.7 乘法与影子相容	10
1.8 本章小结	11
第二章 集合、映射、纤维与关系	13
2.1 对象空间：先有端点，再有坐标	13
2.2 映射：对象之间的确定转移	14
2.2.1 像与原像	14
2.3 纤维：按端点分桶	15
2.4 沿映射的拉回与推前	17
2.4.1 拉回：把端点数据复制到事件上	17
2.4.2 推前：沿纤维求和	17
2.5 关系：不带权重的对应	19
2.5.1 关系的源映射与目标映射	19
2.6 关系复合：存在一条二步路径	21
2.7 纤维积：可复合事件的空间	22
2.7.1 纤维积的普遍性质	23
2.8 用纤维积重写关系复合	24
2.9 三步路径与结合律的集合模型	26
2.10 从关系到对应：为什么需要事件空间	27
2.11 本章小结	28
第三章 半环与权重	29
3.1 从路径权重到代数规则	29

3.2	有限聚合：交换么半群	30
3.3	路径合成：么半群与单位路径	31
3.4	半环的定义	32
3.5	基本例子	33
3.6	对象上的数据与半模	34
3.7	沿映射的拉回与推前	35
3.8	半环矩阵作为核算子	38
3.9	半环矩阵的路径解释	40
3.10	权重语义的几个基本计算	41
3.11	硬件解释：composer 与 reducer	42
3.12	本章小结	42
第四章	加权对应	43
4.1	从矩阵条目到事件	43
4.2	加权对应的定义	44
4.3	源纤维与目标纤维	46
4.4	对应对对象数据的作用	47
4.5	矩阵影子	49
4.6	影子会遗忘什么	50
4.7	加权对应的等价与同构	52
4.8	直和、合并与零对应	54
4.9	恒等对应	55
4.10	转置与反向对应	56
4.11	商、合并与规范化	57
4.12	加权对应的大小与稀疏度	58
4.13	本章小结	59
第五章	纤维复合：拉回、局部合成、推出	60
5.1	矩阵乘法中隐藏的三步	60
5.2	可复合对应	61
5.3	第一步：拉回与二步路径空间	62
5.4	第二步：局部权重合成	64
5.5	第三步：推出端点	65
5.6	端点纤维与路径聚合	66
5.7	复合的矩阵影子	67
5.8	三个基本例子	68
5.8.1	函数复合	68
5.8.2	关系复合	68
5.8.3	普通矩阵乘法	69
5.9	单位对应	69
5.10	复合结合律	71

5.11 算子分解：拉回、乘权、推出	73
5.12 复合之后是否要压缩	74
5.13 复合成本与矩阵展开成本	75
5.14 CoPU 中的复合语义	76
5.15 本章小结	76
第六章 矩阵语言的嵌入与正规化	78
6.1 矩阵语言的朴素形式	78
6.2 从矩阵条目到规范事件	81
6.3 原始复合为什么不是矩阵乘法本身	82
6.4 矩阵化正规化算子	84
6.5 正规化复合与矩阵乘法	86
6.6 矩阵语言作为影子商	88
6.7 为什么嵌入定理不是换皮	90
6.8 硬件解释：矩阵乘法的隐藏流水线	92
6.9 本章小结	92
第七章 迹、闭合路径谱与对应秩	94
7.1 自对应与闭合事件	94
7.2 迹的循环性	96
7.3 对应幂与长度为 k 的路径	98
7.4 闭合 k 路径与迹幂矩	100
7.5 路径 zeta 函数	102
7.6 路径谱与闭合路径增长	103
7.7 对应秩：通过隐藏对象空间分解	105
7.8 强秩、弱秩与硬件含义	107
7.9 本章小结	108
第八章 表达复杂度：为什么不展开矩阵	110
8.1 从条目到事件再到规则	110
8.2 物化成本与表示成本	111
8.3 卷积：从 Toeplitz 矩阵回到局部规则	112
8.4 稀疏性不等于结构	114
8.5 一般矩阵不能免费压缩	114
8.6 图消息传递：边空间优先于邻接矩阵	115
8.7 复合成本由中间纤维控制	116
8.8 商压缩：把重复事件放回对称性	118
8.9 张量积结构与乘积对应	119
8.10 Attention 的动态对应解释	120
8.11 表达复杂度与硬件代价	121
8.12 本章小结	122

第九章 对应处理器 CoPU：从代数原语到硬件架构	123
9.1 设计出发点：不要把结构先展开	123
9.2 对应程序的机器表示	124
9.3 纤维索引的构造	125
9.4 纤维匹配：从纤维积到 join 单元	127
9.5 权重合成与半环数据通路	128
9.6 推前聚合与正规化	129
9.7 CoPU 的最小机器状态	131
9.8 指令语义	131
9.9 微架构：单 tile 与多 tile	132
9.10 商压缩单元	134
9.11 迹单元与闭合路径	135
9.12 代价模型	135
9.13 与 SpMV、Graph Accelerator 和 GPU 的区别	136
9.14 编译器接口：从对应程序到事件流	137
9.15 FPGA 原型的最小闭环	138
9.16 实验指标	139
9.17 本章小结	140
第十章 研究路线：从形式语言到原型系统	141
10.1 为什么研究路线也要形式化	141
10.2 第一个闭环：语言层	142
10.3 第二个闭环：C-IR 中间语言	143
10.3.1 C-IR 的最小语法	143
10.3.2 C-IR 的解释函数	144
10.3.3 C-IR 的类型检查	145
10.4 第三个闭环：参考解释器	146
10.4.1 解释器的状态表示	146
10.4.2 解释器测试的基本原则	148
10.5 第四个闭环：CoPU 原型规格	148
10.5.1 CoPU v0 的最小功能	148
10.5.2 数据宽度与参数	149
10.5.3 RTL 模块划分	149
10.6 第五个闭环：实验设计	150
10.6.1 实验对象的选择	150
10.6.2 指标定义	150
10.6.3 基线的设定	150
10.7 第六个闭环：论文结构	151
10.7.1 主论文命题	151
10.7.2 论文贡献的可审查表述	151

10.8 第七个闭环：版本规划	152
10.8.1 v1: 讲义与参考解释器	152
10.8.2 v2: CoPU RTL 与 FPGA	152
10.8.3 v3: ASIC flow 与论文投稿	152
10.9 失败边界与修正方向	153
10.10 本讲义后续需要补齐的章节	153
10.11 结语：下一步的最小行动	154
记号索引	155

序章：问题与约定

本讲义讨论一个具体问题：能否建立一种数学语言，使矩阵成为其中的一个坐标表示，而不是全部语言本身。我们希望保留矩阵方法中可靠的部分：条目表示局部权重，乘法表示经过中间指标的复合，迹与幂记录闭合路径；同时避免把本来有结构的关系一律展开为二维数组。

讲义暂称这套语言为纤维对应代数。一个基本对象写作

$$X \xleftarrow{s} E \xrightarrow{t} Y, \quad w : E \rightarrow \mathbb{K}.$$

这里 X 和 Y 是端点空间， E 是事件空间， s, t 给出事件的源端点与目标端点， w 给出权重。若取 $E = X \times Y$ ，并令

$$s(x, y) = x, \quad t(x, y) = y, \quad w(x, y) = A_{yx},$$

便得到普通矩阵 $A = (A_{yx})$ 的对应表示。因此这套语言不是放弃矩阵，而是把矩阵解释为一种特殊的、坐标化的对应。

说明

本讲义的出发点是一个朴素事实：矩阵乘法

$$(BA)_{zx} = \sum_y B_{zy} A_{yx}$$

可以读作“所有二步路径 $x \rightarrow y \rightarrow z$ 的权重和”。若把这个路径解释作为基本语言，就得到对应的复合；若再把事件空间的纤维结构保留下来，就得到一种比矩阵表格更接近结构本身的表示。

写作方式

本讲义按教材方式展开。每个概念先说明它要解决的问题，再给出定义，然后证明最基本的性质。证明尽量拆成短步骤，避免把关键推理隐藏在一句“显然”里。当前版本只处理有限集合与有限事件空间；这足以覆盖第一版硬件原型和编译器中间表示。拓扑、测度、范畴论和连续极限只作为后续扩展，不在基础篇中预设。

章节安排

完整讲义计划分为五篇。

第一篇：从矩阵影子到对应对象

1. 矩阵究竟表达了什么：二维表格、线性变换与路径求和。
2. 集合、映射、纤维与关系：从最基础的语言开始。
3. 半环与权重：为什么“加法”和“乘法”不必是实数加乘。
4. 加权对应：事件空间、源目标映射、支撑与局部有限性。
5. 纤维复合：拉回、局部合成、推出。
6. 矩阵嵌入定理：普通矩阵完全忠实地嵌入对应语言。

第二篇：对应代数的基本结构

7. 单位、加法、反向对应、张量积与伴随。
8. 路径、幂、闭合路径、迹与路径 zeta 函数。
9. 对应秩：低秩分解的纤维版本。
10. 商对应与对称性：为什么矩阵展开会浪费结构。
11. 对应范畴与函子观点：把矩阵范畴作为子范畴。

第三篇：表达复杂度与算法

12. 卷积不是一般矩阵：平移不变对应的低描述复杂度。
13. 图消息传递不是邻接矩阵乘法：边空间作为原始对象。
14. Attention 不是固定 $n \times n$ 表格：动态对应空间与局部纤维。
15. 表达复杂度优势定理：某些算子矩阵表示为 $\Omega(n^2)$ ，对应表示为 $O(n)$ 或 $O(\log n)$ 。
16. 复合复杂度定理：计算成本由纤维大小控制，而不是由展开矩阵维度控制。

第四篇：对应处理器 CoPU

17. 从 MAC 到 CORR_COMPOSE：硬件原语的替换。
18. Object Table、Event Memory 与 Fiber Index。
19. Fiber Join、Weight Compose、Push Reduce 的 RTL 级语义。
20. Quotient Engine：在硬件中利用对称性与等价类。
21. 通信下界：为什么纤维匹配给出硬件瓶颈。
22. CoPU 编译器：从模型到对应程序。

第五篇：原型、论文与长期路线

23. 最小 FPGA 原型：卷积、图消息传递、局部 attention、路径迹。
24. ASIC flow：从可综合 RTL 到版图级估计。

25. 论文结构：数学语言、复杂度优势、硬件原型、实验闭环。

26. 长期目标：从矩阵计算程序到对应计算程序。

边界

当前讲义是一套研究性教材的初稿。定义可以大胆，但每个定理必须能回到集合、映射、半环和有限求和上逐步验证。若某个说法还只是路线图，应当明确标出，不能把设想写成已经证明的结论。

矩阵作为对应的坐标影子

本章的目的不是否定矩阵，而是把矩阵放回一个更原始的位置。一个矩阵可以被看成数表，也可以被看成线性映射；这两种读法在传统线性代数中已经足够有效。可是当我们关心结构、稀疏性、生成规则和硬件实现时，数表读法会把许多信息提前压平。本章采用第三种读法：矩阵是一个加权对应坐标下留下的记录。

为了避免过早引入抽象语言，本章只讨论有限集合上的情形。后面章节会把这里出现的图像逐步整理成正式定义。读者应当先记住一个简单事实：矩阵条目记录一条连接的权重，矩阵乘法记录两段连接的复合并对中间端点求和。

说明

本章试图回答三个问题。

1. 一个矩阵条目 A_{ji} 除了是第 j 行第 i 列的数以外，还能表示什么？
2. 为什么矩阵乘法的求和指标应该被理解为一个被消去的中间端点？
3. 若矩阵只是某种结构的坐标影子，那么这个结构最少需要保留哪些信息？

这三个问题给出后面“对应”“纤维”“复合”的必要性。

1.1 条目、端点与权重

设 $X = \{1, \dots, n\}$, $Y = \{1, \dots, m\}$, 并设 $A \in \mathbb{K}^{Y \times X}$ 。这里的记号表示：行由 Y 标号，列由 X 标号。因此 A_{ji} 位于输出标号 $j \in Y$ 与输入标号 $i \in X$ 的交叉处。

若只把 A 看成数表，那么 A_{ji} 只是一个坐标位置上的数。若把 i 看成源端点，把 j 看成目标端点，那么同一个数也可以读成一条连接的权重：

$$i \longrightarrow j, \quad \text{weight} = A_{ji}.$$

于是矩阵不再只是一个二维数组，而是从 X 到 Y 的带权二部连接图。

定义 1.1: 矩阵的关系读法

给定有限集合 X, Y 与矩阵 $A \in \mathbb{K}^{Y \times X}$ 。称 A 的关系读法为如下带权连接族：对每一对 $(i, j) \in X \times Y$ ，允许一条从 i 到 j 的连接，连接权重为 A_{ji} 。若 $A_{ji} = 0$ ，在稀疏表示中通常省略这条连接。

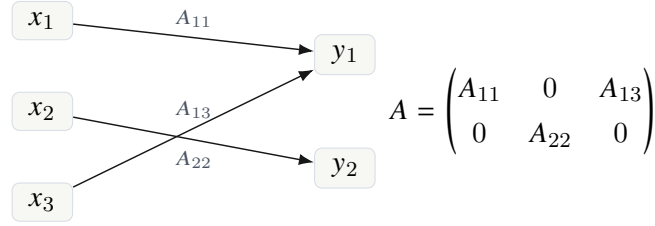


图 1.1 同一个稀疏矩阵的两种表示。右边是坐标表，左边是带权连接。

例 1.1: 一个小矩阵

设

$$A = \begin{pmatrix} 2 & 0 & 5 \\ 0 & 3 & 0 \end{pmatrix}, \quad X = \{1, 2, 3\}, \quad Y = \{1, 2\}.$$

非零连接为

$$1 \rightarrow 1 \text{ 权重 } 2, \quad 3 \rightarrow 1 \text{ 权重 } 5, \quad 2 \rightarrow 2 \text{ 权重 } 3.$$

矩阵有六个坐标位置；稀疏连接图只保留三个非零事件。这里已经出现一个重要区别：矩阵表格记录所有可能位置，事件表示记录实际发生的连接。

注记

本章暂时把 $\mathbb{K}$ 看成普通数域也无妨。后面会把 $\mathbb{K}$ 放宽为半环；那时“加法”和“乘法”可分别表示逻辑或与逻辑且、最大值与加法、概率聚合与概率相乘等运算。这样做的原因是，矩阵语言中真正重要的不是实数本身，而是“沿路径相乘、沿纤维相加”的代数规则。

1.2 矩阵作用是沿目标纤维聚合

矩阵首先作为线性映射出现。若 $u \in \mathbb{K}^X$ 是输入向量， $v = Au \in \mathbb{K}^Y$ ，则

$$v_j = \sum_{i \in X} A_{ji} u_i.$$

按照关系读法， u_i 是输入端点 i 上携带的量， A_{ji} 是从 i 到 j 的连接权重。输出 v_j 是所有到达同一目标端点 j 的贡献之和。

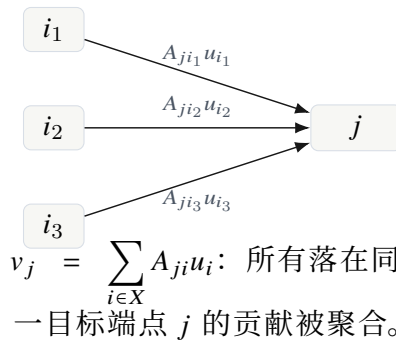


图 1.2 矩阵向量乘法可读作沿目标纤维的推前聚合。

这里的“纤维”一词可以先作朴素理解。对固定的目标端点 j ，所有指向 j 的连接组成一

个集合：

$$\{i \in X : A_{ji} \neq 0\}.$$

输出 v_j 只从这个集合中收集贡献。因此，矩阵作用并不是神秘的线性代数操作；它是“把源端点上的量沿连接送到目标端点，再在每个目标端点求和”。

命题 1.1: 矩阵向量乘法的纤维解释

设 $A \in \mathbb{K}^{Y \times X}$, $u \in \mathbb{K}^X$ 。对每个 $j \in Y$, $(Au)_j$ 等于所有以 j 为目标端点的连接贡献之和：

$$(Au)_j = \sum_{i: i \rightarrow j} A_{ji} u_i,$$

其中若允许所有 $i \in X$ 都有一条可能连接，则零权连接的贡献为零。

证明.

Step 1. 矩阵向量乘法的定义是

$$(Au)_j = \sum_{i \in X} A_{ji} u_i.$$

Step 2. 在关系读法下， A_{ji} 是连接 $i \rightarrow j$ 的权重， u_i 是源端点 i 携带的输入量。

Step 3. 连接 $i \rightarrow j$ 对目标端点 j 的贡献为 $A_{ji} u_i$ 。

Step 4. 对所有源端点 i 求和，得到目标端点 j 收到的总量。这正是 $(Au)_j$ 。

□

这个命题以后会被写成“推前”。现在只要注意：矩阵的行不是单纯的数组片段，而是一个目标端点的入射纤维。

1.3 矩阵乘法是中间端点的消去

设 $A \in \mathbb{K}^{Y \times X}$, $B \in \mathbb{K}^{Z \times Y}$ 。乘积 $BA \in \mathbb{K}^{Z \times X}$ 的条目为

$$(BA)_{zx} = \sum_{y \in Y} B_{zy} A_{yx}.$$

对固定的 $x \in X$ 与 $z \in Z$ ，求和指标 y 不是形式变量，而是一个真实的中间端点。项 A_{yx} 给出连接 $x \rightarrow y$ 的权重，项 B_{zy} 给出连接 $y \rightarrow z$ 的权重。因此 $B_{zy} A_{yx}$ 是二步路径

$$x \longrightarrow y \longrightarrow z$$

的权重。对 y 求和，就是把所有从 x 到 z 、中间经过某个 $y \in Y$ 的二步路径合并。

命题 1.2: 矩阵乘法的路径解释

设 $A \in \mathbb{K}^{Y \times X}$, $B \in \mathbb{K}^{Z \times Y}$ 。则 $(BA)_{zx}$ 等于所有从 $x \in X$ 出发、经过某个 $y \in Y$ 、到达 $z \in Z$ 的二步路径权重之和；其中路径 $x \rightarrow y \rightarrow z$ 的权重为 $B_{zy} A_{yx}$ 。

证明.

Step 1. 固定 $x \in X$ 与 $z \in Z$ 。矩阵乘法的定义给出

$$(BA)_{zx} = \sum_{y \in Y} B_{zy} A_{yx}.$$

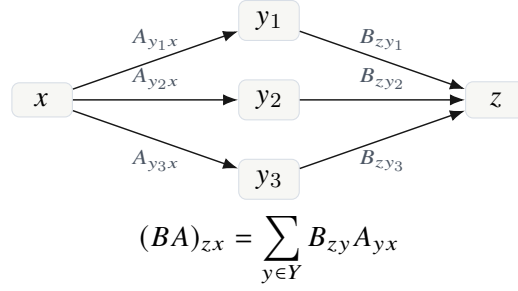


图 1.3 矩阵乘法把所有经过同一中间集合 Y 的二步路径聚合起来。

Step 2. 对每个 $y \in Y$ ，条目 A_{yx} 表示边 $x \rightarrow y$ 的权重，条目 B_{zy} 表示边 $y \rightarrow z$ 的权重。

Step 3. 若二步路径的权重定义为两条边权重的乘积，则经过 y 的路径权重为 $B_{zy}A_{yx}$ 。

Step 4. 对全部 y 求和，得到从 x 到 z 的全部二步路径贡献。因此该和正是 $(BA)_{zx}$ 。

□

这个命题指出了矩阵乘法的两个基本动作：先把能够首尾相接的边配对，再把同一对端点之间的路径权重求和。后面所谓“纤维积”和“推前”只是在抽象层面保留这两个动作。

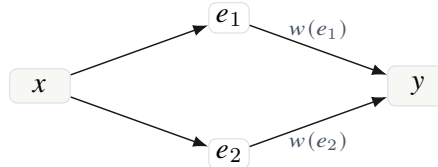
1.4 为什么需要事件空间

如果每一对端点之间至多只有一条连接，那么用 $X \times Y$ 上的矩阵条目记录权重已经足够。但是在许多结构中，一对端点之间可能存在多种不同来源的连接。它们在坐标矩阵中会被压到同一个位置。

设有一个有限集合 E ，其元素称为事件。每个事件 $e \in E$ 有源端点 $s(e) \in X$ 、目标端点 $t(e) \in Y$ 和权重 $w(e) \in \mathbb{K}$ 。若只看坐标矩阵，那么从 x 到 y 的所有事件会被合并为一个数：

$$A_{yx} = \sum_{e: s(e)=x, t(e)=y} w(e).$$

这个矩阵记录总权重，却不再区分这些贡献来自哪些事件。



坐标矩阵只看见 $A_{yx} = w(e_1) + w(e_2)$ ，不再知道两项来自不同事件。

图 1.4 事件空间保留连接的来源；矩阵影子只保留同一端点对上的总权重。

定义 1.2: 事件表示的矩阵影子

设 X, Y 是有限集合， E 是有限事件集，并给定两个端点映射

$$X \xleftarrow{s} E \xrightarrow{t} Y$$

以及权重函数 $w : E \rightarrow \mathbb{K}$ 。定义它的矩阵影子 $A_E \in \mathbb{K}^{Y \times X}$ 为

$$(A_E)_{yx} = \sum_{e \in E: s(e)=x, t(e)=y} w(e).$$

若某个端点对 (x, y) 上没有事件，则对应的和约定为 0。

这个定义把矩阵看成一个“压缩后的影子”：先按端点对把事件分组，再对每组求和。若 E 本身就是 $X \times Y$ 的子集，且每个端点对最多有一个事件，那么矩阵影子没有丢失 multiplicity；但它仍然可能丢失事件的生成方式。

例 1.2: 两种产生同一矩阵的事件结构

令 $X = \{x\}$, $Y = \{y\}$ 。第一种事件结构只有一个事件 e ，权重为 5。第二种事件结构有两个事件 e_1, e_2 ，权重分别为 2 与 3，且二者同样从 x 指向 y 。两者的矩阵影子都是 1×1 矩阵 (5)。可是若后续计算需要区分来源、时间、类型或硬件路由位置，这两种结构并不相同。

注记

这就是本讲义后来坚持保留事件空间 E 的原因。矩阵记录端点对上的总权重；对应记录事件如何产生、如何分组、如何继续复合。对于纯粹有限维线性代数，前者往往足够；对于结构化计算和硬件实现，后者才是更接近计算过程的对象。

1.5 生成规则与矩阵展开

矩阵表格默认使用 $X \times Y$ 作为所有可能连接的索引集合。若一个算子本来就是任意关系，这没有问题。但许多算子由简单规则生成。此时，把规则展开成矩阵会使表示变大，也会遮蔽原有结构。

例 1.3: 一维循环卷积

考虑长度为 n 的循环序列 $u = (u_0, \dots, u_{n-1})$ 与半径为 r 的卷积核 $h_{-r}, \dots, h_r$ 。输出为

$$v_i = \sum_{a=-r}^r h_a u_{i-a},$$

其中指标按模 n 理解。矩阵语言会把这个算子写成一个 $n \times n$ 循环 Toeplitz 矩阵。这个矩阵有 n^2 个坐标位置，但生成规则只需要说明：对每个输出位置 i 和偏移 $a \in \{-r, \dots, r\}$ ，存在事件

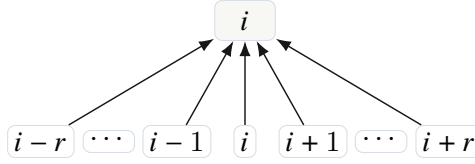
$$i - a \longrightarrow i, \quad \text{weight} = h_a.$$

因此事件数为 $n(2r + 1)$ ，规则本身只由偏移集合和核权重给出。

这个例子不是为了说明稀疏矩阵存储已经足够。稀疏矩阵仍然把事件看成孤立条目；而卷积的要点在于所有事件由同一组偏移规则生成，且不同位置上的连接具有共同权重。矩阵展开之后，这种“共同来源”需要额外恢复。

命题 1.3: 卷积矩阵影子的生成复杂度

对长度为 n 、半径为 r 的一维循环卷积，矩阵影子是一个 $n \times n$ 矩阵；其非零条目数至多为 $n(2r + 1)$ 。但若以偏移规则表示该算子，只需记录 $2r + 1$ 个偏移权重以及“对每个位置平



卷积事件由偏移 a 生成： $(i - a) \rightarrow i$ 。矩阵展开记录坐标结果；事件表示记录生成机制。

图 1.5 局部卷积的连接由少量偏移规则生成，不必先形成完整矩阵。

移使用同一规则”这一生成说明。

证明.

Step 1. 对每个输出位置 i ，公式

$$v_i = \sum_{a=-r}^r h_a u_{i-a}$$

说明 i 至多连接到 $2r + 1$ 个输入位置。

Step 2. 因此全部输出位置贡献的非零连接数至多为 $n(2r + 1)$ 。

Step 3. 若使用生成规则表示，则偏移集合为 $\{-r, \dots, r\}$ ，其大小为 $2r + 1$ ；每个偏移 a 只需记录一次权重 h_a 。

Step 4. 剩余信息是统一规则“在所有 i 上按模 n 平移使用相同偏移集合”。这条规则不随 n^2 个矩阵位置逐项展开。

□

1.6 坐标影子的定义

现在可以给出本章的核心说法。一个结构对象可能先有事件、端点、权重和生成规则；选择坐标并按端点对聚合之后，才得到矩阵。得到的矩阵称为坐标影子。

定义 1.3: 坐标影子

设一个结构对象 $\mathcal{A}$ 在选择有限端点集合、坐标标号以及聚合方式后产生矩阵 A 。称 A 是 $\mathcal{A}$ 在该选择下的坐标影子。若不同坐标或不同聚合方式产生不同矩阵，但它们来自同一结构对象，则这些矩阵只是同一对象的不同影子。

在线性代数中，同一个线性映射在不同基底下有不同矩阵；因此矩阵不是线性映射本身。这里的说法是同一思想的关系版本：同一个事件结构在展开、合并、排序、商化以后可以产生不同矩阵。矩阵是可计算的记录，但未必是最原始的对象。

定理 1.1: 矩阵影子原则的有限版本

设 X, Y 是有限集合。

1. 任意事件表示 $X \xleftarrow{s} E \xrightarrow{t} Y$ 及权重函数 $w : E \rightarrow \mathbb{K}$ 都产生一个矩阵影子 $A_E \in \mathbb{K}^{Y \times X}$:

$$(A_E)_{yx} = \sum_{e: s(e)=x, t(e)=y} w(e).$$

2. 任意矩阵 $A \in \mathbb{K}^{Y \times X}$ 都可由一个事件表示产生: 取事件集

$$E_A = \{(x, y) \in X \times Y : A_{yx} \neq 0\},$$

$$\text{令 } s(x, y) = x, t(x, y) = y, w(x, y) = A_{yx}.$$

因此, 在有限集合上, 矩阵与某些事件表示可以互相转换; 但矩阵影子一般不记录事件 multiplicity、事件类型或生成规则。

证明.

- Step 1.** 对第一部分, 固定 $x \in X$ 与 $y \in Y$ 。所有满足 $s(e) = x$ 、 $t(e) = y$ 的事件组成一个有限集合。把这些事件的权重相加, 即得到矩阵位置 (y, x) 上的数。对每个端点对都这样做, 得到矩阵 A_E 。
- Step 2.** 对第二部分, 从矩阵 A 出发, 只保留非零条目。每个非零条目 A_{yx} 给出一个事件 (x, y) , 其源端点为 x , 目标端点为 y , 权重为 A_{yx} 。
- Step 3.** 按第一部分构造该事件表示的矩阵影子。若 $A_{yx} \neq 0$, 对应端点对上恰有一个事件, 影子条目为 A_{yx} ; 若 $A_{yx} = 0$, 对应端点对上没有事件, 影子条目为 0。因此影子矩阵等于原矩阵 A 。
- Step 4.** 这个构造说明任意矩阵都可以被看成事件表示的影子。但反向并不唯一: 多个事件可落在同一端点对上, 或由同一生成规则产生。取矩阵影子时, 这些信息被合并或遗忘。

□

1.7 乘法与影子相容

如果矩阵只是影子, 那么必须检查矩阵乘法是否也能从事件结构中恢复。答案是肯定的。两个事件结构复合时, 先把目标端点与源端点相同的事件首尾相接, 再把得到的二步事件按外部端点聚合。其矩阵影子正是矩阵乘积。

设

$$X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, \quad Y \xleftarrow{s_B} E_B \xrightarrow{t_B} Z$$

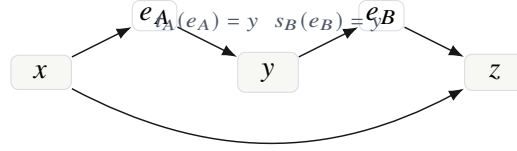
是两个事件表示。可复合的二步事件是所有满足

$$t_A(e_A) = s_B(e_B)$$

的事件对 (e_B, e_A) 。它们形成集合

$$E_B \times_Y E_A = \{(e_B, e_A) : s_B(e_B) = t_A(e_A)\}.$$

这个集合以后称为纤维积。现在只要把它理解成“所有首尾相接的事件对”。二步事件的源端点是 $s_A(e_A)$, 目标端点是 $t_B(e_B)$, 权重是 $w_B(e_B)w_A(e_A)$ 。



只有当第一段的目标端点等于第二段的源端点时，两个事件才能复合成一条二步事件。

图 1.6 事件复合的基本图像：首尾相接，然后按外部端点聚合。

命题 1.4: 复合影子等于影子乘积

令 A 与 B 分别为上述两个事件表示的矩阵影子。由二步事件集合 $E_B \times_Y E_A$ 构造的复合事件表示，其矩阵影子等于矩阵乘积 BA 。

证明.

Step 1. 固定外部端点 $x \in X$ 与 $z \in Z$ 。复合事件影子在位置 (z, x) 上的条目为所有二步事件权重之和：

$$\sum_{(e_B, e_A): s_A(e_A)=x, t_B(e_B)=z, s_B(e_B)=t_A(e_A)} w_B(e_B)w_A(e_A).$$

Step 2. 按共同的中间端点 $y \in Y$ 分组。上式等于

$$\sum_{y \in Y} \left(\sum_{e_B: s_B(e_B)=y, t_B(e_B)=z} w_B(e_B) \right) \left(\sum_{e_A: s_A(e_A)=x, t_A(e_A)=y} w_A(e_A) \right).$$

Step 3. 括号中的两项分别是矩阵影子 B 的条目 B_{zy} 与矩阵影子 A 的条目 A_{yx} 。

Step 4. 因此复合影子在 (z, x) 处的条目为

$$\sum_{y \in Y} B_{zy} A_{yx} = (BA)_{zx}.$$

Step 5. 对任意 x, z 都成立，所以复合事件表示的矩阵影子就是 BA 。

□

这个命题是后续理论的第一块基石。它说明：如果把矩阵替换为事件对应，乘法并不会消失；它变成了更原始的“可接事件配对”和“外部端点聚合”。矩阵乘法只是这种复合在坐标表上的影子。

1.8 本章小结

本章建立了一个有限层面的基本图像。

1. 矩阵条目 A_{ji} 可以读作从输入端点 i 到输出端点 j 的连接权重。
2. 矩阵向量乘法是把源端点上的量沿连接送出，并在每个目标端点的入射纤维上求和。
3. 矩阵乘法是把两段连接按中间端点首尾相接，并把所有二步路径按外部端点聚合。

4. 事件空间 E 比端点对集合 $X \times Y$ 保留更多信息：multiplicity、来源、类型、生成规则和后续可复合结构。
5. 矩阵是事件结构在坐标化和聚合之后得到的影子，而不是必须作为最原始对象。

下一章将从集合、映射、纤维和关系开始，把本章的图像写成可反复使用的语言。

练习 1.1: 路径读法训练

取 $X = \{1, 2\}$, $Y = \{a, b, c\}$, $Z = \{u, v\}$ 。任取两个稀疏矩阵 $A \in \mathbb{K}^{Y \times X}$ 与 $B \in \mathbb{K}^{Z \times Y}$ 。画出对应的二部图，并逐项验证 $(BA)_{zx}$ 等于全部二步路径权重和。

练习 1.2: 同一影子的不同事件结构

构造两个不同的事件表示 $X \leftarrow E \rightarrow Y$ ，使得它们有相同的矩阵影子，但事件数不同。进一步说明：若这些事件带有不同的类型标签，矩阵影子为什么无法恢复这些标签。

练习 1.3: 卷积的事件表示

对长度为 5、半径为 1 的循环卷积写出全部事件 $(i - a) \rightarrow i$ ，并写出对应的 5×5 矩阵影子。比较事件规则、事件列表与矩阵表格三种表示的差异。

集合、映射、纤维与关系

第一章把矩阵解释为某种更原始对象的坐标影子。为了把这个想法真正展开，我们现在必须退回到最朴素的语言：集合、映射、纤维和关系。这里不急着引入加法、乘法和权重。原因很简单：矩阵乘法中最根本的动作并不是数的乘加，而是先把可以连接的项找出来，再沿着中间指标聚合。找出来靠的是端点匹配；聚合靠的是纤维。

本章的目的有三个。第一，建立对象空间与映射的基本语言。第二，解释为什么纤维是“按坐标计算”到“按结构计算”的分界点。第三，把关系复合写成纤维积上的路径复合，为后面的加权对应复合做准备。

安排

本章所有集合先假定为有限集合。这个限制不是理论本身的必要限制，而是为了把代数定义和芯片数据结构保持在同一个层面。有限集合中的映射、纤维、分桶、连接、聚合，都可以直接落到编译器中间表示和硬件存储结构上。

2.1 对象空间：先有端点，再有坐标

矩阵语言一开始给出的是一个数组

$$A = (A_{yx})_{y \in Y, x \in X}.$$

这里的 x 与 y 通常被称为列指标和行指标。但是在对应语言中， x 和 y 首先不是坐标，而是两个集合中的对象。换言之，我们从不从数组出发，而从数组所索引的两个空间出发。

定义 2.1: 对象空间

一个对象空间是一个有限集合 X 。它的元素称为对象、状态、端点或索引。若 $|X| = n$ ，则称 X 有 n 个对象。

对象空间只包含“有哪些对象”这一信息，不包含加法、乘法、距离、顺序或内积。把 X 写成 $\{1, \dots, n\}$ 只是选择了一个编号。选择编号之后，函数 $u : X \rightarrow \mathbb{K}$ 可以被写成一个列向量；但是函数本身并不依赖这个编号。

定义 2.2: 对象上的数据

设 X 是有限集合， $\mathbb{K}$ 是一个数系或权重集合。一个定义在 X 上的数据是一个函数

$$u : X \longrightarrow \mathbb{K}.$$

全体这样的函数记为 $\mathbb{K}^X$ 。

如果 $X = \{x_1, \dots, x_n\}$, 则 $u \in \mathbb{K}^X$ 可以写成

$$\begin{pmatrix} u(x_1) \\ \vdots \\ u(x_n) \end{pmatrix}.$$

这个列向量只是 u 在所选编号下的坐标表达。换一个编号, 列向量的排列会变, 但 u 作为函数没有改变。

例 2.1: 对象空间的几种来源

对象空间的含义随问题而变:

- 图像处理中, X 可以是像素位置集合;
- 图计算中, X 可以是顶点集合;
- 注意力机制中, X 可以是 token 位置集合;
- 芯片结构中, X 可以是 tile、bank 或 routing node 的集合;
- 状态机中, X 可以是离散状态集合。

这些例子都可以被编号成 $\{1, \dots, n\}$, 但编号通常不是结构本身。对应语言的一个基本原则是尽量晚地选择坐标。

注记

后文经常把 X 称为输入端点空间, 把 Y 称为输出端点空间。这只是为了叙述方便。数学上它们都是有限集合; “输入” 和 “输出” 来自我们给对应指定的方向。

2.2 映射: 对象之间的确定转移

矩阵语言中有两种不同类型的结构常被混在一起: 一种是确定映射, 例如每个事件有唯一的目标端点; 另一种是不确定或多值关系, 例如一个输入端点可以连接到许多输出端点。我们先讨论前者。

定义 2.3: 映射

设 E 与 Y 是集合。一个从 E 到 Y 的映射是一个规则 $f: E \rightarrow Y$, 它给每个 $e \in E$ 指定唯一的元素 $f(e) \in Y$ 。

映射的关键词是“唯一”。一个事件可以有唯一的源端点, 也可以有唯一的目标端点; 但一个端点可以接收许多事件, 也可以不接收事件。这种“一对多地被归类”的现象正是纤维的来源。

2.2.1 像与原像

定义 2.4: 像与原像

设 $f: E \rightarrow Y$ 是映射。

1. 若 $A \subseteq E$, 则

$$f(A) = \{f(e) : e \in A\} \subseteq Y$$

称为 A 在 f 下的像。

2. 若 $B \subseteq Y$, 则

$$f^{-1}(B) = \{e \in E : f(e) \in B\} \subseteq E$$

称为 B 在 f 下的原像。

原像比像更稳定，因为它严格保留集合运算。

引理 2.1: 原像保留基本集合运算

设 $f : E \rightarrow Y$ 是映射, $B_1, B_2 \subseteq Y$ 。则

$$f^{-1}(B_1 \cup B_2) = f^{-1}(B_1) \cup f^{-1}(B_2),$$

$$f^{-1}(B_1 \cap B_2) = f^{-1}(B_1) \cap f^{-1}(B_2),$$

并且

$$f^{-1}(Y \setminus B_1) = E \setminus f^{-1}(B_1).$$

证明.

Step 1. 证明第一个等式。任取 $e \in E$ 。有

$$e \in f^{-1}(B_1 \cup B_2) \iff f(e) \in B_1 \cup B_2.$$

Step 2. 按并集定义, $f(e) \in B_1 \cup B_2$ 等价于 $f(e) \in B_1$ 或 $f(e) \in B_2$ 。

Step 3. 这又等价于 $e \in f^{-1}(B_1)$ 或 $e \in f^{-1}(B_2)$, 也就是

$$e \in f^{-1}(B_1) \cup f^{-1}(B_2).$$

第一个等式成立。

Step 4. 第二个等式同理:

$$e \in f^{-1}(B_1 \cap B_2) \iff f(e) \in B_1 \cap B_2$$

等价于 $f(e) \in B_1$ 且 $f(e) \in B_2$, 即 $e \in f^{-1}(B_1) \cap f^{-1}(B_2)$ 。

Step 5. 对补集,

$$e \in f^{-1}(Y \setminus B_1) \iff f(e) \notin B_1 \iff e \notin f^{-1}(B_1).$$

因此 $f^{-1}(Y \setminus B_1) = E \setminus f^{-1}(B_1)$ 。

□

2.3 纤维：按端点分桶

矩阵表达中的求和通常写成

$$\sum_x A_{yx} u_x.$$

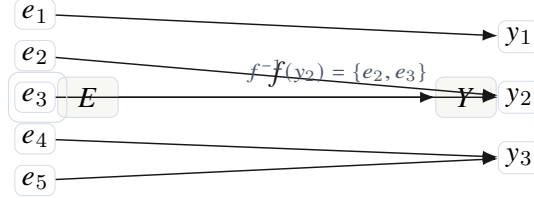
从对应语言看, 这个求和不是“对所有坐标机械地扫一遍”, 而是“把所有落到同一个输出端点 y 的贡献聚合”。因此我们需要精确描述“落到同一个端点”的集合。

定义 2.5: 纤维

设 $f : E \rightarrow Y$ 是一个映射, $y \in Y$ 。集合

$$f^{-1}(y) = \{e \in E : f(e) = y\}$$

称为 f 在 y 上的纤维。若 $f^{-1}(y) = \emptyset$, 则称该纤维为空纤维。



上图中, y_2 的纤维包含两个事件 e_2, e_3 。若硬件按目标端点分桶, 那么这两个事件会进入同一个 bucket。后续推前、聚合、归约都发生在这样的 bucket 内。

引理 2.2: 纤维分解

设 $f : E \rightarrow Y$ 是有限集合之间的映射。则

$$E = \bigsqcup_{y \in Y} f^{-1}(y).$$

特别地,

$$|E| = \sum_{y \in Y} |f^{-1}(y)|.$$

证明.

Step 1. 先证明覆盖性。任取 $e \in E$ 。由于 f 是映射, $f(e)$ 是 Y 中唯一的一个元素。令 $y = f(e)$, 则 $e \in f^{-1}(y)$ 。所以每个 e 都属于右边某个纤维。

Step 2. 再证明互不相交性。若 $e \in f^{-1}(y_1) \cap f^{-1}(y_2)$, 则按照纤维定义有 $f(e) = y_1$ 且 $f(e) = y_2$ 。因此 $y_1 = y_2$ 。所以不同 y 对应的纤维没有公共元素。

Step 3. 由覆盖性和互不相交性, E 是所有纤维的不交并。

Step 4. 有限集合的不交并大小等于各部分大小之和, 于是

$$|E| = \sum_{y \in Y} |f^{-1}(y)|.$$

□

例 2.2: 几类基本映射的纤维

1. 若 $f : E \rightarrow Y$ 是常值映射, $f(e) = y_0$, 则 $f^{-1}(y_0) = E$, 而其他纤维为空。
2. 若 $f : E \rightarrow Y$ 是单射, 则每个纤维至多包含一个元素。
3. 若 $f : E \rightarrow Y$ 是满射, 则每个 $y \in Y$ 的纤维都非空。
4. 若 $\pi_2 : X \times Y \rightarrow Y$ 是投影, $\pi_2(x, y) = y$, 则

$$\pi_2^{-1}(y) = X \times \{y\}.$$

这正是矩阵某一行或某一输出端点对应的整组输入位置。

硬件解释

纤维分解对应于硬件中的 key-indexed bucket。若事件记录为 (src, dst, payload)，按 dst 分桶就是目标映射 $t: E \rightarrow Y$ 的纤维分解。按 src 分桶则是源映射 $s: E \rightarrow X$ 的纤维分解。对应处理器必须把这两种索引都当成一等对象。

2.4 沿映射的拉回与推前

若 $f: E \rightarrow Y$ ，那么 Y 上的数据可以被拉回到 E 上；反过来， E 上的数据可以沿纤维聚合到 Y 上。这两个操作是后面矩阵影子和对应复合的基本构件。

2.4.1 拉回：把端点数据复制到事件上

定义 2.6: 拉回

设 $f: E \rightarrow Y$ 是映射， $v \in \mathbb{K}^Y$ 是 Y 上的数据。定义 v 沿 f 的拉回

$$f^*v \in \mathbb{K}^E$$

为

$$(f^*v)(e) = v(f(e)), \quad e \in E.$$

拉回的意思是：事件 e 继承它所落到的端点 $f(e)$ 的值。它不做求和，只做复制或读取。

例 2.3: 按目标端点读取数据

设 $t: E \rightarrow Y$ 是事件的目标端点映射。若 $v(y)$ 是每个输出端点的某个阈值或状态，则 t^*v 给每个事件 e 分配数值

$$(t^*v)(e) = v(t(e)).$$

因此所有落到同一个目标端点 y 的事件都会读取同一个值 $v(y)$ 。

2.4.2 推前：沿纤维求和

为了定义推前，我们需要 $\mathbb{K}$ 上至少能做加法。本章暂时可以把 $\mathbb{K}$ 看成 $\mathbb{R}$ 、 $\mathbb{C}$ 或任意有加法和零元的交换幺半群。完整的权重半环放在下一章。

定义 2.7: 推前

设 $f: E \rightarrow Y$ 是有限集合之间的映射， $a \in \mathbb{K}^E$ 是 E 上的数据。定义 a 沿 f 的推前

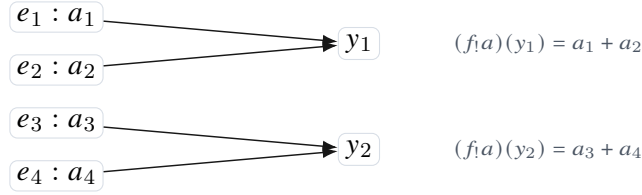
$$f_!a \in \mathbb{K}^Y$$

为

$$(f_!a)(y) = \sum_{e \in f^{-1}(y)} a(e), \quad y \in Y.$$

若纤维为空，则该和定义为 0。

这个公式是矩阵语言中的“行求和”在集合层面的原型。它只说一件事：同一纤维中的贡献要聚合到同一个端点。



引理 2.3: 推前的线性性

设 $f : E \rightarrow Y$ 是有限集合映射, $a, b \in \mathbb{K}^E$, $\lambda \in \mathbb{K}$ 。在通常数域或半环意义下, 有

$$f_!(a + b) = f_!a + f_!b,$$

以及

$$f_!(\lambda a) = \lambda f_!a.$$

证明.

Step 1. 对任意 $y \in Y$, 按照推前定义,

$$(f_!(a + b))(y) = \sum_{e \in f^{-1}(y)} (a(e) + b(e)).$$

Step 2. 有限和可以逐项拆开, 因此

$$\sum_{e \in f^{-1}(y)} (a(e) + b(e)) = \sum_{e \in f^{-1}(y)} a(e) + \sum_{e \in f^{-1}(y)} b(e).$$

Step 3. 右边正是 $(f_!a)(y) + (f_!b)(y)$ 。由于对每个 y 都成立, 得到 $f_!(a + b) = f_!a + f_!b$ 。

Step 4. 对标量乘法同理:

$$(f_!(\lambda a))(y) = \sum_{e \in f^{-1}(y)} \lambda a(e) = \lambda \sum_{e \in f^{-1}(y)} a(e) = \lambda (f_!a)(y).$$

□

命题 2.1: 推前的合成律

设

$$E \xrightarrow{f} Y \xrightarrow{g} Z$$

是有限集合映射。则对任意 $a \in \mathbb{K}^E$, 有

$$(g \circ f)_!a = g_!(f_!a).$$

证明.

Step 1. 任取 $z \in Z$ 。左边为

$$((g \circ f)_!a)(z) = \sum_{e \in (g \circ f)^{-1}(z)} a(e).$$

Step 2. 条件 $e \in (g \circ f)^{-1}(z)$ 等价于 $g(f(e)) = z$ ，也等价于 $f(e) \in g^{-1}(z)$ 。

Step 3. 因此集合 $(g \circ f)^{-1}(z)$ 可以按 $y \in g^{-1}(z)$ 分解为

$$(g \circ f)^{-1}(z) = \bigsqcup_{y \in g^{-1}(z)} f^{-1}(y).$$

Step 4. 于是左边等于

$$\sum_{y \in g^{-1}(z)} \sum_{e \in f^{-1}(y)} a(e).$$

Step 5. 内层和按照推前定义就是 $(f_! a)(y)$ 。所以左边等于

$$\sum_{y \in g^{-1}(z)} (f_! a)(y) = (g_! (f_! a))(z).$$

Step 6. 对每个 z 都成立，因此 $(g \circ f)_! = g_! \circ f_!$ 。

□

说明

这个命题是“沿纤维求和可以分阶段完成”的精确表达。矩阵乘法的结合律、关系复合的结合律、对应复合的结合律，都可以追溯到这种分桶聚合的相容性。

2.5 关系：不带权重的对应

映射 $f : E \rightarrow Y$ 是确定的：每个 e 有唯一的像。关系则允许一个输入对象连到多个输出对象，也允许一个输出对象接收多个输入对象。

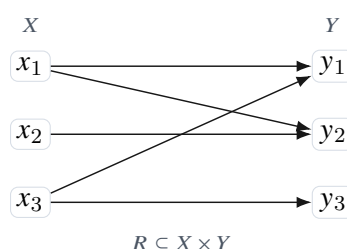
定义 2.8: 二元关系

设 X, Y 是集合。一个从 X 到 Y 的二元关系是笛卡尔积 $X \times Y$ 的一个子集

$$R \subseteq X \times Y.$$

若 $(x, y) \in R$ ，记作 xRy ，也可画成一条边 $x \rightarrow y$ 。

关系可以被看成一张二部图：左边是 X ，右边是 Y ，边集合是 R 。这张图还没有权重，因此只记录“是否存在连接”。



2.5.1 关系的源映射与目标映射

虽然关系定义为 $X \times Y$ 的子集，但每条关系边本身有两个端点。因此它天然带有两个映射。

定义 2.9: 关系的源映射与目标映射

设 $R \subseteq X \times Y$ 是关系。定义

$$s_R : R \rightarrow X, \quad s_R(x, y) = x,$$

$$t_R : R \rightarrow Y, \quad t_R(x, y) = y.$$

分别称为 R 的源映射和目标映射。

于是一个关系可以写成一幅图式

$$X \xleftarrow{s_R} R \xrightarrow{t_R} Y.$$

这已经是对应语言的雏形。差别在于，普通关系要求 R 嵌入 $X \times Y$ ，所以同一个端点对 (x, y) 最多只有一条边。一般对应允许多个不同事件拥有同一对端点。

定义 2.10: 简单对应

设 X, Y 是有限集合。一个从 X 到 Y 的简单对应是一组三元数据

$$X \xleftarrow{s} E \xrightarrow{t} Y$$

使得映射

$$(s, t) : E \rightarrow X \times Y, \quad e \mapsto (s(e), t(e))$$

是单射。

若 (s, t) 是单射，那么每个端点对至多对应一个事件，故 E 可以被识别为 $X \times Y$ 的一个子集。这正是普通关系。

命题 2.2: 关系与简单对应等价

从 X 到 Y 的二元关系与从 X 到 Y 的简单对应是一回事，差别只在表示方式。

证明.

Step 1. 给定关系 $R \subseteq X \times Y$ ，取事件空间 $E = R$ ，并定义源映射和目标映射

$$s(x, y) = x, \quad t(x, y) = y.$$

则 $(s, t) : R \rightarrow X \times Y$ 正是包含映射，因此是单射。所以得到一个简单对应。

Step 2. 反过来，给定简单对应 $X \xleftarrow{s} E \xrightarrow{t} Y$ ，由于 (s, t) 是单射， E 中不同事件给出不同端点对。令

$$R_E = \{(s(e), t(e)) : e \in E\} \subseteq X \times Y.$$

则 R_E 是一个关系。

Step 3. 因为 (s, t) 是单射， E 与 R_E 之间由 $e \mapsto (s(e), t(e))$ 给出双射。因此两种表示包含同样信息。

□

边界

一般对应不要求 (s, t) 单射。也就是说，可以有 $e_1 \neq e_2$ ，但

$$s(e_1) = s(e_2), \quad t(e_1) = t(e_2).$$

这不是冗余，而是为了保留不同机制、不同路径、不同类型或不同硬件来源。矩阵影子会把这些事件压成同一个坐标项；对应语言把它们保留下来。

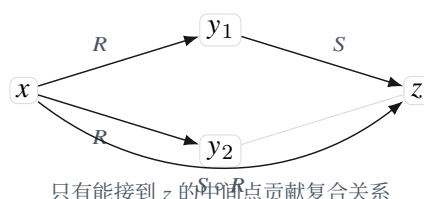
2.6 关系复合：存在一条二步路径

关系复合是矩阵乘法的无权版本。它不计算路径权重，只判断是否存在可以连接的二步路径。

定义 2.11: 关系复合

设 $R \subseteq X \times Y$, $S \subseteq Y \times Z$ 。它们的复合 $S \circ R \subseteq X \times Z$ 定义为

$$x(S \circ R)z \iff \exists y \in Y \text{ 使得 } xRy \text{ 且 } ySz.$$

**命题 2.3: 关系复合的结合律**

设

$$R \subseteq W \times X, \quad S \subseteq X \times Y, \quad T \subseteq Y \times Z.$$

则

$$T \circ (S \circ R) = (T \circ S) \circ R.$$

证明.

Step 1. 任取 $w \in W$ 与 $z \in Z$ 。我们证明 w 与 z 在左边有关系当且仅当在右边有关系。

Step 2. 左边成立，指的是

$$w(T \circ (S \circ R))z.$$

按复合定义，存在 $y \in Y$ ，使得

$$w(S \circ R)y, \quad yTz.$$

Step 3. 再展开 $w(S \circ R)y$ ，得到存在 $x \in X$ ，使得

$$wRx, \quad xSy.$$

因此左边成立等价于存在 $x \in X$ 与 $y \in Y$ ，使得

$$wRx, \quad xSy, \quad yTz.$$

Step 4. 右边成立，指的是

$$w((T \circ S) \circ R)z.$$

按复合定义，存在 $x \in X$ ，使得

$$wRx, \quad x(T \circ S)z.$$

Step 5. 展开 $x(T \circ S)z$ ，得到存在 $y \in Y$ ，使得

$$xSy, \quad yTz.$$

因此右边也等价于存在 $x \in X$ 与 $y \in Y$ ，使得

$$wRx, \quad xSy, \quad yTz.$$

Step 6. 左右两边描述的是同一组三步路径 $w \rightarrow x \rightarrow y \rightarrow z$ 的存在性。因此两边关系相同。

□

这个证明的关键不在符号，而在路径数据。无论先复合 R, S 还是先复合 S, T ，最后被记录的都是同样的三步路径是否存在。

2.7 纤维积：可复合事件的空间

关系复合的定义中出现了“存在中间点 y ”。但是如果我们以后要给路径赋权，就不能只知道存在性；我们需要把所有可复合路径本身收集成一个集合。这个集合就是纤维积。

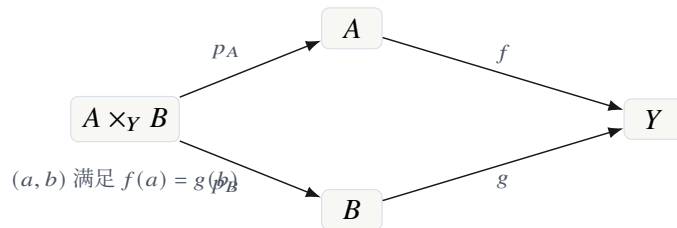
定义 2.12: 纤维积

设 $f: A \rightarrow Y$ ， $g: B \rightarrow Y$ 是两个映射。它们在 Y 上的纤维积定义为

$$A \times_Y B = \{(a, b) \in A \times B : f(a) = g(b)\}.$$

条件 $f(a) = g(b)$ 称为匹配条件。

纤维积不是普通笛卡尔积。笛卡尔积 $A \times B$ 包含所有配对；纤维积只保留中间标签相同的配对。换言之，纤维积是带约束的连接操作。



引理 2.4: 纤维积的分桶分解

若 A, B, Y 有限，则

$$A \times_Y B = \bigsqcup_{y \in Y} f^{-1}(y) \times g^{-1}(y).$$

因此

$$|A \times_Y B| = \sum_{y \in Y} |f^{-1}(y)| |g^{-1}(y)|.$$

证明.

Step 1. 任取 $(a, b) \in A \times_Y B$ 。由定义, $f(a) = g(b)$ 。记这个共同值为 y 。

Step 2. 于是 $a \in f^{-1}(y)$ 且 $b \in g^{-1}(y)$, 所以

$$(a, b) \in f^{-1}(y) \times g^{-1}(y).$$

这证明左边包含在右边各桶的并中。

Step 3. 反过来, 若 $(a, b) \in f^{-1}(y) \times g^{-1}(y)$, 则 $f(a) = y$ 且 $g(b) = y$, 所以 $f(a) = g(b)$ 。因此 $(a, b) \in A \times_Y B$ 。

Step 4. 不交性来自共同值的唯一性。若同一个 (a, b) 同时属于 $f^{-1}(y_1) \times g^{-1}(y_1)$ 与 $f^{-1}(y_2) \times g^{-1}(y_2)$, 则 $f(a) = y_1 = y_2$ 。

Step 5. 因此得到不交并分解。由于每个桶的大小为 $|f^{-1}(y)| |g^{-1}(y)|$, 对 y 求和即得基数公式。

□

硬件解释

这是对应处理器的第一条核心成本公式。若硬件要匹配两组事件 A 与 B , 不需要尝试所有 $|A||B|$ 个配对, 而是按中间键 y 分桶, 只在同一桶内连接。实际候选数量是

$$\sum_{y \in Y} |f^{-1}(y)| |g^{-1}(y)|.$$

这个数可能远小于 $|A||B|$, 也可能因为某些桶过载而很大。因此纤维负载分布是后续架构设计的基本对象。

2.7.1 纤维积的普遍性质

为了避免过早引入范畴论, 我们只给出有限集合中的直接表述。纤维积的本质是: 它是所有满足两个映射到 Y 后相同的配对的最大空间。

命题 2.4: 纤维积的普遍性质

设 $f: A \rightarrow Y$, $g: B \rightarrow Y$, 并令 $P = A \times_Y B$ 。记投影为

$$p_A: P \rightarrow A, \quad p_B: P \rightarrow B.$$

则

$$f \circ p_A = g \circ p_B.$$

并且若集合 Q 带有映射 $\alpha: Q \rightarrow A$ 与 $\beta: Q \rightarrow B$, 满足

$$f \circ \alpha = g \circ \beta,$$

则存在唯一映射 $h: Q \rightarrow P$, 使得

$$p_A \circ h = \alpha, \quad p_B \circ h = \beta.$$

证明.

Step 1. 对任意 $(a, b) \in P$, 按照 P 的定义有 $f(a) = g(b)$ 。而 $p_A(a, b) = a$, $p_B(a, b) = b$ 。所以

$$(f \circ p_A)(a, b) = f(a) = g(b) = (g \circ p_B)(a, b).$$

这说明 $f \circ p_A = g \circ p_B$ 。

Step 2. 现在给定 Q 、 α 、 β , 且 $f \circ \alpha = g \circ \beta$ 。对每个 $q \in Q$, 定义

$$h(q) = (\alpha(q), \beta(q)).$$

Step 3. 因为 $f(\alpha(q)) = g(\beta(q))$, 所以 $(\alpha(q), \beta(q)) \in A \times_Y B = P$ 。因此 h 的确是从 Q 到 P 的映射。

Step 4. 由定义立即有

$$p_A(h(q)) = \alpha(q), \quad p_B(h(q)) = \beta(q),$$

故 $p_A \circ h = \alpha$, $p_B \circ h = \beta$ 。

Step 5. 唯一性如下。若 $h' : Q \rightarrow P$ 也满足同样条件, 则对每个 q , $h'(q)$ 是 P 中某个有序对。其第一分量必须是 $\alpha(q)$, 第二分量必须是 $\beta(q)$ 。所以

$$h'(q) = (\alpha(q), \beta(q)) = h(q).$$

因此 $h' = h$ 。

□

这个命题说明, 纤维积不是任意构造出来的集合, 而是由“两个映射必须匹配”这一要求唯一决定的对象。后面处理三重复合时, 我们会不断用到这一思想。

2.8 用纤维积重写关系复合

设关系

$$R \subseteq X \times Y, \quad S \subseteq Y \times Z.$$

它们有源映射与目标映射

$$X \xleftarrow{s_R} R \xrightarrow{t_R} Y, \quad Y \xleftarrow{s_S} S \xrightarrow{t_S} Z.$$

一条可复合二步路径应当是一对比边 (e_S, e_R) , 其中 e_R 的目标端点等于 e_S 的源端点:

$$t_R(e_R) = s_S(e_S).$$

因此二步路径空间是纤维积

$$S \times_Y R = \{(e_S, e_R) : s_S(e_S) = t_R(e_R)\}.$$

注意这里写作 $S \times_Y R$, 顺序与复合 $S \circ R$ 一致: 先走 R , 再走 S 。

定义 2.13: 二步路径的端点映射

对 $S \times_Y R$ 中的元素 (e_S, e_R) , 定义源端点和目标端点为

$$s(e_S, e_R) = s_R(e_R) \in X, \quad t(e_S, e_R) = t_S(e_S) \in Z.$$

于是有一个对应

$$X \xleftarrow{s} S \times_Y R \xrightarrow{t} Z.$$

命题 2.5: 关系复合是二步路径的端点影子

设 $R \subseteq X \times Y$, $S \subseteq Y \times Z$ 。由二步路径空间 $S \times_Y R$ 得到的端点对集合

$$\{(s_R(e_R), t_S(e_S)) : (e_S, e_R) \in S \times_Y R\} \subseteq X \times Z$$

正是关系复合 $S \circ R$ 。

证明.

Step 1. 先取二步路径 $(e_S, e_R) \in S \times_Y R$ 。把 e_R 写成 (x, y) , 把 e_S 写成 (y', z) 。

Step 2. 纤维积条件给出 $t_R(e_R) = s_S(e_S)$, 即 $y = y'$ 。因此 xRy 且 ySz 。

Step 3. 按关系复合定义, 存在这样的 y 使 xRy 且 ySz , 所以 $x(S \circ R)z$ 。

Step 4. 反过来, 若 $x(S \circ R)z$, 则存在 $y \in Y$, 使得 xRy 且 ySz 。

Step 5. 令 $e_R = (x, y) \in R$, $e_S = (y, z) \in S$ 。于是 $t_R(e_R) = y = s_S(e_S)$, 故 $(e_S, e_R) \in S \times_Y R$ 。

Step 6. 此二步路径的源端点是 x , 目标端点是 z 。所以 (x, z) 出现在二步路径的端点影子中。

Step 7. 两个方向合起来, 二步路径端点影子与 $S \circ R$ 完全相同。

□

这个命题揭示了一个重要区别: 关系复合 $S \circ R$ 只保留端点对 (x, z) , 而纤维积 $S \times_Y R$ 保留所有具体二步路径。若两个不同的中间点都连接 x 到 z , 关系复合只记录一次; 加权矩阵乘法却需要把这些路径的贡献全部加起来。因此后面的对应语言必须保留路径空间, 而不能只保留关系复合后的端点集合。

例 2.4: 两个中间点导致同一端点对

设

$$X = \{x\}, \quad Y = \{y_1, y_2\}, \quad Z = \{z\},$$

并令

$$R = \{(x, y_1), (x, y_2)\}, \quad S = \{(y_1, z), (y_2, z)\}.$$

则 $S \circ R = \{(x, z)\}$, 只含一个端点对。但是二步路径空间 $S \times_Y R$ 有两个元素, 对应两条路径

$$x \rightarrow y_1 \rightarrow z, \quad x \rightarrow y_2 \rightarrow z.$$

在矩阵乘法中, 这两条路径会给 (z, x) 项贡献两个求和项。若提前把它们压成单一关系, 就会丢掉 multiplicity。

2.9 三步路径与结合律的集合模型

后面证明对应复合结合律时，最重要的是识别左右两种加括号方式都描述同一个三步路径空间。本节先在无权重层面做这件事。

设有三个关系或事件空间

$$W \leftarrow R \rightarrow X, \quad X \leftarrow S \rightarrow Y, \quad Y \leftarrow T \rightarrow Z.$$

三步路径应当是三元组 (e_T, e_S, e_R) ，满足

$$s_S(e_S) = t_R(e_R), \quad s_T(e_T) = t_S(e_S).$$

如果先复合 R 与 S ，再与 T 复合，得到的集合可写为

$$T \times_Y (S \times_X R).$$

如果先复合 S 与 T ，再与 R 复合，得到的集合可写为

$$(T \times_Y S) \times_X R.$$

这两个集合在括号上不同，但它们包含同样的三步路径数据。

命题 2.6: 三步纤维积的典范双射

在上述记号下，存在自然双射

$$T \times_Y (S \times_X R) \cong (T \times_Y S) \times_X R.$$

该双射把

$$(e_T, (e_S, e_R))$$

送到

$$((e_T, e_S), e_R).$$

证明.

Step 1. 左边元素是 $(e_T, (e_S, e_R))$ ，其中 $(e_S, e_R) \in S \times_X R$ 且 e_T 可以接在 (e_S, e_R) 后面。

Step 2. 条件 $(e_S, e_R) \in S \times_X R$ 等价于

$$s_S(e_S) = t_R(e_R).$$

Step 3. e_T 可以接在 (e_S, e_R) 后面，等价于

$$s_T(e_T) = t_S(e_S).$$

Step 4. 因此左边元素等价于三元组 (e_T, e_S, e_R) ，满足两个匹配条件

$$s_T(e_T) = t_S(e_S), \quad s_S(e_S) = t_R(e_R).$$

Step 5. 右边元素 $((e_T, e_S), e_R)$ 同样等价于三元组 (e_T, e_S, e_R) ，并满足同样两个匹配条件。第一个条件使 $(e_T, e_S) \in T \times_Y S$ ，第二个条件使该二步路径可以接到 e_R 后面。

Step 6. 因此映射

$$(e_T, (e_S, e_R)) \mapsto ((e_T, e_S), e_R)$$

是良定义的。

Step 7. 其逆映射为

$$((e_T, e_S), e_R) \mapsto (e_T, (e_S, e_R)).$$

两者互为逆映射，所以得到双射。

□

说明

这条双射是后面“对应复合结合律”的真正原因。数的加法和乘法当然需要满足结合律与分配律，但路径层面的括号无关性首先来自这个集合双射。对应计算强调的是：先弄清楚路径空间，再处理路径权重。

2.10 从关系到对应：为什么需要事件空间

普通关系 $R \subseteq X \times Y$ 把一条边等同于一个端点对 (x, y) 。这在很多基础问题中够用，但在对应计算中不够。

原因有三点。

第一，多个不同机制可以连接同一个端点对。比如从 x 到 y 的连接可能来自局部卷积、跳连、注意力边或硬件路由边。它们端点相同，但来源不同。

第二，路径复合会产生 multiplicity。即使初始关系是简单的，复合之后从 x 到 z 的三步路径可能有很多条。矩阵乘法中的求和正是在统计这些不同路径的贡献。

第三，芯片执行时需要记录事件类型、时间戳、路由端口、量化格式等附加信息。这些信息无法放进端点对 (x, y) 本身。

因此，我们从关系升级到一般事件空间。

定义 2.14: 无权对应

设 X, Y 是有限集合。一个从 X 到 Y 的无权对应是一个图式

$$X \xleftarrow{s} E \xrightarrow{t} Y,$$

其中 E 是有限集合， $s: E \rightarrow X$ 与 $t: E \rightarrow Y$ 是映射。集合 E 称为事件空间，元素 $e \in E$ 称为事件。

无权对应只记录事件以及事件的两个端点，还不记录数值权重。下一章会给每个事件加入权重 $w(e)$ ，从而得到加权对应。

定义 2.15: 端点影子

设 $X \xleftarrow{s} E \xrightarrow{t} Y$ 是无权对应。它的端点影子是关系

$$R_E = \{(s(e), t(e)) : e \in E\} \subseteq X \times Y.$$

端点影子丢弃事件的 multiplicity。若有两个不同事件 e_1, e_2 具有相同端点，则它们在 R_E 中只留下同一个端点对。

例 2.5: 端点影子会丢失 multiplicity

令 $X = \{x\}$, $Y = \{y\}$, $E = \{e_1, e_2\}$, 并令

$$s(e_1) = s(e_2) = x, \quad t(e_1) = t(e_2) = y.$$

这是一个含两个事件的对应。但端点影子为

$$R_E = \{(x, y)\},$$

只含一个关系边。因此端点影子不能恢复原事件空间。

硬件解释

在硬件原型中，事件空间 E 对应 event memory 中的记录。端点影子只相当于记录某两个端点之间是否有连接；它无法记录多条 packet、多种算子来源、多种精度格式或多种路由路径。因此 CoPU 的存储单位应当是事件，而不是矩阵坐标。

2.11 本章小结

本章建立了后续理论所需的无权基础。主要结论可以概括为以下几点。

第一，对象空间是有限集合；坐标只是对有限集合的编号。数据是定义在对象空间上的函数，而不是先验给定的列向量。

第二，映射的纤维提供了分桶语言。沿映射的推前就是沿纤维聚合：

$$(f_! a)(y) = \sum_{e \in f^{-1}(y)} a(e).$$

第三，关系可以写成

$$X \leftarrow R \rightarrow Y,$$

它是最简单的无权对应。关系复合的真实路径空间是纤维积，而不仅是复合后的端点集合。

第四，纤维积的大小公式

$$|A \times_Y B| = \sum_{y \in Y} |f^{-1}(y)| |g^{-1}(y)|$$

已经给出了对应计算的第一条成本公式。后续的硬件复杂度不应只看矩阵大小，而应看中间纤维负载。

第五，一般对应允许多个事件具有相同端点。这是替代矩阵语言的关键一步：矩阵坐标只记录端点对；对应语言记录生成端点对的事件。

下一章开始引入权重半环。加上权重之后，二步路径不再只是存在或不存在，而会携带路径权重；沿纤维的存在性聚合将升级为代数求和，矩阵乘法也将在这一层自然出现。

半环与权重

上一章只处理了一个问题：某个输入对象是否与某个输出对象相连。这样的语言足以描述关系复合，却还不能描述矩阵。矩阵条目不是简单的是或否，而是一个数；矩阵乘法也不仅仅判断二步路径是否存在，而是把每条二步路径的权重相乘，然后把所有中间点给出的贡献相加。

本章的目标是把这一点从实数矩阵中抽出来。我们不急于定义加权对应，而先问：如果只保留矩阵乘法真正用到的代数规则，最小需要什么结构？答案是半环。半环同时提供两种操作：一种用于同一路径上的权重合成，另一种用于同一纤维内不同路径的权重聚合。后面的对应复合、路径计数、最短路、布尔可达性、卷积和芯片中的聚合器都会建立在这一层之上。

说明

本章的逻辑顺序如下。先从有限聚合需要的交换幺半群讲起，再加入路径合成需要的乘法幺半群；随后用分配律解释为什么“先按路径合成再按纤维聚合”可以稳定地组成一种乘法；最后把半环矩阵看成作用在对象数据上的核算子。这样到下一章时，加权对应只需要把矩阵的坐标表替换为事件空间即可。

3.1 从路径权重到代数规则

设有两层关系

$$X \leftarrow E_A \rightarrow Y, \quad Y \leftarrow E_B \rightarrow Z.$$

若 e_A 把 x 连到 y ， e_B 把同一个 y 连到 z ，则二者组成一条二步路径

$$x \xrightarrow{e_A} y \xrightarrow{e_B} z.$$

若每条事件 e 带有权重 $w(e)$ ，那么这条二步路径的权重应当由 $w(e_A)$ 与 $w(e_B)$ 合成。另一方面，从 x 到 z 可能有许多条二步路径；它们的贡献应当再被聚合成一个总权重。

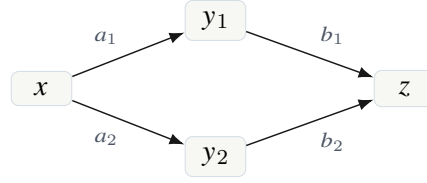
于是矩阵乘法背后有两个不同的动作：

沿同一条路径合成权重， 沿同一纤维聚合不同路径。

普通实数矩阵中，这两个动作分别是乘法和加法：

$$B_{zy}A_{yx}, \quad \sum_{y \in Y} B_{zy}A_{yx}.$$

但这并不是唯一可能的语义。若权重表示真假，合成应为“且”，聚合应为“或”；若权重表示路径代价，合成应为代价相加，聚合应为取最小。因此我们不应过早把权重固定为实数。



路径权重: $b_1 \otimes a_1, b_2 \otimes a_2$
 总权重: $(b_1 \otimes a_1) \oplus (b_2 \otimes a_2)$

图 3.1 路径合成与纤维聚合是两个不同的代数动作。

定义 3.1: 路径合成与纤维聚合的原始要求

设 $\mathbb{K}$ 是权重集合。为了把路径复合变成稳定的计算规则，我们至少需要：

1. 一个聚合运算 $\oplus$ ，用于把同一输出纤维中的多条贡献合并；
2. 一个合成运算 $\otimes$ ，用于把同一路径上的连续权重合成；
3. 一个零权重 0 ，表示没有贡献；
4. 一个单位权重 1 ，表示长度为零的恒等路径不改变权重；
5. 分配律，保证合成可以穿过聚合。

满足这些要求的结构正是半环。

这里的第五条最容易被忽略。矩阵乘法的结合律并不是凭空成立的。若没有分配律，三段路径的总权重就可能依赖我们先把哪两段合起来。这会破坏复合的结合律，从而使“算子”这个概念不稳定。

3.2 有限聚合：交换幺半群

半环的加法部分不是为了模仿普通加法，而是为了允许我们对一个有限纤维中的贡献做无歧义的聚合。由于纤维中的路径通常没有自然顺序，聚合结果不应依赖枚举顺序。因此加法首先应当交换、结合，并有一个空聚合的值。

定义 3.2: 交换幺半群

一个交换幺半群是一个集合 M ，配备一个二元运算 $\oplus : M \times M \rightarrow M$ 和一个元素 $0 \in M$ ，满足对任意 $a, b, c \in M$ ：

1. 结合律: $(a \oplus b) \oplus c = a \oplus (b \oplus c)$;
2. 交换律: $a \oplus b = b \oplus a$;
3. 单位元: $a \oplus 0 = 0 \oplus a = a$ 。

元素 0 称为加法单位元或零元素。

这个定义的直接结果是，有限多个元素的聚合是良定义的。换言之，只要知道有限多重集 $\{a_i : i \in I\}$ ，就可以写

$$\bigoplus_{i \in I} a_i$$

而不必指定 I 的排列方式。

命题 3.1: 有限聚合的良好定义性

设 $(M, \oplus, 0)$ 是交换么半群。对任意有限集合 I 与任意函数 $a : I \rightarrow M$ ，聚合

$$\bigoplus_{i \in I} a_i$$

可以定义为任意枚举下的迭代运算，且结果与枚举顺序无关。若 $I = \emptyset$ ，约定

$$\bigoplus_{i \in \emptyset} a_i = 0.$$

证明.

Step 1. 当 $I = \emptyset$ 时，没有任何贡献需要聚合。若希望以后公式

$$b \oplus \bigoplus_{i \in I} a_i$$

在 I 为空时仍等于 b ，唯一合理的约定就是空聚合等于单位元 0 。

Step 2. 当 I 非空，取一个枚举 $I = \{i_1, \dots, i_n\}$ ，定义

$$\bigoplus_{k=1}^n a_{i_k} = (((a_{i_1} \oplus a_{i_2}) \oplus a_{i_3}) \cdots) \oplus a_{i_n}.$$

结合律说明括号位置不会改变结果。

Step 3. 任意两个枚举之间可以通过有限次相邻交换相互得到。交换律说明每次交换相邻两项不改变结果。因此结果与枚举无关。

Step 4. 综上，有限集合上的聚合只依赖函数 $a : I \rightarrow M$ ，不依赖任何人为选择的顺序。

□

注记

矩阵语言通常把求和符号写成 $\sum_j$ ，默认使用实数加法。但在对应语言中，求和真正表达的是“沿纤维聚合”。因此符号 $\bigoplus$ 比 $\sum$ 更适合本讲义：它提醒我们聚合可以是加法、逻辑或、最大值、最小值，或者其他满足交换么半群公理的操作。

3.3 路径合成：么半群与单位路径

聚合只处理“多条路径如何汇总”。路径本身还需要一种合成规则。若一条路径先获得权重 a ，再获得权重 b ，则整条路径的权重记为 $b \otimes a$ 。这里写成 $b \otimes a$ 是为了与矩阵乘法顺序一致：先作用 A ，后作用 B ，复合为 BA 。

定义 3.3: 么半群

一个么半群是一个集合 M ，配备二元运算 $\otimes : M \times M \rightarrow M$ 和元素 $1 \in M$ ，满足：

1. 结合律： $(a \otimes b) \otimes c = a \otimes (b \otimes c)$ ；
2. 单位元： $a \otimes 1 = 1 \otimes a = a$ 。

若还满足 $a \otimes b = b \otimes a$ ，则称为交换么半群。

在路径语言中，乘法结合律的含义很具体。三段路径

$$x \rightarrow y \rightarrow z \rightarrow u$$

可以先把前两段合成，也可以先把后两段合成；最终整条三段路径的权重应相同。单位元 1 则对应长度为零的恒等路径。若 x 保持为 x 而没有走任何事件，它对权重的影响应为 1。

例 3.1: 路径代价与概率的乘法语义

不同权重语义给出不同的 $\otimes$ 。

1. 若 $w(e)$ 是概率或强度，连续事件的权重常用普通乘法合成。
2. 若 $w(e)$ 是代价或能量，连续事件的总代价通常是普通加法。因此在最短路语义中， $\otimes$ 实际上是 $+$ 。
3. 若 $w(e)$ 是真假值，连续事件同时成立才构成有效路径。因此在布尔语义中， $\otimes$ 是逻辑与。

3.4 半环的定义

现在把聚合结构和路径合成结构放在同一个集合上，并要求二者相容。

定义 3.4: 半环

一个半环是一个集合 $\mathbb{K}$ ，配备两个二元运算 $\oplus$ 与 $\otimes$ ，以及两个特殊元素 $0, 1 \in \mathbb{K}$ ，满足：

1. $(\mathbb{K}, \oplus, 0)$ 是交换幺半群；
2. $(\mathbb{K}, \otimes, 1)$ 是幺半群；
3. 左分配律与右分配律成立：

$$a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c),$$

$$(a \oplus b) \otimes c = (a \otimes c) \oplus (b \otimes c);$$

4. 0 吸收乘法：

$$0 \otimes a = a \otimes 0 = 0.$$

若 $a \otimes b = b \otimes a$ 对所有 $a, b \in \mathbb{K}$ 成立，则称 $\mathbb{K}$ 为交换半环。

本讲义中，若无特别说明，所有纤维都是有限的，因此不需要处理无限求和。若以后考虑可数状态空间或连续状态空间，就必须另外加入完备半环、测度或积分结构。

命题 3.2: 有限分配律

设 $\mathbb{K}$ 是半环， I 是有限集合， $a_i \in \mathbb{K}$ ， $b, c \in \mathbb{K}$ 。则

$$b \otimes \left(\bigoplus_{i \in I} a_i \right) = \bigoplus_{i \in I} (b \otimes a_i),$$

$$\left(\bigoplus_{i \in I} a_i \right) \otimes c = \bigoplus_{i \in I} (a_i \otimes c).$$

证明.

Step 1. 只证明第一式；第二式完全类似。

Step 2. 若 $I = \emptyset$ ，左边为 $b \otimes 0 = 0$ ，右边为空聚合，也等于 0。所以命题成立。

Step 3. 若 $|I| = 1$ ，结论显然成立。

Step 4. 假设命题对大小为 n 的有限集合成立。设 $I = J \sqcup \{i_0\}$ ，其中 $|J| = n$ 。则

$$b \otimes \left(\bigoplus_{i \in I} a_i \right) = b \otimes \left(\left(\bigoplus_{i \in J} a_i \right) \oplus a_{i_0} \right).$$

Step 5. 由二元分配律，

$$= \left(b \otimes \bigoplus_{i \in J} a_i \right) \oplus (b \otimes a_{i_0}).$$

由归纳假设，第一项等于 $\bigoplus_{i \in J} (b \otimes a_i)$ 。于是得到

$$= \bigoplus_{i \in I} (b \otimes a_i).$$

Step 6. 归纳完成。

□

3.5 基本例子

半环抽象之所以自然，是因为许多常见计算共享同一个形式，但权重语义不同。

例 3.2: 数值半环

以下是最接近通常矩阵计算的半环：

1. 自然数半环 $(\mathbb{N}, +, \cdot, 0, 1)$ 。它适合计数路径条数。
2. 非负实数半环 $(\mathbb{R}_{\geq 0}, +, \cdot, 0, 1)$ 。它适合概率质量、强度和非负权重。
3. 实数半环 $(\mathbb{R}, +, \cdot, 0, 1)$ 。它是普通实矩阵计算使用的结构。
4. 复数半环 $(\mathbb{C}, +, \cdot, 0, 1)$ 。它用于频域、量子振幅等含相位的计算。

这些例子实际上都是环或域，但半环理论并不要求加法逆元存在。

例 3.3: 布尔半环

集合 $\mathbb{B} = \{0, 1\}$ 上取

$$a \oplus b = a \vee b, \quad a \otimes b = a \wedge b.$$

加法单位元为 0，乘法单位元为 1。于是

$$\bigoplus_i a_i$$

表示“至少有一个 a_i 为真”，而

$$b \otimes a$$

表示两个条件同时为真。关系复合正是布尔半环矩阵乘法。

例 3.4: min-plus 半环

在集合 $\mathbb{R} \cup \{\infty\}$ 上定义

$$a \oplus b = \min\{a, b\}, \quad a \otimes b = a + b.$$

加法单位元是 ∞ ，因为 $\min\{a, \infty\} = a$ ；乘法单位元是 0，因为 $a + 0 = a$ 。吸收律为

$$\infty + a = a + \infty = \infty.$$

因此 min-plus 半环把“路径合成”解释为代价相加，把“路径聚合”解释为选择最小代价。

例 3.5: max-plus 半环

在 $\mathbb{R} \cup \{-\infty\}$ 上定义

$$a \oplus b = \max\{a, b\}, \quad a \otimes b = a + b.$$

加法单位元是 $-\infty$ ，乘法单位元是 0。该半环常用于最长路、动态规划和离散事件系统。

例 3.6: 有限精度权重的注意事项

芯片中常用定点数、饱和加法或量化乘法。并非所有有限精度运算都严格构成半环。例如，带舍入的浮点加法一般不严格结合，饱和乘法也可能破坏分配律。因此在数学层面，我们先使用理想半环；在硬件层面，再把有限精度误差作为近似实现分析。若选择模整数 $\mathbb{Z}/q\mathbb{Z}$ ，则可以得到严格的有限环，也因而得到严格的半环。

语义	集合 $\mathbb{K}$	$\oplus$: 纤维聚合	$\otimes$: 路径合成
数值线性代数	$\mathbb{R}$	加法	乘法
路径计数	$\mathbb{N}$	加法	乘法
可达性	$\{0, 1\}$	或	且
最短路	$\mathbb{R} \cup \{\infty\}$	最小值	加法
最长路/调度	$\mathbb{R} \cup \{-\infty\}$	最大值	加法

表 3.1 同一对应复合形式在不同半环中的语义。

3.6 对象上的数据与半模

为了把矩阵看成算子，需要先说明“对象空间上的数据”是什么。若 X 是有限集合，则一个 $\mathbb{K}$ -值数据就是函数

$$u : X \rightarrow \mathbb{K}.$$

可以把 $u(x)$ 理解为对象 x 上的质量、激活、真假状态、代价或计数。

定义 3.5: 对象空间上的自由半模

设 X 是有限集合， $\mathbb{K}$ 是半环。记

$$\mathbb{K}^X = \{u : X \rightarrow \mathbb{K}\}.$$

对 $u, v \in \mathbb{K}^X$ 与 $a \in \mathbb{K}$, 定义

$$(u \oplus v)(x) = u(x) \oplus v(x),$$

$$(a \otimes u)(x) = a \otimes u(x).$$

这个集合称为由 X 生成的自由 $\mathbb{K}$ -半模。

这里使用“半模”而不是“向量空间”，是因为 $\mathbb{K}$ 可能没有加法逆元。例如布尔半环上的 $\mathbb{K}^X$ 不是向量空间，但它仍然可以承载关系、可达性与图传播计算。

定义 3.6: 标准基函数

对每个 $x \in X$, 定义函数 $\delta_x \in \mathbb{K}^X$ 为

$$\delta_x(x') = \begin{cases} 1, & x' = x, \\ 0, & x' \neq x. \end{cases}$$

称 δ_x 为 x 对应的标准基函数。

命题 3.3: 有限展开公式

任意 $u \in \mathbb{K}^X$ 都可写成

$$u = \bigoplus_{x \in X} u(x) \otimes \delta_x.$$

更具体地，对每个 $x_0 \in X$, 右边在 x_0 处的值等于 $u(x_0)$ 。

证明.

Step 1. 固定 $x_0 \in X$. 右边在 x_0 处的值为

$$\bigoplus_{x \in X} u(x) \otimes \delta_x(x_0).$$

Step 2. 当 $x \neq x_0$ 时, $\delta_x(x_0) = 0$, 因此

$$u(x) \otimes \delta_x(x_0) = u(x) \otimes 0 = 0.$$

Step 3. 当 $x = x_0$ 时, $\delta_{x_0}(x_0) = 1$, 因此

$$u(x_0) \otimes 1 = u(x_0).$$

Step 4. 整个聚合中只有 $x = x_0$ 的项可能非零, 其余项都是 0。由 0 是 $\oplus$ 的单位元, 结果为 $u(x_0)$ 。

□

注记

这个命题是通常向量展开 $u = \sum_x u_x e_x$ 的半环版本。它没有使用减法, 也没有使用内积, 只使用有限聚合、乘法单位元和零元素吸收律。

3.7 沿映射的拉回与推前

上一章已经出现了纤维 $f^{-1}(y)$ 。现在当对象上有权重数据时, 一个映射 $f : X \rightarrow Y$ 会诱导两种基本操作: 沿 f 拉回数据, 以及沿 f 对纤维求和推前数据。

定义 3.7: 拉回

设 $f: X \rightarrow Y$ 是映射。对 $v \in \mathbb{K}^Y$ ，定义 $f^*v \in \mathbb{K}^X$ 为

$$(f^*v)(x) = v(f(x)).$$

称 f^* 为沿 f 的拉回。

拉回只是把 Y 上的数据复制到 X 上：每个 x 读取其像 $f(x)$ 上的值。

定义 3.8: 推前

设 $f: X \rightarrow Y$ 是映射，且 X 有限。对 $u \in \mathbb{K}^X$ ，定义 $f_!u \in \mathbb{K}^Y$ 为

$$(f_!u)(y) = \bigoplus_{x \in f^{-1}(y)} u(x).$$

称 $f_!$ 为沿 f 的推前，或沿纤维的聚合。

推前就是把同一纤维中的权重合并到基点 y 上。若某个纤维为空，聚合值为 0。

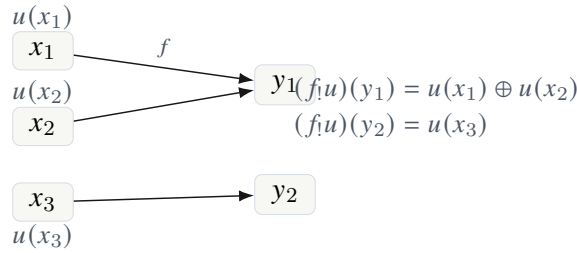


图 3.2 推前是沿映射纤维的聚合。

命题 3.4: 推前的加法线性性

对任意映射 $f: X \rightarrow Y$ 与 $u, v \in \mathbb{K}^X$ ，有

$$f_!(u \oplus v) = f_!u \oplus f_!v.$$

其中右边的 $\oplus$ 是 $\mathbb{K}^Y$ 上的逐点聚合。

证明.

Step 1. 固定 $y \in Y$ 。左边在 y 处的值为

$$(f_!(u \oplus v))(y) = \bigoplus_{x \in f^{-1}(y)} (u \oplus v)(x).$$

Step 2. 由逐点定义， $(u \oplus v)(x) = u(x) \oplus v(x)$ ，故

$$(f_!(u \oplus v))(y) = \bigoplus_{x \in f^{-1}(y)} (u(x) \oplus v(x)).$$

Step 3. 由于 $\oplus$ 交换且结合，一个有限集合上逐点两项的聚合可以重新排列为

$$\left(\bigoplus_{x \in f^{-1}(y)} u(x) \right) \oplus \left(\bigoplus_{x \in f^{-1}(y)} v(x) \right).$$

Step 4. 这正是

$$(f_!u)(y) \oplus (f_!v)(y) = (f_!u \oplus f_!v)(y).$$

Step 5. 因为对每个 y 都相等, 命题成立。 □

命题 3.5: 推前的合成律

若 $X \xrightarrow{f} Y \xrightarrow{g} Z$ 是有限集合之间的映射, 则

$$(g \circ f)_! = g_! \circ f_!.$$

证明.

Step 1. 固定 $z \in Z$ 。左边为

$$((g \circ f)_!u)(z) = \bigoplus_{x \in (g \circ f)^{-1}(z)} u(x).$$

Step 2. 注意到

$$(g \circ f)^{-1}(z) = \bigsqcup_{y \in g^{-1}(z)} f^{-1}(y),$$

其中右边是不交并。原因是: 若 $g(f(x)) = z$, 则 $y = f(x)$ 属于 $g^{-1}(z)$, 且 $x \in f^{-1}(y)$; 反过来也显然。

Step 3. 因此左边可以按中间纤维先分组:

$$((g \circ f)_!u)(z) = \bigoplus_{y \in g^{-1}(z)} \left(\bigoplus_{x \in f^{-1}(y)} u(x) \right).$$

Step 4. 内层聚合正是 $(f_!u)(y)$ 。于是

$$((g \circ f)_!u)(z) = \bigoplus_{y \in g^{-1}(z)} (f_!u)(y) = (g_!(f_!u))(z).$$

Step 5. 对所有 z 成立, 故 $(g \circ f)_! = g_! \circ f_!$ 。 □

命题 3.6: 拉回的合成律

若 $X \xrightarrow{f} Y \xrightarrow{g} Z$, 则

$$(g \circ f)^* = f^* \circ g^*.$$

证明.

Step 1. 取 $h \in \mathbb{K}^Z$ 与 $x \in X$ 。

Step 2. 左边为

$$((g \circ f)^*h)(x) = h(g(f(x))).$$

Step 3. 右边为

$$(f^*(g^*h))(x) = (g^*h)(f(x)) = h(g(f(x))).$$

Step 4. 二者在每个 x 上相等，故映射相等。

□

3.8 半环矩阵作为核算子

现在回到矩阵。设 $A \in \mathbb{K}^{Y \times X}$ ，即每个输入 $x \in X$ 与输出 $y \in Y$ 之间有一个权重 A_{yx} 。这个矩阵可以作用在 X 上的数据 $u \in \mathbb{K}^X$ 上，得到 Y 上的数据。

定义 3.9: 半环矩阵的作用

设 $A \in \mathbb{K}^{Y \times X}$ 。定义 $A_* : \mathbb{K}^X \rightarrow \mathbb{K}^Y$ 为

$$(A_*u)(y) = \bigoplus_{x \in X} A_{yx} \otimes u(x).$$

称 A_* 为由核 A 定义的半环线性算子。

这个公式的读法是：输入对象 x 上的质量 $u(x)$ 沿权重为 A_{yx} 的边传到 y ，然后所有到达同一个 y 的贡献被聚合。

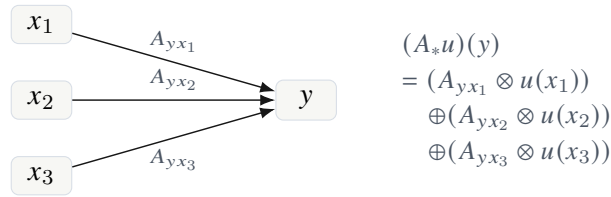


图 3.3 矩阵作用是沿输出纤维的加权推前。

定义 3.10: 半环矩阵乘法

设 $A \in \mathbb{K}^{Y \times X}$ ， $B \in \mathbb{K}^{Z \times Y}$ 。定义其半环乘积 $B \odot A \in \mathbb{K}^{Z \times X}$ 为

$$(B \odot A)_{zx} = \bigoplus_{y \in Y} B_{zy} \otimes A_{yx}.$$

当 $\mathbb{K} = \mathbb{R}$ 且 $\oplus = +$ 、 $\otimes = \cdot$ 时，这就是普通矩阵乘法。

定理 3.1: 矩阵乘法就是核算子的复合

对 $A \in \mathbb{K}^{Y \times X}$ 、 $B \in \mathbb{K}^{Z \times Y}$ ，有

$$(B \odot A)_* = B_* \circ A_*.$$

也就是说，半环矩阵乘法正是相应核算子的复合。

证明.

Step 1. 取任意 $u \in \mathbb{K}^X$ 与 $z \in Z$ 。左边为

$$((B \odot A)_*u)(z) = \bigoplus_{x \in X} (B \odot A)_{zx} \otimes u(x).$$

Step 2. 展开乘积矩阵的条目：

$$= \bigoplus_{x \in X} \left(\bigoplus_{y \in Y} B_{zy} \otimes A_{yx} \right) \otimes u(x).$$

Step 3. 由有限分配律与乘法结合律，可写成

$$= \bigoplus_{x \in X} \bigoplus_{y \in Y} B_{zy} \otimes A_{yx} \otimes u(x).$$

Step 4. 由于外层聚合是有限交换聚合，可以交换 x 与 y 的聚合顺序：

$$= \bigoplus_{y \in Y} B_{zy} \otimes \left(\bigoplus_{x \in X} A_{yx} \otimes u(x) \right).$$

Step 5. 括号内正是 $(A_* u)(y)$ ，所以

$$= \bigoplus_{y \in Y} B_{zy} \otimes (A_* u)(y) = (B_* (A_* u))(z).$$

Step 6. 对所有 u, z 成立，故 $(B \odot A)_* = B_* \circ A_*$

□

推论 3.1: 半环矩阵乘法的结合律

在任意半环 $\mathbb{K}$ 上，若矩阵维度匹配，则

$$C \odot (B \odot A) = (C \odot B) \odot A.$$

证明.

Step 1. 由上一定理，矩阵乘法对应核算子的复合。

Step 2. 算子复合满足结合律：

$$C_* \circ (B_* \circ A_*) = (C_* \circ B_*) \circ A_*.$$

Step 3. 因此 $C \odot (B \odot A)$ 与 $(C \odot B) \odot A$ 诱导同一个算子。

Step 4. 为了推出矩阵条目相等，只需把该算子作用在每个标准基函数 δ_x 上。对任意 $u \in Z$ ，矩阵条目可由

$$M_{ux} = (M_* \delta_x)(u)$$

恢复。

Step 5. 所以两个矩阵相等。

□

定义 3.11: 恒等矩阵

对有限集合 X ，定义 $I_X \in \mathbb{K}^{X \times X}$ 为

$$(I_X)_{x'x} = \begin{cases} 1, & x' = x, \\ 0, & x' \neq x. \end{cases}$$

称 I_X 为 X 上的恒等矩阵。

命题 3.7: 恒等矩阵的单位性质

对任意 $A \in \mathbb{K}^{Y \times X}$, 有

$$A \odot I_X = A, \quad I_Y \odot A = A.$$

证明.

Step 1. 先证 $A \odot I_X = A$ 。固定 $y \in Y$ 与 $x \in X$,

$$(A \odot I_X)_{yx} = \bigoplus_{x' \in X} A_{yx'} \otimes (I_X)_{x'x}.$$

Step 2. 当 $x' \neq x$ 时, $(I_X)_{x'x} = 0$, 相应项为 $A_{yx'} \otimes 0 = 0$ 。当 $x' = x$ 时, 相应项为 $A_{yx} \otimes 1 = A_{yx}$ 。

Step 3. 因此整个聚合等于 A_{yx} 。

Step 4. $I_Y \odot A = A$ 的证明相同:

$$(I_Y \odot A)_{yx} = \bigoplus_{y' \in Y} (I_Y)_{yy'} \otimes A_{y'x},$$

只有 $y' = y$ 的项不为零。

□

3.9 半环矩阵的路径解释

半环矩阵乘法可以直接读成路径语言。 A_{yx} 是从 x 到 y 的一步路径权重, B_{zy} 是从 y 到 z 的一步路径权重。于是

$$B_{zy} \otimes A_{yx}$$

是经过中间点 y 的二步路径权重, 而

$$\bigoplus_{y \in Y} B_{zy} \otimes A_{yx}$$

是所有二步路径的聚合。

命题 3.8: 三段路径的括号无关性

设 $A \in \mathbb{K}^{Y \times X}$, $B \in \mathbb{K}^{Z \times Y}$, $C \in \mathbb{K}^{U \times Z}$ 。从 $x \in X$ 到 $u \in U$ 的三段路径总权重等于

$$\bigoplus_{z \in Z} \bigoplus_{y \in Y} C_{uz} \otimes B_{zy} \otimes A_{yx},$$

无论先复合 A, B 还是先复合 B, C , 都得到同一表达式。

证明.

Step 1. 先复合 A, B , 再与 C 复合, 得到

$$(C \odot (B \odot A))_{ux} = \bigoplus_{z \in Z} C_{uz} \otimes (B \odot A)_{zx}.$$

Step 2. 展开内层乘积:

$$= \bigoplus_{z \in Z} C_{uz} \otimes \left(\bigoplus_{y \in Y} B_{zy} \otimes A_{yx} \right).$$

Step 3. 用有限分配律得到

$$= \bigoplus_{z \in Z} \bigoplus_{y \in Y} C_{uz} \otimes B_{zy} \otimes A_{yx}.$$

Step 4. 先复合 B, C , 再与 A 复合, 得到

$$((C \odot B) \odot A)_{ux} = \bigoplus_{y \in Y} \left(\bigoplus_{z \in Z} C_{uz} \otimes B_{zy} \right) \otimes A_{yx}.$$

Step 5. 再由有限分配律与乘法结合律, 得到

$$= \bigoplus_{y \in Y} \bigoplus_{z \in Z} C_{uz} \otimes B_{zy} \otimes A_{yx}.$$

Step 6. 因 $\oplus$ 交换且结合, 两个双重聚合相同。

□

3.10 权重语义的几个基本计算

例 3.7: 布尔半环给出关系复合

设 $A \in \mathbb{B}^{Y \times X}$ 与 $B \in \mathbb{B}^{Z \times Y}$ 是两个关系的布尔矩阵。则

$$(B \odot A)_{zx} = \bigvee_{y \in Y} (B_{zy} \wedge A_{yx}).$$

因此 $(B \odot A)_{zx} = 1$ 当且仅当存在 $y \in Y$, 使得 $A_{yx} = 1$ 且 $B_{zy} = 1$ 。这正是关系复合的定义。

例 3.8: 自然数半环给出路径条数

若 A_{yx} 表示从 x 到 y 的边数, B_{zy} 表示从 y 到 z 的边数, 则

$$(B \odot A)_{zx} = \sum_{y \in Y} B_{zy} A_{yx}$$

表示从 x 到 z 的二步路径总数。这里乘法计算经过固定 y 的组合数, 加法对所有中间点求和。

例 3.9: min-plus 半环给出最短二段路

若 A_{yx} 是从 x 到 y 的代价, B_{zy} 是从 y 到 z 的代价, 则

$$(B \odot A)_{zx} = \min_{y \in Y} \{B_{zy} + A_{yx}\}.$$

这表示所有二段路径中代价最小的一条。

例 3.10: 实数半环给出普通线性传播

若 $\mathbb{K} = \mathbb{R}$, 则

$$(A_* u)(y) = \sum_{x \in X} A_{yx} u(x)$$

就是普通矩阵向量乘法。此时输入 $u(x)$ 的贡献按边权 A_{yx} 缩放, 并在输出端相加。

3.11 硬件解释: composer 与 reducer

半环抽象也给出芯片结构上的分工。若一个计算核心要执行路径复合, 它不应只被理解为乘法器, 而应分为两个可替换部件: 路径合成器和纤维聚合器。

硬件解释

在 CoPU 中, $\otimes$ 对应 *weight composer*, 负责把连续事件的权重合成为路径权重; $\oplus$ 对应 *push reducer*, 负责把落入同一输出纤维的路径贡献聚合。普通 MAC 只是特殊情形: composer 是乘法器, reducer 是加法器。

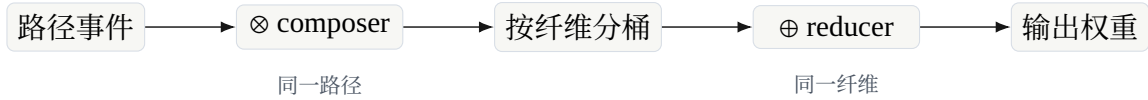


图 3.4 半环把硬件中的路径合成与纤维聚合分开。

这一区分很重要。若固定为浮点乘加, 硬件天然偏向普通线性代数; 若把 composer 与 reducer 参数化, 就能在同一结构上执行可达性、路径计数、最短路、动态规划和若干稀疏神经算子。

3.12 本章小结

本章完成了对应语言的代数底座。主要结论如下:

1. 有限纤维聚合需要交换么半群结构; 空纤维聚合对应零元素。
2. 路径权重合成需要么半群结构; 长度为零的路径对应单位元素。
3. 分配律保证路径合成与纤维聚合相容, 从而得到半环。
4. 半环矩阵作用在对象数据上, 矩阵乘法正是这些核算子的复合。
5. 不同半环给出不同语义: 数值线性代数、关系复合、路径计数和最短路都服从同一个公式。

下一章将在这个基础上正式定义加权对应。届时矩阵 $A \in \mathbb{K}^{Y \times X}$ 将被替换为一个带权事件空间

$$X \xleftarrow{s} E \xrightarrow{t} Y, \quad w : E \rightarrow \mathbb{K}.$$

矩阵条目只是把同一输入输出对之间的事件权重沿纤维聚合后的影子。

加权对应

前三章已经准备好了三个层次的材料。第一章说明矩阵条目可以看成端点之间的聚合权重，第二章建立了集合、纤维、关系、纤维积和沿映射推前，第三章把“沿路径合成”和“沿纤维聚合”抽象为半环运算。本章把这些材料合在一起，正式定义本讲义的基本对象：加权对应。

矩阵语言把一个算子写成坐标表

$$A = (A_{yx})_{y \in Y, x \in X}.$$

这个写法当然有效，但它已经做了一次压缩：所有从 x 到 y 的贡献都被合并到单个数 A_{yx} 中。若我们只关心最终的线性变换，这种压缩没有问题；若我们关心计算如何生成、路径如何复合、结构如何保留、硬件如何索引，那么这种压缩就太早了。加权对应的目的，是在聚合之前保留事件空间。

说明

本章的基本问题是：如何在不展开成矩阵坐标表的前提下，描述一个从输入对象空间 X 到输出对象空间 Y 的加权计算？答案是保存三件事：事件空间 E 、端点映射 $s : E \rightarrow X$ 与 $t : E \rightarrow Y$ 、权重函数 $w : E \rightarrow \mathbb{K}$ 。矩阵只是在最后把同端点事件沿纤维聚合之后得到的影子。

4.1 从矩阵条目到事件

设 X 是输入索引集合， Y 是输出索引集合。一个矩阵条目 A_{yx} 常被读作“从 x 到 y 的权重”。这种读法已经暗示了一个图像：存在某种从 x 指向 y 的连接。但矩阵条目本身没有告诉我们这条连接从何而来。

例如，在卷积中，从像素 x 到像素 y 的连接不是任意存在的，而是由一个局部偏移 r 决定：

$$y = x + r.$$

若使用矩阵表示，就必须把所有 (x, y) 坐标条目写出来，或者至少把非零条目列出来。但真正的生成机制不是 (x, y) ，而是 (x, r) 。这里的 r 是事件的类型，也是连接的来源。把卷积写成矩阵会把 r 消掉，只留下 x 与 y 。

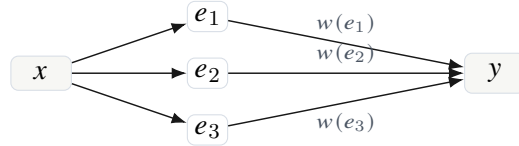
图消息传递中也有同样现象。邻接矩阵只记录两个顶点是否相连，或边权是多少。但一条边可能有类型、方向、时间戳、来源传感器、可信度，甚至是多个平行边。若直接落成矩阵条目，这些信息会在聚合中消失。

定义 4.1: 事件

在本讲义中, 事件是一次从某个源对象指向某个目标对象的基本贡献。一个事件 e 至少携带三类信息:

源端点 $s(e)$, 目标端点 $t(e)$, 权重 $w(e)$.

事件还可以带有额外结构, 例如类型、时间、几何位置、生成规则或硬件路由标签。本章先只把源端点、目标端点和权重放进定义, 其余结构以后可以作为事件空间 E 上的附加结构加入。



矩阵条目只看到 $A_{yx} = w(e_1) \oplus w(e_2) \oplus w(e_3)$;
事件表示保留了三条贡献的区别。

图 4.1 同一端点对之间可以有多个事件。矩阵影子会把它们聚合为一个条目。

注记

这里的“事件”不是概率论或操作系统中的特殊术语, 而是一个中性词: 它表示一次可被计数、可被赋权、可被复合的基本贡献。矩阵语言通常从已经聚合好的数开始; 对应语言从这些数的组成单位开始。

4.2 加权对应的定义

现在正式给出定义。为了避免无限求和带来的技术问题, 本讲义的基本版本先限制在有限集合上。后续若要处理连续空间, 可以把有限集合替换为可测空间, 把有限聚合替换为积分或核算子。

定义 4.2: 加权对应

设 X, Y 是有限集合, $\mathbb{K}$ 是半环。一个从 X 到 Y 的 $\mathbb{K}$ -加权对应是四元组

$$\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, w_A),$$

其中:

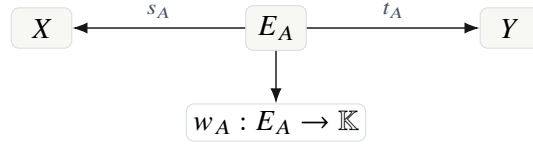
1. E_A 是有限集合, 称为事件空间;
2. $s_A : E_A \rightarrow X$ 是源映射;
3. $t_A : E_A \rightarrow Y$ 是目标映射;
4. $w_A : E_A \rightarrow \mathbb{K}$ 是权重函数。

若上下文清楚, 常写作 $\mathcal{A} : X \rightsquigarrow Y$ 。源映射和目标映射有时合称为端点映射。

定义中的箭头

$$X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y$$

应当从中间读起。事件 $e \in E_A$ 通过 s_A 选择输入端点，通过 t_A 选择输出端点。箭头方向画成从 E_A 指向 X, Y ，是因为 s_A, t_A 是函数；但计算方向通常读作从 X 经由事件到 Y 。



一个加权对应不是一张表，而是一个带端点映射和权重函数的事件空间。

图 4.2 加权对应的四个组成部分。

边界

不要把 E_A 自动理解为 $X \times Y$ 的子集。这样做会把对应语言重新压回矩阵语言。 E_A 可以等于边集合、程序依赖集合、卷积局部偏移集合、注意力候选集合、硬件路由事件集合，或者任何产生输入输出贡献的有限集合。

例 4.1: 函数作为对应

若 $f : X \rightarrow Y$ 是普通函数，则可把 f 看成一个加权对应。取

$$E_f = X, \quad s_f(x) = x, \quad t_f(x) = f(x), \quad w_f(x) = 1.$$

于是每个输入 x 产生唯一事件，这个事件从 x 到 $f(x)$ ，权重为单位元 $1 \in \mathbb{K}$ 。这说明函数是对应的一种极端情形：每个源纤维恰有一个单位权重事件。

例 4.2: 关系作为布尔加权对应

设 $R \subseteq X \times Y$ 是一个关系。令 $E_R = R$ ，并定义

$$s_R(x, y) = x, \quad t_R(x, y) = y, \quad w_R(x, y) = 1 \in \mathbb{B}.$$

其中 $\mathbb{B} = \{0, 1\}$ 是布尔半环。这样得到一个布尔加权对应。关系复合就是这种对应复合在布尔半环下的影子。

例 4.3: 矩阵作为加权对应

给定矩阵 $A \in \mathbb{K}^{Y \times X}$ 。最直接的对应表示是取

$$E_A = X \times Y, \quad s_A(x, y) = x, \quad t_A(x, y) = y, \quad w_A(x, y) = A_{yx}.$$

若希望避免零权重事件，也可以取非零支撑

$$E_A = \{(x, y) \in X \times Y : A_{yx} \neq 0\}.$$

这两种表示在多数半环上具有相同的矩阵影子；但作为事件空间，它们并不完全相同，因为第一种保留零事件，第二种删除零事件。

注记

如果目标只是恢复矩阵值, 删除零权重事件通常无害; 如果目标是记录某种结构性占位, 例如硬件连线位置、静态可路由槽位或潜在可激活边, 那么零事件也可能有意义。因此“零权重事件是否保留”不是纯数学问题, 而取决于我们想保留什么结构。

4.3 源纤维与目标纤维

对应语言的第一个基本分析工具是纤维。第二章已经对一般映射 $f: E \rightarrow X$ 定义了纤维; 现在把它应用到源映射和目标映射上。

定义 4.3: 源纤维与目标纤维

设

$$\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, w_A)$$

是加权对应。对 $x \in X$, 称

$$E_A(x, -) := s_A^{-1}(x) = \{e \in E_A : s_A(e) = x\}$$

为 x 的源纤维。对 $y \in Y$, 称

$$E_A(-, y) := t_A^{-1}(y) = \{e \in E_A : t_A(e) = y\}$$

为 y 的目标纤维。对端点对 (x, y) , 称

$$E_A(x, y) := \{e \in E_A : s_A(e) = x, t_A(e) = y\}$$

为 (x, y) 上的双端点纤维。

这三个记号分别表示三种不同的局部观察方式:

$E_A(x, -)$ 看从 x 发出的所有事件,

$E_A(-, y)$ 看汇入 y 的所有事件,

$E_A(x, y)$ 看从 x 到 y 的所有平行事件。

矩阵只有第三种纤维聚合后的结果。对应语言保留三种纤维本身。

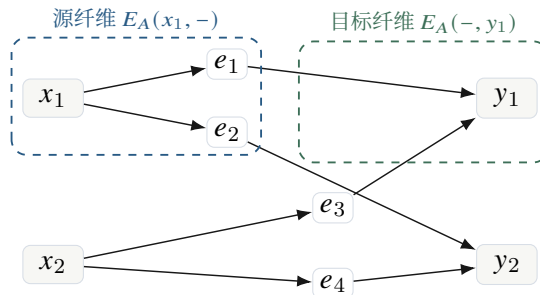


图 4.3 源纤维按输入端点分桶, 目标纤维按输出端点分桶。

引理 4.1: 事件空间的三种分解

设 $\mathcal{A} : X \rightsquigarrow Y$ 是有限加权对应。则

$$E_A = \bigsqcup_{x \in X} E_A(x, -) = \bigsqcup_{y \in Y} E_A(-, y) = \bigsqcup_{(x, y) \in X \times Y} E_A(x, y).$$

因此

$$|E_A| = \sum_{x \in X} |E_A(x, -)| = \sum_{y \in Y} |E_A(-, y)| = \sum_{(x, y) \in X \times Y} |E_A(x, y)|.$$

证明.

Step 1. 对源映射 $s_A : E_A \rightarrow X$ 应用纤维分解, 得到

$$E_A = \bigsqcup_{x \in X} s_A^{-1}(x) = \bigsqcup_{x \in X} E_A(x, -).$$

这是因为每个事件 e 有唯一源端点 $s_A(e)$ 。

Step 2. 对目标映射 $t_A : E_A \rightarrow Y$ 应用同样论证, 得到

$$E_A = \bigsqcup_{y \in Y} t_A^{-1}(y) = \bigsqcup_{y \in Y} E_A(-, y).$$

Step 3. 定义端点对映射

$$(s_A, t_A) : E_A \rightarrow X \times Y, \quad e \mapsto (s_A(e), t_A(e)).$$

其在 (x, y) 上的纤维正是

$$(s_A, t_A)^{-1}(x, y) = E_A(x, y).$$

再次应用纤维分解, 得到双端点纤维分解。

Step 4. 由于三种分解都是不交并, 对有限集合取基数即可得到三个基数公式。

□

硬件解释

在硬件实现中, 源纤维索引用于快速枚举从某个输入状态发出的事件, 目标纤维索引用于快速聚合落到同一个输出状态的贡献, 双端点纤维索引用于检测平行事件并生成矩阵影子。CoPU 的事件存储不是单纯数组, 而是带有这些纤维索引的事件数据库。

4.4 对应对对象数据的作用

加权对应不仅是静态结构, 也可以作用在对象空间上的数据。设 $u : X \rightarrow \mathbb{K}$ 是输入数据, $u(x)$ 表示输入对象 x 上的权重、激活、质量或信号强度。对应 $\mathcal{A} : X \rightsquigarrow Y$ 应当把 u 传到 Y 上的输出数据。

沿一个事件 e , 输入值 $u(s_A(e))$ 被事件权重 $w_A(e)$ 调制, 得到贡献

$$w_A(e) \otimes u(s_A(e)).$$

落到同一个 y 的所有事件贡献再沿目标纤维聚合。因此定义

$$(\mathcal{A}_! u)(y) = \bigoplus_{e \in E_A(-, y)} w_A(e) \otimes u(s_A(e)).$$

这里下标！提醒我们这是沿目标映射的推前式聚合。

定义 4.4: 加权对应诱导的算子

设 $\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, w_A)$ 是 $\mathbb{K}$ -加权对应。定义算子

$$\mathcal{A}_! : \mathbb{K}^X \rightarrow \mathbb{K}^Y$$

为

$$(\mathcal{A}_! u)(y) = \bigoplus_{e \in t_A^{-1}(y)} w_A(e) \otimes u(s_A(e)).$$

若目标纤维为空，则右端为空聚合，值为 0。

这个公式可以分解成三步：先沿源映射拉回输入数据，再逐点乘以事件权重，最后沿目标映射推前。即

$$\mathbb{K}^X \xrightarrow{s_A^*} \mathbb{K}^{E_A} \xrightarrow{w_A \otimes -} \mathbb{K}^{E_A} \xrightarrow{(t_A)_!} \mathbb{K}^Y.$$

更具体地，

$$\mathcal{A}_! = (t_A)_! \circ M_{w_A} \circ s_A^*,$$

其中 M_{w_A} 是在事件空间上逐点左乘 w_A 的算子。



先把输入数据复制到事件上，再用事件权重调制，最后按目标纤维聚合。

图 4.4 对应作用的三步分解：拉回、加权、推前。

命题 4.1: 对应算子的线性性

设 $\mathbb{K}$ 是半环, $\mathcal{A} : X \rightsquigarrow Y$ 是 $\mathbb{K}$ -加权对应。则 $\mathcal{A}_! : \mathbb{K}^X \rightarrow \mathbb{K}^Y$ 保持有限聚合：对任意 $u, v \in \mathbb{K}^X$,

$$\mathcal{A}_!(u \oplus v) = \mathcal{A}_! u \oplus \mathcal{A}_! v,$$

其中左边 $u \oplus v$ 与右边的 $\oplus$ 都逐点定义。并且

$$\mathcal{A}_!(0) = 0.$$

证明.

Step 1. 固定 $y \in Y$ 。由定义，

$$(\mathcal{A}_!(u \oplus v))(y) = \bigoplus_{e \in E_A(-, y)} w_A(e) \otimes (u \oplus v)(s_A(e)).$$

Step 2. 逐点加法给出

$$(u \oplus v)(s_A(e)) = u(s_A(e)) \oplus v(s_A(e)).$$

代入得

$$= \bigoplus_{e \in E_A(-, y)} w_A(e) \otimes (u(s_A(e)) \oplus v(s_A(e))).$$

Step 3. 由半环分配律,

$$w_A(e) \otimes (u(s_A(e)) \oplus v(s_A(e))) = (w_A(e) \otimes u(s_A(e))) \oplus (w_A(e) \otimes v(s_A(e))).$$

Step 4. 再用有限聚合的交换律与结合律, 可把所有 u -贡献与所有 v -贡献分组:

$$= \left(\bigoplus_{e \in E_A(-, y)} w_A(e) \otimes u(s_A(e)) \right) \oplus \left(\bigoplus_{e \in E_A(-, y)} w_A(e) \otimes v(s_A(e)) \right).$$

Step 5. 右边正是

$$(\mathcal{A}_! u)(y) \oplus (\mathcal{A}_! v)(y).$$

由于 y 任意, 得到第一式。

Step 6. 若 $u = 0$, 则对每个事件 e , 有

$$w_A(e) \otimes u(s_A(e)) = w_A(e) \otimes 0 = 0.$$

目标纤维中的所有贡献都是 0, 聚合结果为 0。因此 $\mathcal{A}_!(0) = 0$ 。

□

注记

这里的“线性”只使用半环意义下的加法和乘法, 不要求负数、减法或加法逆元。这一点对硬件很重要: 布尔逻辑、最短路、概率质量、量化整数传播都可以放进同一个形式, 而不需要强行嵌入实向量空间。

4.5 矩阵影子

现在把加权对应压缩回矩阵。这个压缩只保留端点对 (x, y) 上所有事件的聚合权重。

定义 4.5: 对应的矩阵影子

设

$$\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, w_A)$$

是 $\mathbb{K}$ -加权对应。它的矩阵影子 $M(\mathcal{A}) \in \mathbb{K}^{Y \times X}$ 定义为

$$M(\mathcal{A})_{y,x} = \bigoplus_{e \in E_A(x,y)} w_A(e) = \bigoplus_{\substack{e \in E_A: s_A(e)=x \\ t_A(e)=y}} w_A(e).$$

若 $E_A(x, y) = \emptyset$, 则该条目为空聚合, 等于 0。

命题 4.2: 对应作用等于矩阵影子作用

设 $\mathcal{A} : X \rightsquigarrow Y$ 是 $\mathbb{K}$ -加权对应。对任意 $u \in \mathbb{K}^X$, 有

$$\mathcal{A}_! u = M(\mathcal{A})u,$$

其中右边是第三章定义的半环矩阵作用：

$$(M(\mathcal{A})u)(y) = \bigoplus_{x \in X} M(\mathcal{A})_{yx} \otimes u(x).$$

证明.

Step 1. 固定 $y \in Y$ 。由对应作用定义，

$$(\mathcal{A}!u)(y) = \bigoplus_{e \in E_A(-, y)} w_A(e) \otimes u(s_A(e)).$$

Step 2. 目标纤维 $E_A(-, y)$ 可按源端点进一步分解：

$$E_A(-, y) = \bigsqcup_{x \in X} E_A(x, y).$$

这是因为每个落到 y 的事件仍有唯一源端点。

Step 3. 因此

$$(\mathcal{A}!u)(y) = \bigoplus_{x \in X} \bigoplus_{e \in E_A(x, y)} w_A(e) \otimes u(x),$$

因为在 $E_A(x, y)$ 上有 $s_A(e) = x$ 。

Step 4. 对固定的 x ， $u(x)$ 与内层聚合指标 e 无关。由有限分配律，

$$\bigoplus_{e \in E_A(x, y)} w_A(e) \otimes u(x) = \left(\bigoplus_{e \in E_A(x, y)} w_A(e) \right) \otimes u(x).$$

Step 5. 括号中的聚合正是 $M(\mathcal{A})_{yx}$ 。所以

$$(\mathcal{A}!u)(y) = \bigoplus_{x \in X} M(\mathcal{A})_{yx} \otimes u(x) = (M(\mathcal{A})u)(y).$$

Step 6. 由于 y 任意，命题成立。 □

推论 4.1: 矩阵是对应作用的坐标影子

若只关心 $\mathcal{A}$ 对所有输入数据 $u \in \mathbb{K}^X$ 的作用，则矩阵影子 $M(\mathcal{A})$ 已经足够。但若关心事件的数量、类型、生成规则、可复合结构或硬件访问方式，则 $M(\mathcal{A})$ 一般不足以恢复 $\mathcal{A}$ 。

4.6 影子会遗忘什么

矩阵影子把每个双端点纤维 $E_A(x, y)$ 中的权重聚合成单个条目。因此它至少会遗忘四类信息：平行事件的数量、事件的类型、事件的生成规则，以及事件空间上的附加结构。下面从最简单的例子开始。

命题 4.3: 矩阵影子不是忠实表示

存在两个不同的加权对应 $\mathcal{A} \neq \mathcal{B}$, 使得

$$M(\mathcal{A}) = M(\mathcal{B}).$$

证明.

Step 1. 取 $X = Y = \{*\}$, 取半环 $\mathbb{K} = \mathbb{R}$ 。

Step 2. 定义 $\mathcal{A}$ 的事件空间为单点集 $E_A = \{e\}$, 源端点和目标端点都只能是 $*$, 权重为

$$w_A(e) = 2.$$

Step 3. 定义 $\mathcal{B}$ 的事件空间为两点集 $E_B = \{f_1, f_2\}$, 源端点和目标端点也都只能是 $*$, 权重为

$$w_B(f_1) = 1, \quad w_B(f_2) = 1.$$

Step 4. 于是

$$M(\mathcal{A})_{**} = 2,$$

而

$$M(\mathcal{B})_{**} = 1 + 1 = 2.$$

因此两个矩阵影子相同。

Step 5. 但是 $|E_A| = 1$, $|E_B| = 2$, 所以事件空间不同, 对应不同。

□

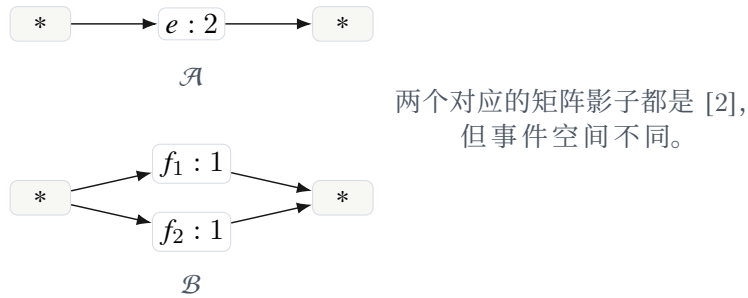


图 4.5 矩阵影子会把一个权重为 2 的事件与两个权重为 1 的事件混同。

这个例子看似简单, 却触及了对应语言的根本目的。矩阵影子记录的是聚合后的数值, 而不是数值的形成过程。对于纯线性代数, 这可能不重要; 对于复合、压缩、路由、并行化和硬件映射, 形成过程非常重要。

例 4.4: 平行边与图消息传递

设 $X = Y = V$ 是顶点集合。在多重图中, 两个顶点 u, v 之间可能有多条边, 分别代表不同关系类型。矩阵邻接表示通常把它们合并成一个数, 例如边数或权重和。对应表示则取 E 为边集合, 每条边作为独立事件。

若两条边分别是“时间邻近”和“空间邻近”, 它们可以在后续复合中服从不同规则。矩阵影子把它们加起来之后, 这种类型差异就消失了。

例 4.5: 卷积的事件空间

设 $X = Y = \{0, 1, \dots, n-1\}$, 考虑一维局部卷积

$$v(y) = \bigoplus_{r \in R} k(r) \otimes u(y-r),$$

其中 $R \subseteq \mathbb{Z}$ 是有限偏移集合。自然的事件空间不是 $X \times Y$, 而是

$$E = \{(x, r) : x \in X, x+r \in Y, r \in R\}.$$

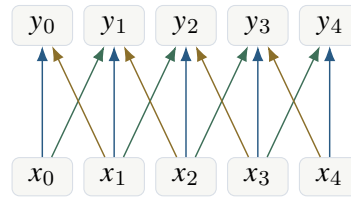
源映射和目标映射为

$$s(x, r) = x, \quad t(x, r) = x + r,$$

权重为

$$w(x, r) = k(r).$$

矩阵影子是一个带状 Toeplitz 型矩阵, 但事件空间直接记录了局部偏移规则。若 $|R| \ll n$, 则 $|E| \approx n|R|$, 远小于 n^2 。



$R = \{-1, 0, 1\}$ 生成局部事件, 而不是任意 $X \times Y$ 连接

图 4.6 一维卷积对应由少量偏移规则生成。矩阵表示会把这些规则展开成坐标条目。

4.7 加权对应的等价与同构

如果事件空间只是被重新命名, 对应不应当被视为本质不同。我们需要定义加权对应之间的同构。

定义 4.6: 加权对应的同构

设

$$\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, w_A), \quad \mathcal{B} = (X \xleftarrow{s_B} E_B \xrightarrow{t_B} Y, w_B)$$

是从同一个 X 到同一个 Y 的两个 $\mathbb{K}$ -加权对应。一个同构 $\phi : \mathcal{A} \rightarrow \mathcal{B}$ 是一个双射

$$\phi : E_A \rightarrow E_B$$

满足对所有 $e \in E_A$,

$$s_B(\phi(e)) = s_A(e), \quad t_B(\phi(e)) = t_A(e), \quad w_B(\phi(e)) = w_A(e).$$

若存在这样的 ϕ , 称 $\mathcal{A}$ 与 $\mathcal{B}$ 同构, 记作 $\mathcal{A} \cong \mathcal{B}$ 。

同构只允许重命名事件, 不允许改变端点与权重。它保留了对应作为事件系统的全部结构。注意, 矩阵影子相同远弱于同构。

命题 4.4: 同构保持矩阵影子

若 $\mathcal{A} \cong \mathcal{B}$, 则

$$M(\mathcal{A}) = M(\mathcal{B}).$$

证明.

Step 1. 设 $\phi : E_A \rightarrow E_B$ 是同构。固定端点对 (x, y) 。

Step 2. 若 $e \in E_A(x, y)$, 则

$$s_B(\phi(e)) = s_A(e) = x, \quad t_B(\phi(e)) = t_A(e) = y.$$

所以 $\phi(e) \in E_B(x, y)$ 。

Step 3. 同理, 由 ϕ^{-1} 可知每个 $E_B(x, y)$ 中的事件都来自唯一的 $E_A(x, y)$ 中的事件。因此 ϕ 限制为双射

$$E_A(x, y) \xrightarrow{\sim} E_B(x, y).$$

Step 4. 同构还保持权重, 即 $w_B(\phi(e)) = w_A(e)$ 。因此

$$\bigoplus_{e \in E_A(x, y)} w_A(e) = \bigoplus_{f \in E_B(x, y)} w_B(f).$$

Step 5. 左右两边分别是 $M(\mathcal{A})_{yx}$ 与 $M(\mathcal{B})_{yx}$ 。由于 (x, y) 任意, 矩阵影子相同。

□

命题 4.5: 矩阵影子相同不推出同构

一般地,

$$M(\mathcal{A}) = M(\mathcal{B})$$

不推出 $\mathcal{A} \cong \mathcal{B}$ 。

证明.

Step 1. 使用命题 4.3 中的两个对应 $\mathcal{A}$ 与 $\mathcal{B}$ 。

Step 2. 它们的矩阵影子相同, 都是单个条目 2。

Step 3. 但 $|E_A| = 1$, $|E_B| = 2$ 。若存在同构 $\phi : E_A \rightarrow E_B$, 则 ϕ 必须是双射, 从而要求两个事件空间基数相同。这与 $1 \neq 2$ 矛盾。

Step 4. 因此矩阵影子相同不推出同构。

□

注记

这一区分对本讲义后续发展很重要。矩阵语言通常把“产生相同矩阵”的所有表示都视为等价; 对应语言则区分不同的事件分解, 因为不同分解在复合时会产生不同的中间路径空间和不同的计算成本。

4.8 直和、并合与零对应

加权对应也有加法结构。矩阵加法把两个矩阵的同一条目相加；对应语言中，更自然的操作是把事件空间并起来。聚合发生在矩阵影子层面，而不是在事件空间层面提前发生。

定义 4.7: 并合和直和

设 $\mathcal{A}, \mathcal{B} : X \rightsquigarrow Y$ 是两个 $\mathbb{K}$ -加权对应，具有事件空间 E_A, E_B 。定义它们的并合

$$\mathcal{A} \sqcup \mathcal{B} : X \rightsquigarrow Y$$

为

$$X \xleftarrow{s} E_A \sqcup E_B \xrightarrow{t} Y,$$

其中 s, t 在 E_A 上分别等于 s_A, t_A ，在 E_B 上分别等于 s_B, t_B ；权重函数 w 在两个分量上分别等于 w_A, w_B 。

定义 4.8: 零对应

从 X 到 Y 的零对应 $0_{Y,X} : X \rightsquigarrow Y$ 定义为事件空间为空的对应：

$$X \leftarrow \emptyset \rightarrow Y.$$

权重函数是唯一的空函数 $\emptyset \rightarrow \mathbb{K}$ 。

命题 4.6: 并合对应矩阵加法

对任意 $\mathcal{A}, \mathcal{B} : X \rightsquigarrow Y$ ，有

$$M(\mathcal{A} \sqcup \mathcal{B}) = M(\mathcal{A}) \oplus M(\mathcal{B}),$$

其中右边是矩阵的逐点加法。并且

$$M(0_{Y,X}) = 0.$$

证明.

Step 1. 固定 $(x, y) \in X \times Y$ 。并合对应在 (x, y) 上的双端点纤维为不交并

$$(E_A \sqcup E_B)(x, y) = E_A(x, y) \sqcup E_B(x, y).$$

Step 2. 因此

$$M(\mathcal{A} \sqcup \mathcal{B})_{yx} = \bigoplus_{e \in E_A(x, y) \sqcup E_B(x, y)} w(e).$$

Step 3. 由于这是不交并上的有限聚合，按两部分分组得到

$$= \left(\bigoplus_{e \in E_A(x, y)} w_A(e) \right) \oplus \left(\bigoplus_{f \in E_B(x, y)} w_B(f) \right).$$

Step 4. 这正是

$$M(\mathcal{A})_{yx} \oplus M(\mathcal{B})_{yx}.$$

端点对任意，所以矩阵影子满足逐点加法公式。

Step 5. 零对应没有事件，故每个双端点纤维都是空集，矩阵影子每个条目都是空聚合 0。

□

硬件解释

合并对应硬件上不是立刻把权重加起来，而是把事件流拼接起来。是否提前合并平行事件，是优化策略；数学语言允许我们在“保留事件”和“聚合成矩阵条目”之间选择合适的时机。

4.9 恒等对应

矩阵语言中的恒等矩阵在对应语言中也有更基本的表示。它由每个对象上的单位事件组成。

定义 4.9: 恒等对应

设 X 是有限集合。 X 上的恒等对应 $1_X : X \rightsquigarrow X$ 定义为

$$X \xleftarrow{\text{id}_X} X \xrightarrow{\text{id}_X} X,$$

其中事件空间就是 X 本身，源映射与目标映射都是恒等映射，权重函数恒等于半环单位元 $1 \in \mathbb{K}$ 。

也就是说，对每个 $x \in X$ ，有一个单位事件

$$x \xrightarrow{1} x.$$

它表示长度为零或“不改变对象”的路径贡献。

命题 4.7: 恒等对应的矩阵影子

恒等对应的矩阵影子是恒等矩阵：

$$M(1_X) = I_X,$$

其中

$$(I_X)_{yx} = \begin{cases} 1, & y = x, \\ 0, & y \neq x. \end{cases}$$

证明.

Step 1. 恒等对应的事件空间为 X 。对事件 $e \in X$ ，有

$$s(e) = e, \quad t(e) = e, \quad w(e) = 1.$$

Step 2. 固定端点对 (x, y) 。若 $x = y$ ，则双端点纤维 $E(x, x)$ 只含事件 x ，所以

$$M(1_X)_{xx} = w(x) = 1.$$

Step 3. 若 $x \neq y$ ，不存在事件同时满足 $s(e) = x$ 与 $t(e) = y$ ，因为恒等事件只能从自身到自身。因此 $E(x, y) = \emptyset$ ，于是

$$M(1_X)_{yx} = 0.$$

Step 4. 这正是恒等矩阵的定义。

□

命题 4.8: 恒等对应数据的作用

对任意 $u \in \mathbb{K}^X$, 有

$$(1_X)_! u = u.$$

证明.

Step 1. 固定 $y \in X$ 。恒等对应落到 y 的目标纤维只含事件 y 本身。

Step 2. 因此

$$((1_X)_! u)(y) = 1 \otimes u(y) = u(y).$$

Step 3. 对每个 y 都成立, 所以 $(1_X)_! u = u$ 。

□

4.10 转置与反向对应

矩阵转置把输入输出坐标交换。对应语言中的操作更直观：把源映射和目标映射互换, 事件空间不变, 权重不变。

定义 4.10: 反向对应

设

$$\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, w_A)$$

是从 X 到 Y 的加权对应。其反向对应或对偶对应

$$\mathcal{A}^{\text{op}} : Y \rightsquigarrow X$$

定义为

$$Y \xleftarrow{t_A} E_A \xrightarrow{s_A} X,$$

权重函数仍为 w_A 。

命题 4.9: 反向对应的矩阵影子

有

$$M(\mathcal{A}^{\text{op}}) = M(\mathcal{A})^T.$$

证明.

Step 1. $\mathcal{A}^{\text{op}}$ 从 Y 到 X , 因此其矩阵影子的行索引在 X , 列索引在 Y 。

Step 2. 固定 $x \in X$ 与 $y \in Y$ 。根据定义,

$$M(\mathcal{A}^{\text{op}})_{xy} = \bigoplus_{e: t_A(e)=y, s_A(e)=x} w_A(e).$$

Step 3. 条件 $t_A(e) = y$, $s_A(e) = x$ 与 $s_A(e) = x$, $t_A(e) = y$ 相同。因此

$$M(\mathcal{A}^{\text{op}})_{xy} = \bigoplus_{e \in E_A(x,y)} w_A(e) = M(\mathcal{A})_{yx}.$$

Step 4. 这正是转置矩阵条目的定义。

□

注记

若权重半环带有额外的对合，例如复共轭，则可以定义伴随对应：反向端点的同时对权重取对合。普通复矩阵的共轭转置就是这种结构的坐标影子。本讲义先只使用不带对合的反向对应。

4.11 商、合并与规范化

因为矩阵影子会沿双端点纤维聚合，一个自然问题是：能否把平行事件先合并，得到一个没有平行事件的规范对应？答案是可以，但这种规范化会丢失事件级结构。

定义 4.11: 矩阵化规范对应

设 $\mathcal{A} : X \rightsquigarrow Y$ 是加权对应。定义其矩阵化规范对应 $\overline{\mathcal{A}}$ 为：事件空间

$$\overline{E}_A = \{(x, y) \in X \times Y : M(\mathcal{A})_{yx} \neq 0\},$$

源映射与目标映射为投影，权重为

$$\overline{w}_A(x, y) = M(\mathcal{A})_{yx}.$$

若希望保留零条目，也可取 $\overline{E}_A = X \times Y$ 。

命题 4.10: 规范对应具有相同矩阵影子

对任意 $\mathcal{A} : X \rightsquigarrow Y$ ，有

$$M(\overline{\mathcal{A}}) = M(\mathcal{A}).$$

证明.

Step 1. 固定 (x, y) 。若使用保留非零支撑的定义，则 $\overline{\mathcal{A}}$ 在 (x, y) 上至多有一个事件。

Step 2. 若 $M(\mathcal{A})_{yx} \neq 0$ ，该事件存在，其权重就是 $M(\mathcal{A})_{yx}$ 。于是

$$M(\overline{\mathcal{A}})_{yx} = M(\mathcal{A})_{yx}.$$

Step 3. 若 $M(\mathcal{A})_{yx} = 0$ ，该双端点纤维为空，影子条目为空聚合 0，仍等于 $M(\mathcal{A})_{yx}$ 。

Step 4. 因此所有条目相同。

□

边界

规范对应 $\overline{\mathcal{A}}$ 并不是 $\mathcal{A}$ 的无损替代。它只保留矩阵作用。若后续要分析事件复合的中间路径数、平行事件的来源、图边类型或硬件路由，过早规范化会破坏这些信息。

4.12 加权对应的大小与稀疏度

矩阵的存储大小通常由 $|X||Y|$ 控制。对应的存储大小首先由 $|E_A|$ 控制，但还应考虑端点映射与权重函数的描述方式。若 E_A 由少量生成规则给出，实际描述复杂度可以比 $|E_A|$ 更小。

定义 4.12: 事件大小、端点度数与平均纤维大小

设 $\mathcal{A} : X \rightsquigarrow Y$ 。定义事件大小为

$$\|\mathcal{A}\|_E := |E_A|.$$

定义最大源度数与最大目标度数为

$$d_s(\mathcal{A}) = \max_{x \in X} |E_A(x, -)|, \quad d_t(\mathcal{A}) = \max_{y \in Y} |E_A(-, y)|.$$

定义平均源度数与平均目标度数为

$$\overline{d}_s(\mathcal{A}) = \frac{|E_A|}{|X|}, \quad \overline{d}_t(\mathcal{A}) = \frac{|E_A|}{|Y|},$$

其中假设 X, Y 非空。

命题 4.11: 事件大小与纤维度数的关系

若 X, Y 非空，则

$$|E_A| = |X|\overline{d}_s(\mathcal{A}) = |Y|\overline{d}_t(\mathcal{A}),$$

并且

$$|E_A| \leq |X|d_s(\mathcal{A}), \quad |E_A| \leq |Y|d_t(\mathcal{A}).$$

证明.

Step 1. 平均度数公式由定义直接得到。

Step 2. 由事件空间的源纤维分解，

$$|E_A| = \sum_{x \in X} |E_A(x, -)|.$$

每一项都不超过 $d_s(\mathcal{A})$ ，所以

$$|E_A| \leq \sum_{x \in X} d_s(\mathcal{A}) = |X|d_s(\mathcal{A}).$$

Step 3. 目标度数的不等式同理，由

$$|E_A| = \sum_{y \in Y} |E_A(-, y)|$$

推出。

□

例 4.6: 局部卷积的事件大小

继续考虑一维卷积事件空间

$$E = \{(x, r) : r \in R, x + r \in Y\}.$$

若忽略边界效应，每个 x 至多发出 $|R|$ 个事件，因此

$$|E| \leq |X||R|.$$

而矩阵坐标表大小是 $|X||Y|$ 。当 $|R| \ll |Y|$ 时，对应表示保留了局部结构，避免了展开成大矩阵。

硬件解释

对 CoPU 而言， $|E_A|$ 近似对应事件存储规模， d_s 影响从输入激活枚举事件的并行度， d_t 影响输出聚合器的拥塞程度。矩阵大小 $|X||Y|$ 并不能准确反映这些硬件成本。

4.13 本章小结

本章定义了加权对应，并把它与矩阵语言精确连接起来。一个加权对应

$$\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, w_A)$$

由事件空间、端点映射和权重函数组成。它对输入数据的作用是

$$(\mathcal{A}!u)(y) = \bigoplus_{e \in E_A(-, y)} w_A(e) \otimes u(s_A(e)),$$

也可以分解为

$$\mathcal{A}! = (t_A)! \circ M_{w_A} \circ s_A^*.$$

它的矩阵影子为

$$M(\mathcal{A})_{yx} = \bigoplus_{e \in E_A(x, y)} w_A(e).$$

矩阵影子决定对应对对象数据的作用，但一般不能恢复事件空间。并合对应矩阵加法，反向对应矩阵转置，恒等对应给出恒等矩阵。下一章将处理最重要的结构：两个加权对应如何复合。那里的核心对象将是纤维积

$$E_B \times_Y E_A,$$

它表示所有能够首尾相接的二步路径。

纤维复合：拉回、局部合成、推出

上一章把一个矩阵条目替换成一个事件纤维，把一个矩阵替换成一个带权事件空间

$$X \xleftarrow{s} E \xrightarrow{t} Y, \quad w : E \rightarrow \mathbb{K}.$$

本章讨论下一步：两个这样的对象如何相乘。这里的“相乘”不能简单地理解为把两组事件并在一起，也不能理解为逐点相乘。矩阵乘法真正做的事情是：先选择一个中间坐标，再把前后两段连接成路径，最后对所有中间坐标求和。对应语言要保留的正是这个过程。

因此，对应复合分成三个动作：

$$\text{拉回} \longrightarrow \text{局部权重合成} \longrightarrow \text{推出与聚合}.$$

拉回负责找出可以首尾相接的事件对；局部合成负责把两段事件的权重合成一段路径权重；推出负责把路径投到新的源端点与目标端点上。矩阵乘法只是这三个动作在完全坐标化情形下的影子。

说明

本章的目的不是给出一个形式定义就结束，而是解释为什么这个定义几乎是被矩阵乘法唯一地逼出来的。只要我们要求：

1. 函数复合在新语言中仍是函数复合；
2. 关系复合在新语言中仍是关系复合；
3. 有限矩阵的乘法在新语言中仍是矩阵乘法；
4. 事件的 multiplicity 不被提前遗忘，

那么复合必须通过中间纤维的拉回来定义。

5.1 矩阵乘法中隐藏的三步

先回到普通矩阵。设

$$A \in \mathbb{K}^{Y \times X}, \quad B \in \mathbb{K}^{Z \times Y}.$$

矩阵乘法给出

$$(BA)_{zx} = \bigoplus_{y \in Y} B_{zy} \otimes A_{yx}.$$

这个公式看起来只是一个代数式，但它包含三个不同层次的操作。

第一，选择一个中间点 $y \in Y$ 。这一点决定了 x 到 z 的二步路径是否经过 y 。

第二，把两段权重合成：

$$A_{yx} \text{ 与 } B_{zy}$$

合成为

$$B_{zy} \otimes A_{yx}.$$

这里乘积的顺序不是任意的。先发生的是 A ，后发生的是 B ，因此复合写作 $B \circ A$ ，权重写作 $B_{zy} \otimes A_{yx}$ 。

第三，对所有可能的中间点 y 做聚合：

$$\bigoplus_{y \in Y}.$$

这一步把许多二步路径压缩成一个矩阵条目。

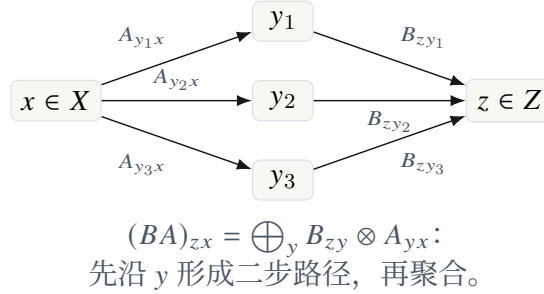


图 5.1 矩阵乘法中的中间纤维求和。

对应复合只是在事件层面保留上述三步，而不是一开始就把它们压成矩阵条目。这样做的好处是，若同一个端点对之间存在多条不同来源、不同类型或不同生成规则的事件，它们在复合时仍然保持可区分，直到我们明确要求取矩阵影子或做某种压缩。

5.2 可复合对应

定义 5.1: 可复合加权对应

设 X, Y, Z 是有限对象空间。一个从 X 到 Y 的加权对应记为

$$\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, w_A),$$

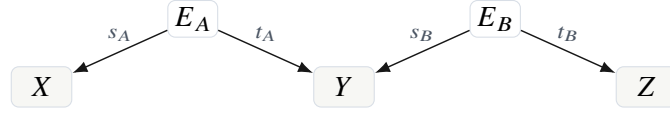
一个从 Y 到 Z 的加权对应记为

$$\mathcal{B} = (Y \xleftarrow{s_B} E_B \xrightarrow{t_B} Z, w_B).$$

由于 $\mathcal{A}$ 的目标对象与 $\mathcal{B}$ 的源对象同为 Y ，称 $\mathcal{A}$ 与 $\mathcal{B}$ 可复合，并把复合方向写作

$$\mathcal{B} \circ \mathcal{A} : X \rightsquigarrow Z.$$

在这个定义中，中间对象 Y 是真正的接口。 $\mathcal{A}$ 的事件到达 Y ， $\mathcal{B}$ 的事件从 Y 出发。只有当二者在 Y 上落到同一个点时，才形成一条合法路径。



公共对象 Y 是复合接口；匹配条件是 $t_A(e_A) = s_B(e_B)$ 。

图 5.2 两个可复合对应。箭头方向表示端点映射，而不是事件流方向。

边界

端点图中的箭头

$$X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y$$

是从事件空间投到对象空间的映射。事件的直观流向是 $X \rightarrow Y$ ，但结构映射的箭头从 E_A 指向 X 与 Y 。这种记号来自对应与 span 的传统写法，后面做纤维积时非常方便。

5.3 第一步：拉回与二步路径空间

矩阵乘法中， y 是一个中间坐标。对应语言中，中间坐标不再只是一个求和指标，而是一个匹配条件。我们必须找出所有满足

$$t_A(e_A) = s_B(e_B)$$

的事件对。

定义 5.2: 可复合事件空间

设 $\mathcal{A}: X \rightsquigarrow Y$, $\mathcal{B}: Y \rightsquigarrow Z$ 如上。它们的可复合事件空间定义为纤维积

$$E_B \times_Y E_A = \{(e_B, e_A) \in E_B \times E_A : s_B(e_B) = t_A(e_A)\}.$$

其中公共值

$$y = s_B(e_B) = t_A(e_A)$$

称为二步路径 (e_B, e_A) 的中间端点。

注记

我们写 $E_B \times_Y E_A$ ，而不是 $E_A \times_Y E_B$ ，是为了与复合记号 $\mathcal{B} \circ \mathcal{A}$ 对齐。右边的 $\mathcal{A}$ 先发生，左边的 $\mathcal{B}$ 后发生；二步路径的元素也写成 (e_B, e_A) 。

命题 5.1: 纤维分解公式

可复合事件空间有自然分解

$$E_B \times_Y E_A = \bigsqcup_{y \in Y} s_B^{-1}(y) \times t_A^{-1}(y).$$

特别地，若所有集合有限，则

$$|E_B \times_Y E_A| = \sum_{y \in Y} |s_B^{-1}(y)| |t_A^{-1}(y)|.$$

证明.

Step 1. 对任意 $(e_B, e_A) \in E_B \times_Y E_A$, 由定义有

$$s_B(e_B) = t_A(e_A).$$

把这个共同值记作 y 。于是

$$e_B \in s_B^{-1}(y), \quad e_A \in t_A^{-1}(y),$$

所以

$$(e_B, e_A) \in s_B^{-1}(y) \times t_A^{-1}(y).$$

Step 2. 反过来, 若

$$(e_B, e_A) \in s_B^{-1}(y) \times t_A^{-1}(y),$$

则

$$s_B(e_B) = y = t_A(e_A),$$

所以 (e_B, e_A) 满足纤维积匹配条件, 属于 $E_B \times_Y E_A$ 。

Step 3. 若 $y \neq y'$, 则

$$s_B^{-1}(y) \times t_A^{-1}(y)$$

与

$$s_B^{-1}(y') \times t_A^{-1}(y')$$

没有公共元素。因为若同一个 (e_B, e_A) 同时属于二者, 则 $s_B(e_B) = y$ 且 $s_B(e_B) = y'$, 从而 $y = y'$, 矛盾。

Step 4. 因此得到不交并分解。有限情形下, 对不交并取基数, 得到

$$|E_B \times_Y E_A| = \sum_{y \in Y} |s_B^{-1}(y) \times t_A^{-1}(y)| = \sum_{y \in Y} |s_B^{-1}(y)| |t_A^{-1}(y)|.$$

□

硬件解释

这个公式是硬件实现的第一条成本公式。若把 Y 看成中间桶, $t_A^{-1}(y)$ 是进入桶 y 的事件, $s_B^{-1}(y)$ 是从桶 y 出发的事件, 那么二步路径数正是每个桶的进入数与出发数的乘积之和。CoPU 的 FIBER_JOIN 本质上就是按 y 分桶并在桶内做局部笛卡尔积。

例 5.1: 函数图的拉回

设 $f: X \rightarrow Y$ 与 $g: Y \rightarrow Z$ 是两个函数。把它们看成无权对应:

$$E_f = X, \quad qquads_f = \text{id}_X, \quad t_f = f,$$

$$E_g = Y, \quad s_g = \text{id}_Y, \quad t_g = g.$$

则

$$E_g \times_Y E_f = \{(y, x) \in Y \times X : y = f(x)\}.$$

这个集合与 X 自然双射:

$$x \mapsto (f(x), x).$$

在这个双射下, 复合对应正是函数 $g \circ f: X \rightarrow Z$ 的图。

证明.

Step 1. 对函数 f 的图对应，事件 $x \in X$ 的目标端点是 $f(x)$ 。

Step 2. 对函数 g 的图对应，事件 $y \in Y$ 的源端点是 y 本身。

Step 3. 因此可复合条件是

$$s_g(y) = t_f(x),$$

也就是

$$y = f(x).$$

Step 4. 所以每个 x 唯一决定一个可复合事件对 $(f(x), x)$ 。

Step 5. 该二步路径的源端点是 x ，目标端点是 $g(f(x))$ ，正是 $g \circ f$ 。

□

这个例子说明：纤维积不是额外复杂化，而是在函数情形中退化为普通函数复合。它只是把“中间值必须相等”这件事显式写出来。

5.4 第二步：局部权重合成

纤维积只产生二步路径，还没有给二步路径赋权。设

$$(e_B, e_A) \in E_B \times_Y E_A.$$

其中 e_A 是第一段事件， e_B 是第二段事件。它们的路径权重定义为

$$w_{BA}(e_B, e_A) = w_B(e_B) \otimes w_A(e_A).$$

定义 5.3: 路径权重

给定可复合加权对应 $\mathcal{A}: X \rightsquigarrow Y$ 与 $\mathcal{B}: Y \rightsquigarrow Z$ ，其二步路径权重是映射

$$w_{BA}: E_B \times_Y E_A \rightarrow \mathbb{K}, \quad w_{BA}(e_B, e_A) = w_B(e_B) \otimes w_A(e_A).$$

为什么不是 $w_A(e_A) \otimes w_B(e_B)$ ？如果 $\mathbb{K}$ 的乘法交换，两者相同；但原则上路径合成有时间顺序，后发生的算子在左边，先发生的算子在右边。这与矩阵乘法约定一致：

$$(BA)_{zx} = \bigoplus_y B_{zy} \otimes A_{yx}.$$

边界

本讲义大部分章节默认 $\mathbb{K}$ 是交换半环，是为了简化矩阵影子与硬件聚合的讨论。但复合公式的书写顺序仍然保留非交换情形的习惯。后续若把权重扩展为小矩阵、算子块或非交换代数元素，这一点会变得必要。

例 5.2: 几种半环中的路径权重

同一个二步路径在不同半环中有不同语义。

半环	路径权重	解释
实数半环 $(\mathbb{R}, +, \cdot)$	$w_B w_A$	线性传播中的乘法贡献
布尔半环 $(\{0, 1\}, \vee, \wedge)$	$w_B \wedge w_A$	两段关系同时存在
计数半环 $(\mathbb{N}, +, \cdot)$	$w_B w_A$	路径数量相乘后求和
min-plus 半环	$w_B + w_A$	两段路径代价相加
max-plus 半环	$w_B + w_A$	两段收益或得分相加

5.5 第三步：推出端点

现在每个二步路径已经有了权重。为了得到一个从 X 到 Z 的对应，还需要规定二步路径的源端点与目标端点。

定义 5.4: 复合端点映射

对任意二步路径 $(e_B, e_A) \in E_B \times_Y E_A$ ，定义

$$s_{BA}(e_B, e_A) = s_A(e_A) \in X,$$

$$t_{BA}(e_B, e_A) = t_B(e_B) \in Z.$$

也就是说，整条二步路径的起点由第一段事件决定，终点由第二段事件决定。

于是我们得到一个新的加权对应。

定义 5.5: 对应复合

设 $\mathcal{A} : X \rightsquigarrow Y$ ， $\mathcal{B} : Y \rightsquigarrow Z$ 是两个 $\mathbb{K}$ -加权对应。它们的复合定义为

$$\mathcal{B} \circ \mathcal{A} = \left(X \xleftarrow{s_{BA}} E_B \times_Y E_A \xrightarrow{t_{BA}} Z, w_{BA} \right),$$

其中

$$s_{BA}(e_B, e_A) = s_A(e_A), \quad t_{BA}(e_B, e_A) = t_B(e_B),$$

$$w_{BA}(e_B, e_A) = w_B(e_B) \otimes w_A(e_A).$$

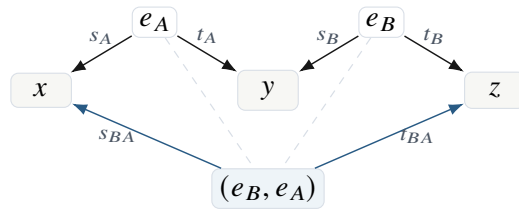


图 5.3 二步路径 (e_B, e_A) 的新源端点与新目标端点。

注记

复合对应的事件空间不是 $X \times Z$ ，而是二步路径空间 $E_B \times_Y E_A$ 。只有当取矩阵影子时，才把所有具有同一端点 (x, z) 的路径聚合在一起。因此，对应复合保留了路径来源；矩阵乘

法只保留了路径总和。

5.6 端点纤维与路径聚合

复合对应的一个端点对 $(x, z) \in X \times Z$ 对应多少条路径？答案由端点纤维给出。

定义 5.6: 复合端点纤维

复合对应 $\mathcal{B} \circ \mathcal{A}$ 在端点对 (x, z) 上的纤维定义为

$$\text{Fib}_{BA}(z, x) = \{(e_B, e_A) \in E_B \times_Y E_A : s_A(e_A) = x, t_B(e_B) = z\}.$$

等价地,

$$\text{Fib}_{BA}(z, x) = \bigsqcup_{y \in Y} \left(s_B^{-1}(y) \cap t_B^{-1}(z) \right) \times \left(t_A^{-1}(y) \cap s_A^{-1}(x) \right).$$

命题 5.2: 端点纤维分解

对任意 $x \in X$ 与 $z \in Z$, 上面的不交并分解成立。有限情形下,

$$|\text{Fib}_{BA}(z, x)| = \sum_{y \in Y} |s_B^{-1}(y) \cap t_B^{-1}(z)| |t_A^{-1}(y) \cap s_A^{-1}(x)|.$$

证明.

Step 1. 设 $(e_B, e_A) \in \text{Fib}_{BA}(z, x)$ 。由定义,

$$s_A(e_A) = x, \quad t_B(e_B) = z,$$

并且由于它属于纤维积, 存在共同中间点

$$y = s_B(e_B) = t_A(e_A).$$

Step 2. 因此

$$e_B \in s_B^{-1}(y) \cap t_B^{-1}(z), \quad e_A \in t_A^{-1}(y) \cap s_A^{-1}(x).$$

所以 (e_B, e_A) 属于右边某个 y 对应的乘积。

Step 3. 反过来, 若 (e_B, e_A) 属于右边 y 对应的乘积, 则

$$s_B(e_B) = y = t_A(e_A),$$

所以它是可复合事件对; 同时 $s_A(e_A) = x$ 、 $t_B(e_B) = z$, 故属于 $\text{Fib}_{BA}(z, x)$ 。

Step 4. 不同 y 的部分不交, 理由与纤维积分解相同。

Step 5. 对不交并取基数, 得到公式。

□

这一定理把矩阵乘法中的双重求和拆开了。对固定端点 (x, z) , 矩阵条目不是由一个数直接生成, 而是由所有中间纤维上的路径贡献聚合而成。

定义 5.7: 复合的矩阵影子

复合对应的矩阵影子是矩阵

$$M(\mathcal{B} \circ \mathcal{A}) \in \mathbb{K}^{Z \times X},$$

其 (z, x) 条目为

$$M(\mathcal{B} \circ \mathcal{A})_{zx} = \bigoplus_{(e_B, e_A) \in \text{Fib}_{BA}(z, x)} w_B(e_B) \otimes w_A(e_A).$$

5.7 复合的矩阵影子

现在证明最关键的一致性：先复合对应再取矩阵影子，等价于先取矩阵影子再做半环矩阵乘法。这个定理说明，对应复合不是另起炉灶，而是矩阵乘法的事件层提升。

定理 5.1: 矩阵影子与复合相容

设 $\mathcal{A} : X \rightsquigarrow Y$, $\mathcal{B} : Y \rightsquigarrow Z$ 是有限 $\mathbb{K}$ -加权对应。则

$$M(\mathcal{B} \circ \mathcal{A}) = M(\mathcal{B}) \odot M(\mathcal{A}),$$

其中右边是半环矩阵乘法。

证明.

Step 1. 固定 $x \in X$ 与 $z \in Z$ 。根据复合矩阵影子的定义，左边的 (z, x) 条目为

$$M(\mathcal{B} \circ \mathcal{A})_{zx} = \bigoplus_{\substack{(e_B, e_A) \in E_B \times_Y E_A \\ s_A(e_A) = x, t_B(e_B) = z}} w_B(e_B) \otimes w_A(e_A).$$

Step 2. 展开纤维积条件，得到

$$= \bigoplus_{\substack{e_B \in E_B, e_A \in E_A \\ s_B(e_B) = t_A(e_A) \\ s_A(e_A) = x, t_B(e_B) = z}} w_B(e_B) \otimes w_A(e_A).$$

Step 3. 按共同中间端点

$$y = s_B(e_B) = t_A(e_A)$$

分组。由于 Y 有限，上式等于

$$\bigoplus_{y \in Y} \bigoplus_{\substack{e_B: s_B(e_B) = y, t_B(e_B) = z \\ e_A: s_A(e_A) = x, t_A(e_A) = y}} w_B(e_B) \otimes w_A(e_A).$$

Step 4. 对固定 y ，内层是两组事件贡献的分配展开。由半环分配律，

$$\bigoplus_{\substack{e_B: s_B(e_B) = y, t_B(e_B) = z \\ e_A: s_A(e_A) = x, t_A(e_A) = y}} w_B(e_B) \otimes w_A(e_A)$$

等于

$$\left(\bigoplus_{e_B: s_B(e_B) = y, t_B(e_B) = z} w_B(e_B) \right) \otimes \left(\bigoplus_{e_A: s_A(e_A) = x, t_A(e_A) = y} w_A(e_A) \right).$$

Step 5. 根据矩阵影子的定义，两个括号分别是

$$M(\mathcal{B})_{zy}, \quad M(\mathcal{A})_{yx}.$$

Step 6. 因此

$$M(\mathcal{B} \circ \mathcal{A})_{zx} = \bigoplus_{y \in Y} M(\mathcal{B})_{zy} \otimes M(\mathcal{A})_{yx},$$

这正是半环矩阵乘积 $M(\mathcal{B}) \odot M(\mathcal{A})$ 的 (z, x) 条目。

Step 7. 对所有 x, z 成立，所以两个矩阵相等。

□

注记

证明中的关键不是某个复杂技巧，而是分配律。矩阵乘法能够先“沿路径相乘”再“沿中间点求和”，正是因为

$$b \otimes (a_1 \oplus a_2) = (b \otimes a_1) \oplus (b \otimes a_2),$$

以及右分配律成立。第三章引入半环，就是为了保证这一点。

5.8 三个基本例子

5.8.1 函数复合

前面已经看到，函数图的复合退化为普通函数复合。现在把权重也考虑进去。若每条函数图事件的权重都是 1，则路径权重仍是

$$1 \otimes 1 = 1.$$

因此 $g \circ f$ 的图对应也是单位权重。函数复合是对应复合中最薄的一种情形：每个输入点只有一条事件，每个中间点上的匹配也没有 multiplicity。

5.8.2 关系复合

若 $R \subset X \times Y$, $S \subset Y \times Z$ 是二元关系，把它们看成布尔加权对应。对应复合的事件空间是

$$S \times_Y R = \{((y, z), (x, y)) : (x, y) \in R, (y, z) \in S\}.$$

取矩阵影子后， (x, z) 条目为真当且仅当存在 y 使得

$$xRy, \quad ySz.$$

这正是关系复合。

命题 5.3: 布尔影子给出关系复合

在布尔半环上，两个关系对应的复合矩阵影子等于通常的关系复合矩阵。

证明.

Step 1. 布尔半环中，聚合 $\oplus$ 是“或” $\vee$ ，合成 $\otimes$ 是“与” $\wedge$ 。

Step 2. 因此复合影子的 (z, x) 条目为

$$\bigvee_{y \in Y} (S_{zy} \wedge R_{yx}).$$

Step 3. 该值为 1 当且仅当存在 y 使得 $R_{yx} = 1$ 且 $S_{zy} = 1$ 。

Step 4. 这正是存在中间点 y 使 xRy 且 ySz 的定义。

□

5.8.3 普通矩阵乘法

给定矩阵 $A \in \mathbb{K}^{Y \times X}$, 可以构造规范对应

$$\hat{A} = (X \xleftarrow{\pi_X} Y \times X \xrightarrow{\pi_Y} Y, w_A), \quad w_A(y, x) = A_{yx}.$$

同理, 对 $B \in \mathbb{K}^{Z \times Y}$ 有

$$\hat{B} = (Y \xleftarrow{\pi_Y} Z \times Y \xrightarrow{\pi_Z} Z, w_B), \quad w_B(z, y) = B_{zy}.$$

它们的纤维积元素可以写成

$$((z, y), (y, x)),$$

也就是一个三元组 (z, y, x) 。路径权重是

$$B_{zy} \otimes A_{yx}.$$

对端点 (x, z) 聚合所有 y , 得到普通矩阵乘法。

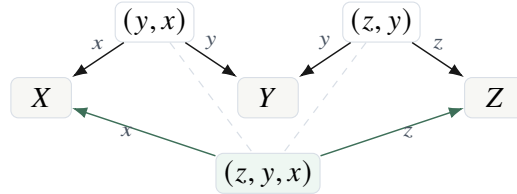


图 5.4 规范矩阵对应的复合。二步路径等价于三元组 (z, y, x) 。

5.9 单位对应

若对应复合要替代矩阵乘法, 就必须有单位元。矩阵语言中的单位矩阵 I_X 应当被提升为一个从 X 到 X 的单位对应。

定义 5.8: 单位对应

对象空间 X 上的单位对应定义为

$$1_X = (X \xleftarrow{\text{id}_X} X \xrightarrow{\text{id}_X} X, 1_X),$$

其中事件空间就是 X 本身, 两个端点映射都是恒等映射, 权重函数恒为半环乘法单位元 1。

定理 5.2: 单位律

设 $\mathcal{A} = (X \xleftarrow{s} E \xrightarrow{t} Y, w)$ 是一个 $\mathbb{K}$ -加权对应。则

$$1_Y \circ \mathcal{A} \cong \mathcal{A}, \quad \mathcal{A} \circ 1_X \cong \mathcal{A},$$

其中同构保持源端点、目标端点与事件权重。

证明.

Step 1. 先证明左单位律。 1_Y 的事件空间是 Y ，源端点与目标端点都是 id_Y 。因此

$$E_{1_Y} \times_Y E = Y \times_Y E = \{(y, e) : y = t(e)\}.$$

Step 2. 定义映射

$$\Phi : Y \times_Y E \rightarrow E, \quad \Phi(y, e) = e.$$

这是双射。其逆映射为

$$e \mapsto (t(e), e).$$

Step 3. 在该双射下，复合对应的源端点为

$$s(e),$$

目标端点为

$$\text{id}_Y(y) = y = t(e),$$

因此与 $\mathcal{A}$ 的端点一致。

Step 4. 权重为

$$1 \otimes w(e) = w(e).$$

所以 $1_Y \circ \mathcal{A} \cong \mathcal{A}$ 。

Step 5. 右单位律类似。事件空间为

$$E \times_X X = \{(e, x) : s(e) = x\}.$$

映射 $(e, x) \mapsto e$ 是双射，逆为 $e \mapsto (e, s(e))$ 。

Step 6. 在该双射下端点保持，权重为

$$w(e) \otimes 1 = w(e).$$

所以 $\mathcal{A} \circ 1_X \cong \mathcal{A}$ 。

□

注记

这里得到的是自然同构，而不是字面相等。因为 $1_Y \circ \mathcal{A}$ 的事件写成 $(t(e), e)$ ，而 $\mathcal{A}$ 的事件写成 e 。二者不是同一个集合，但有不依赖选择的典范双射。对应语言中，许多等式都应理解为“在自然同构下相等”。

5.10 复合结合律

矩阵乘法有结合律，对应复合也必须有结合律。区别在于，矩阵结合律通常通过代数展开证明；对应结合律来自三步路径空间的自然重配对。

设有三个可复合对应：

$$\mathcal{A} : W \rightsquigarrow X, \quad \mathcal{B} : X \rightsquigarrow Y, \quad \mathcal{C} : Y \rightsquigarrow Z.$$

三步路径应该由三个事件

$$e_A \in E_A, \quad e_B \in E_B, \quad e_C \in E_C$$

组成，并满足两个匹配条件：

$$s_B(e_B) = t_A(e_A), \quad s_C(e_C) = t_B(e_B).$$

无论先把 $\mathcal{A}$ 与 $\mathcal{B}$ 复合，还是先把 $\mathcal{B}$ 与 $\mathcal{C}$ 复合，最终都应该得到同一批三步路径。

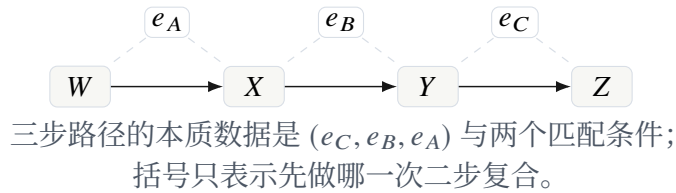


图 5.5 对应复合结合律中的三步路径。

定理 5.3: 对应复合的结合律

设

$$\mathcal{A} : W \rightsquigarrow X, \quad \mathcal{B} : X \rightsquigarrow Y, \quad \mathcal{C} : Y \rightsquigarrow Z$$

是有限 $\mathbb{K}$ -加权对应。则 $C \circ (\mathcal{B} \circ \mathcal{A})$ 与 $(C \circ \mathcal{B}) \circ \mathcal{A}$ 有自然同构，并且在该同构下源端点、目标端点与路径权重一致。

证明.

Step 1. 左边 $C \circ (\mathcal{B} \circ \mathcal{A})$ 的事件空间是

$$E_C \times_Y (E_B \times_X E_A).$$

其元素可以写作

$$(e_C, (e_B, e_A)),$$

并满足

$$s_C(e_C) = t_B(e_B), \quad s_B(e_B) = t_A(e_A).$$

Step 2. 右边 $(C \circ \mathcal{B}) \circ \mathcal{A}$ 的事件空间是

$$(E_C \times_Y E_B) \times_X E_A.$$

其元素可以写作

$$((e_C, e_B), e_A),$$

并满足同样的两个匹配条件。

Step 3. 定义映射

$$\Phi : E_C \times_Y (E_B \times_X E_A) \rightarrow (E_C \times_Y E_B) \times_X E_A$$

为

$$\Phi(e_C, (e_B, e_A)) = ((e_C, e_B), e_A).$$

它只是改变括号，不改变三个事件。

Step 4. 反向映射为

$$\Psi((e_C, e_B), e_A) = (e_C, (e_B, e_A)).$$

显然 Φ 与 Ψ 互为逆映射。因此两个事件空间自然双射。

Step 5. 左边复合对应的源端点是 $s_A(e_A)$ ，右边也是 $s_A(e_A)$ ；左边的目标端点是 $t_C(e_C)$ ，右边也是 $t_C(e_C)$ 。

Step 6. 左边路径权重为

$$w_C(e_C) \otimes (w_B(e_B) \otimes w_A(e_A)).$$

右边路径权重为

$$(w_C(e_C) \otimes w_B(e_B)) \otimes w_A(e_A).$$

由半环乘法结合律，两者相等。

Step 7. 因此两个复合对应自然双射下端点与权重完全一致。

□

推论 5.1: 多步路径复合无歧义

给定有限可复合对应链

$$\mathcal{A}_1 : X_0 \rightsquigarrow X_1, \mathcal{A}_2 : X_1 \rightsquigarrow X_2, \dots, \mathcal{A}_m : X_{m-1} \rightsquigarrow X_m,$$

其复合

$$\mathcal{A}_m \circ \dots \circ \mathcal{A}_1$$

在自然同构意义下与加括号方式无关。其事件可以看作长度 m 的路径

$$(e_m, e_{m-1}, \dots, e_1)$$

满足

$$s_{i+1}(e_{i+1}) = t_i(e_i), \quad i = 1, \dots, m-1,$$

权重为

$$w_m(e_m) \otimes \dots \otimes w_1(e_1).$$

证明.

Step 1. $m = 2$ 时这是对复合定义。

Step 2. $m = 3$ 时由结合律定理给出。

Step 3. 一般情形可对 m 归纳。每次改变括号只是在相邻的三段复合中应用结合律自然同构。

Step 4. 这些同构都只改变括号，不改变事件序列本身。因此最终得到的路径空间可以统一写成满足相邻匹配条件的 m 元组。

□

5.11 算子分解：拉回、乘权、推出

对应不仅可以被看作事件结构，也可以作用在对象空间上的数据上。这个观点把本章的三步复合与第三章的半模算子联系起来。

设 $\mathcal{A} = (X \xleftarrow{s} E \xrightarrow{t} Y, w)$ 。给定函数 $u : X \rightarrow \mathbb{K}$ ，先沿 s 拉回到事件空间：

$$s^*u : E \rightarrow \mathbb{K}, \quad (s^*u)(e) = u(s(e)).$$

再乘上事件权重：

$$(W_A s^*u)(e) = w(e) \otimes u(s(e)).$$

最后沿 t 推出到 Y ：

$$(t! W_A s^*u)(y) = \bigoplus_{e:t(e)=y} w(e) \otimes u(s(e)).$$

定义 5.9: 对应诱导的算子

加权对应 $\mathcal{A} = (X \xleftarrow{s} E \xrightarrow{t} Y, w)$ 诱导半模同态

$$T_{\mathcal{A}} : \mathbb{K}^X \rightarrow \mathbb{K}^Y$$

定义为

$$T_{\mathcal{A}} = t! \circ W_A \circ s^*,$$

即

$$(T_{\mathcal{A}}u)(y) = \bigoplus_{e:t(e)=y} w(e) \otimes u(s(e)).$$

定理 5.4: 复合对应诱导算子复合

若 $\mathcal{A} : X \rightsquigarrow Y$ 与 $\mathcal{B} : Y \rightsquigarrow Z$ 可复合，则

$$T_{\mathcal{B} \circ \mathcal{A}} = T_{\mathcal{B}} \circ T_{\mathcal{A}}.$$

证明.

Step 1. 固定 $u \in \mathbb{K}^X$ 与 $z \in Z$ 。先看左边：

$$(T_{\mathcal{B} \circ \mathcal{A}}u)(z) = \bigoplus_{(e_B, e_A): t_B(e_B)=z, s_B(e_B)=t_A(e_A)} w_B(e_B) \otimes w_A(e_A) \otimes u(s_A(e_A)).$$

Step 2. 按中间点 $y = s_B(e_B) = t_A(e_A)$ 分组，得

$$= \bigoplus_{y \in Y} \bigoplus_{\substack{e_B: s_B(e_B)=y, t_B(e_B)=z \\ e_A: t_A(e_A)=y}} w_B(e_B) \otimes w_A(e_A) \otimes u(s_A(e_A)).$$

Step 3. 对固定 e_B ，内层关于 e_A 的部分为

$$\bigoplus_{e_A: t_A(e_A)=s_B(e_B)} w_A(e_A) \otimes u(s_A(e_A)) = (T_{\mathcal{A}}u)(s_B(e_B)).$$

Step 4. 因此左边等于

$$\bigoplus_{e_B: t_B(e_B)=z} w_B(e_B) \otimes (T_{\mathcal{A}}u)(s_B(e_B)).$$

Step 5. 这正是

$$(T_{\mathcal{B}}(T_{\mathcal{A}}u))(z).$$

Step 6. 对所有 u, z 成立，因此两个算子相等。

□

注记

这个定理给出了与矩阵影子定理等价但更结构化的表述。矩阵影子定理是在标准基下看 $T_{\mathcal{A}}$ 的矩阵；本定理直接说明对应本身作为算子可以复合。

5.12 复合之后是否要压缩

定义 $\mathcal{B} \circ \mathcal{A}$ 时，我们保留了完整二步路径空间 $E_B \times_Y E_A$ 。这通常比矩阵影子大，但携带的信息更多。实际计算中，有时需要保留全部路径，有时需要压缩。

定义 5.10: 端点压缩

复合对应 $\mathcal{B} \circ \mathcal{A}$ 的端点压缩，是把所有具有相同端点 (x, z) 的二步路径合并为一个规范事件。压缩后的事件空间是

$$\text{supp} M(\mathcal{B} \circ \mathcal{A}) \subseteq Z \times X$$

或直接取 $Z \times X$ ，其权重为复合矩阵影子的条目。

端点压缩正是矩阵语言默认做的事情。它适合只关心线性算子值的场景，却会丢失路径来源。

定义 5.11: 结构压缩

设 $\sim$ 是二步路径空间 $E_B \times_Y E_A$ 上的等价关系。如果 $\sim$ 保持源端点、目标端点，并且每个等价类上的权重可以用一个聚合规则表示，则可以把复合事件空间替换为商空间

$$(E_B \times_Y E_A) / \sim.$$

这称为结构压缩。

结构压缩比端点压缩更细。它不一定把所有同端点路径都合并，而是按生成规则、路径类型、局部几何、对称性或硬件位置进行合并。

例 5.3: 卷积中的结构压缩

在一维卷积中，事件可以写成

$$e = (x, r), \quad t(e) = x + r.$$

两层卷积复合后的二步路径由

$$(x, r_1, r_2)$$

给出，目标点是 $x + r_1 + r_2$ 。矩阵端点压缩只保留 $(x, x + r_1 + r_2)$ ，而结构压缩可以保留总

偏移

$$r = r_1 + r_2$$

以及不同分解 $r = r_1 + r_2$ 的聚合权重。这就是卷积核复合，而不是完整矩阵乘法。

硬件解释

端点压缩对应 PUSH_REDUCE：按 (x, z) 聚合。结构压缩对应 QUOTIENT：按更高层的等价类聚合。CoPU 不能只支持端点压缩，否则它会退化成稀疏矩阵乘法器；真正区别在于它能够按事件结构做商压缩。

5.13 复合成本与矩阵展开成本

现在可以明确看到，复合成本不应首先由矩阵维度估计，而应由中间纤维大小估计。

定义 5.12: 纤维复合成本

两个对应 $\mathcal{A} : X \rightsquigarrow Y$ 与 $\mathcal{B} : Y \rightsquigarrow Z$ 的原始复合成本定义为二步路径数

$$\text{Cost}_{\text{raw}}(\mathcal{B}, \mathcal{A}) = |E_B \times_Y E_A|.$$

有限情形下，

$$\text{Cost}_{\text{raw}}(\mathcal{B}, \mathcal{A}) = \sum_{y \in Y} |s_B^{-1}(y)| |t_A^{-1}(y)|.$$

若 $\mathcal{A}$ 与 $\mathcal{B}$ 是完全展开的 dense 矩阵对应，并且

$$|X| = n, \quad |Y| = m, \quad |Z| = p,$$

则对每个 $y \in Y$ ，有

$$|t_A^{-1}(y)| = n, \quad |s_B^{-1}(y)| = p.$$

所以

$$\text{Cost}_{\text{raw}} = mnp,$$

这就是普通矩阵乘法的乘加级别。

但若事件空间稀疏或有结构，则成本由实际纤维大小决定，而不是由 mnp 决定。这是对语言在硬件上可能产生优势的第一处。

命题 5.4: 稀疏纤维成本上界

设存在常数 d_A, d_B ，使得对所有 $y \in Y$ ，

$$|t_A^{-1}(y)| \leq d_A, \quad |s_B^{-1}(y)| \leq d_B.$$

则

$$\text{Cost}_{\text{raw}}(\mathcal{B}, \mathcal{A}) \leq |Y| d_A d_B.$$

证明.

Step 1. 由纤维复合成本公式，

$$\text{Cost}_{\text{raw}} = \sum_{y \in Y} |s_B^{-1}(y)| |t_A^{-1}(y)|.$$

Step 2. 对每个 y , 假设给出

$$|s_B^{-1}(y)| \leq d_B, \quad |t_A^{-1}(y)| \leq d_A.$$

因此

$$|s_B^{-1}(y)| |t_A^{-1}(y)| \leq d_B d_A.$$

Step 3. 对 $y \in Y$ 求和, 得到

$$\text{Cost}_{\text{raw}} \leq \sum_{y \in Y} d_B d_A = |Y| d_A d_B.$$

□

注记

这个上界说明：如果每个中间点只接收和发出少量事件，那么复合复杂度随 $|Y|$ 线性增长。矩阵语言若先展开成 $Z \times X$ 的表，再做通用矩阵乘法，就会掩盖这种纤维稀疏性。

5.14 CoPU 中的复合语义

本章定义可以直接翻译成最小硬件语义。设事件存储格式为

$$e = (s(e), t(e), w(e), \text{tag}(e)).$$

复合 $\mathcal{B} \circ \mathcal{A}$ 的数据流如下。

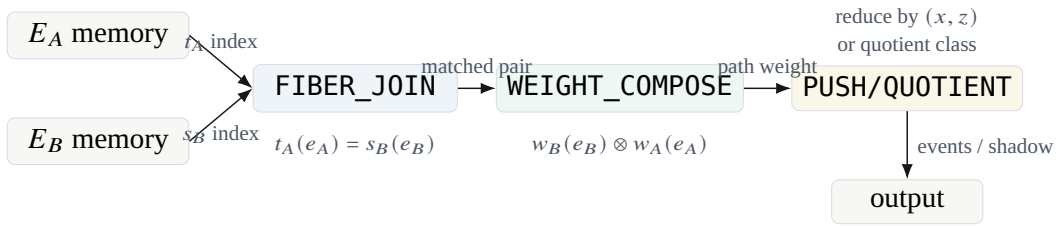


图 5.6 CoPU 中对应复合的最小数据流。

安排

从硬件角度看，本章定义给出四层语义：

1. FIBER_JOIN: 根据 $t_A(e_A) = s_B(e_B)$ 做事件匹配；
2. WEIGHT_COMPOSE: 计算路径权重 $w_B \otimes w_A$ ；
3. PUSH_REDUCE: 按端点 (x, z) 聚合，得到矩阵影子；
4. QUOTIENT: 按结构等价关系聚合，保留比矩阵影子更细的结构。

前三项已经足以实现矩阵乘法、关系复合和图消息传播；第四项是超越矩阵展开的关键。

5.15 本章小结

本章建立了对应语言中的核心乘法。矩阵乘法被拆解为三个几何动作：沿中间对象做纤维拉回，沿路径做局部权重合成，再沿新端点推出并聚合。我们证明了复合的矩阵影子正是半环矩阵乘法，证明了单位律和结合律，并说明对应诱导的算子满足复合律。

从现在开始，有限矩阵不再是基本对象，而是加权对应在端点纤维上聚合后的影子。下一章将把这一点整理成矩阵嵌入定理：矩阵范畴可以完全嵌入对应范畴，而对应范畴保留了矩阵展开之前被遗忘的事件结构。

练习 5.1: 纤维积计算

设 $Y = \{1, 2, 3\}$ ，并且

$$|t_A^{-1}(1)| = 2, \quad |t_A^{-1}(2)| = 0, \quad |t_A^{-1}(3)| = 4,$$

$$|s_B^{-1}(1)| = 3, \quad |s_B^{-1}(2)| = 5, \quad |s_B^{-1}(3)| = 1.$$

计算 $|E_B \times_Y E_A|$ 。

练习 5.2: 函数图的单位律

把函数 $f: X \rightarrow Y$ 看成单位权重对应。直接写出 $1_Y \circ f$ 的事件空间，并构造它与 X 的自然双射。

练习 5.3: 矩阵影子检验

取 $X = \{x_1, x_2\}$, $Y = \{y_1, y_2\}$, $Z = \{z\}$ 。给定若干事件及其权重，手工列出 $E_B \times_Y E_A$ ，并验证

$$M(\mathcal{B} \circ \mathcal{A}) = M(\mathcal{B})M(\mathcal{A}).$$

矩阵语言的嵌入与正规化

前面几章已经建立了加权对应与纤维复合。本章要回答一个必须严肃处理的问题：如果对应语言的目标是替代矩阵语言，那么矩阵语言到底以什么方式保留在新语言中？

这个问题不能只用一句“矩阵是对应的特例”带过。原因在于，矩阵乘法会把中间指标求和掉；而对应复合首先产生的是二步路径空间。也就是说，矩阵语言在每一次乘法之后都会自动做一次聚合，而对应语言可以选择保留路径事件，也可以在需要时聚合。因此，矩阵与对应之间不是简单的一一替换关系，而是包含三个层次：

$$\text{矩阵条目} \longleftrightarrow \text{矩阵化对应} \longleftrightarrow \text{一般事件对应}.$$

本章的目标是把这三个层次完全分清楚，并给出一个严格表述：矩阵语言不是被抛弃，而是对应语言中的一个正规化子语言；同时，普通矩阵代数也可以看作一般对应代数在“忘掉事件结构”之后得到的影子商。

安排

本章的证明路线如下。

1. 先把矩阵语言写成一个由有限集合和半环矩阵构成的运算系统。
2. 再定义矩阵到对应的坐标嵌入，把矩阵条目解释为端点对上的规范事件。
3. 然后指出原始对应复合产生的是路径空间，而不是已经聚合后的矩阵条目。
4. 最后引入矩阵化正规化算子，将路径事件按端点聚合，从而精确恢复矩阵乘法。

关键不是把对应硬塞成矩阵，而是说明矩阵乘法到底隐藏了哪些对应操作。

6.1 矩阵语言的朴素形式

我们先把矩阵语言本身写清楚。这里的“矩阵语言”并不需要预设向量空间；只要有有限集合和半环，就可以谈矩阵。这样做的好处是，后面与对应语言比较时，所有对象都在同一个有限组合层面上。

定义 6.1: 有限矩阵语言

设 $\mathbb{K}$ 是一个半环。有限矩阵语言 $\text{Mat}_{\mathbb{K}}$ 由以下数据构成：

1. 对象是有限集合 $X, Y, Z, \dots$;

2. 从 X 到 Y 的态射是一个函数

$$A : Y \times X \longrightarrow \mathbb{K}, \quad (y, x) \longmapsto A_{yx};$$

3. 若 $A : X \rightarrow Y$ 与 $B : Y \rightarrow Z$, 则复合 $B \odot A : X \rightarrow Z$ 定义为

$$(B \odot A)_{zx} = \bigoplus_{y \in Y} B_{zy} \otimes A_{yx};$$

4. X 上的单位态射是 Kronecker 矩阵 $I_X : X \rightarrow X$, 其条目为

$$(I_X)_{x'x} = \begin{cases} 1, & x' = x, \\ 0, & x' \neq x. \end{cases}$$

这里 $\oplus$ 表示半环加法, $\otimes$ 表示半环乘法。

这个定义强调了一个点: 矩阵不是必须附着在线性空间上。只要 X 是输入索引集合, Y 是输出索引集合, 矩阵就可以被看成一个 $Y \times X$ 上的权重函数。矩阵乘法则是沿中间集合 Y 的有限聚合。

引理 6.1: 矩阵单位律

设 $A : X \rightarrow Y$ 是一个 $\mathbb{K}$ -矩阵。则

$$A \odot I_X = A, \quad I_Y \odot A = A.$$

证明.

Step 1. 先证明右单位律。固定 $y \in Y$ 与 $x \in X$ 。根据矩阵乘法定义,

$$(A \odot I_X)_{yx} = \bigoplus_{x' \in X} A_{yx'} \otimes (I_X)_{x'x}.$$

Step 2. 当 $x' = x$ 时, $(I_X)_{x'x} = 1$, 对应项为

$$A_{yx} \otimes 1 = A_{yx}.$$

Step 3. 当 $x' \neq x$ 时, $(I_X)_{x'x} = 0$, 对应项为

$$A_{yx'} \otimes 0 = 0.$$

Step 4. 因此整个有限和只有 $x' = x$ 这一项可能贡献非零值, 故

$$(A \odot I_X)_{yx} = A_{yx}.$$

由于 x, y 任意, 得到 $A \odot I_X = A$ 。

Step 5. 左单位律同理。固定 y, x , 有

$$(I_Y \odot A)_{yx} = \bigoplus_{y' \in Y} (I_Y)_{yy'} \otimes A_{y'x}.$$

只有 $y' = y$ 时单位矩阵条目等于 1, 其余项均为 0, 所以结果仍为 A_{yx} 。

□

矩阵乘法的结合律是后面所有嵌入讨论的代数基础。为了不留下黑箱，我们把有限和换序的步骤写出来。

引理 6.2: 有限和的换序

设 I, J 是有限集合, $a_{ij} \in \mathbb{K}$ 。则

$$\bigoplus_{i \in I} \bigoplus_{j \in J} a_{ij} = \bigoplus_{(i,j) \in I \times J} a_{ij} = \bigoplus_{j \in J} \bigoplus_{i \in I} a_{ij}.$$

证明.

Step 1. 半环加法 $\oplus$ 构成交换幺半群。因此有限多个元素的聚合值不依赖于列举顺序。

Step 2. 左边是先按 i 分组,再在每个组内按 j 聚合。它列举的元素正是所有 a_{ij} ,其中 $(i, j) \in I \times J$ 。

Step 3. 中间表达式是直接在笛卡尔积 $I \times J$ 上聚合全部元素。

Step 4. 右边只是换成先按 j 分组,再按 i 聚合。由于有限聚合与顺序无关,三者相等。

□

命题 6.1: 矩阵乘法结合律

设

$$A : X \rightarrow Y, \quad B : Y \rightarrow Z, \quad C : Z \rightarrow W$$

是三个 $\mathbb{K}$ -矩阵。则

$$C \odot (B \odot A) = (C \odot B) \odot A.$$

证明.

Step 1. 固定 $w \in W$ 与 $x \in X$ 。先展开左边:

$$\begin{aligned} (C \odot (B \odot A))_{wx} &= \bigoplus_{z \in Z} C_{wz} \otimes (B \odot A)_{zx} \\ &= \bigoplus_{z \in Z} C_{wz} \otimes \left(\bigoplus_{y \in Y} B_{zy} \otimes A_{yx} \right). \end{aligned}$$

Step 2. 使用乘法对加法的分配律,把 C_{wz} 分配进去,得到

$$\bigoplus_{z \in Z} \bigoplus_{y \in Y} C_{wz} \otimes B_{zy} \otimes A_{yx}.$$

Step 3. 由有限和换序,这等于

$$\bigoplus_{y \in Y} \bigoplus_{z \in Z} C_{wz} \otimes B_{zy} \otimes A_{yx}.$$

Step 4. 再利用半环乘法结合律,将前两项先合成:

$$\bigoplus_{y \in Y} \left(\bigoplus_{z \in Z} C_{wz} \otimes B_{zy} \right) \otimes A_{yx}.$$

Step 5. 括号内正是 $(C \odot B)_{wy}$ 。因此上式等于

$$\bigoplus_{y \in Y} (C \odot B)_{wy} \otimes A_{yx} = ((C \odot B) \odot A)_{wx}.$$

Step 6. 对所有 w, x 成立, 所以两个矩阵相等。

□

说明

这个证明值得保留。矩阵乘法结合律看似是线性代数中的基础事实, 但在对应语言中, 它会变成“三步路径空间的两种括号方式存在典范双射”。代数上的换序, 对应于组合结构上的重新分组。

6.2 从矩阵条目到规范事件

现在把一个矩阵变成一个对应。最直接的办法是: 每一个端点对 (x, y) 放置一个规范事件, 其权重就是矩阵条目 A_{yx} 。这给出的是完整坐标嵌入。它可能包含很多零权重事件, 但在数学上最干净。

定义 6.2: 完整坐标对应

设 $A : X \rightarrow Y$ 是一个 $\mathbb{K}$ -矩阵, 即 $A \in \mathbb{K}^{Y \times X}$ 。定义它的完整坐标对应

$$\iota(A) : X \rightsquigarrow Y$$

如下:

$$E_A = X \times Y, \quad s_A(x, y) = x, \quad t_A(x, y) = y, \quad w_A(x, y) = A_{yx}.$$

这里 ι 表示 coordinate inclusion, 即把矩阵写成规范对应。

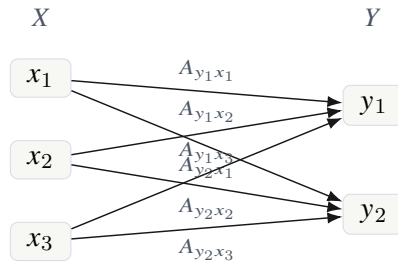


图 6.1 完整坐标对应: 每个端点对 (x, y) 有一个规范事件, 权重为矩阵条目 A_{yx} 。

引理 6.3: 完整坐标对应的矩阵影子

对任意矩阵 $A : X \rightarrow Y$, 有

$$M(\iota(A)) = A.$$

证明.

Step 1. 固定 $x \in X$ 与 $y \in Y$ 。根据矩阵影子的定义,

$$M(\iota(A))_{yx} = \bigoplus_{e \in E_A: s_A(e)=x, t_A(e)=y} w_A(e).$$

Step 2. 在完整坐标对应中, 事件空间为 $X \times Y$ 。满足 $s_A(e) = x$ 且 $t_A(e) = y$ 的事件只有一个, 即 $e = (x, y)$ 。

Step 3. 因此

$$M(\iota(A))_{yx} = w_A(x, y) = A_{yx}.$$

Step 4. 对所有 x, y 成立, 故 $M(\iota(A)) = A$ 。

□

实际计算中, 我们当然不愿意存储所有零权重事件。因此还可以定义一个支撑版本。但支撑版本依赖于“哪些元素等于零”的判定, 在抽象半环中有时不如完整版本稳妥。

定义 6.3: 支撑坐标对应

若可以判定 $A_{yx} = 0$ 与否, 则定义矩阵 $A : X \rightarrow Y$ 的支撑坐标对应 $\iota_{\text{supp}}(A)$ 为

$$E_A^{\text{supp}} = \{(x, y) \in X \times Y : A_{yx} \neq 0\},$$

端点映射仍为

$$s_A(x, y) = x, \quad t_A(x, y) = y,$$

权重为

$$w_A(x, y) = A_{yx}.$$

注记

完整坐标对应适合证明定理, 因为它不需要讨论零判定。支撑坐标对应适合硬件和算法, 因为零权重事件不需要存储。二者的矩阵影子相同。本章所有严格嵌入命题默认使用完整坐标对应; 当讨论存储复杂度时, 再使用支撑版本。

6.3 原始复合为什么不是矩阵乘法本身

矩阵乘法把中间指标 y 求和掉。对应复合则先保留经过 y 的路径事件。这一差异是本章的关键。

设

$$A : X \rightarrow Y, \quad B : Y \rightarrow Z.$$

对应嵌入后有

$$\iota(A) : X \leftarrow X \times Y \rightarrow Y, \quad \iota(B) : Y \leftarrow Y \times Z \rightarrow Z.$$

它们的原始对应复合的事件空间为

$$(Y \times Z) \times_Y (X \times Y).$$

一个复合事件由一对事件组成:

$$(y, z) \in Y \times Z, \quad (x, y') \in X \times Y,$$

并要求中间端点匹配:

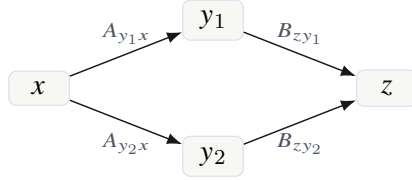
$$y = y'.$$

所以复合事件等价于三元组

$$(x, y, z) \in X \times Y \times Z.$$

其权重为

$$B_{zy} \otimes A_{yx}.$$



原始对应复合保留两条路径 (x, y_1, z) 与 (x, y_2, z)

图 6.2 矩阵乘法会把所有经过 Y 的路径聚合到同一个条目 (z, x) ；原始对应复合则先把这些路径作为事件保留下来。

命题 6.2: 坐标对应复合的路径描述

设 $A : X \rightarrow Y$ 与 $B : Y \rightarrow Z$ 是两个矩阵。则原始对应复合

$$\iota(B) \circ \iota(A) : X \rightsquigarrow Z$$

的事件空间自然同构于 $X \times Y \times Z$ 。在该同构下，事件 (x, y, z) 的源端点为 x ，目标端点为 z ，权重为

$$B_{zy} \otimes A_{yx}.$$

证明.

Step 1. $\iota(A)$ 的事件空间为 $E_A = X \times Y$ ， $\iota(B)$ 的事件空间为 $E_B = Y \times Z$ 。

Step 2. 对应复合的事件空间是纤维积

$$E_B \times_Y E_A = \{((y, z), (x, y')) : s_B(y, z) = t_A(x, y')\}.$$

Step 3. 由于 $s_B(y, z) = y$ 且 $t_A(x, y') = y'$ ，匹配条件就是 $y = y'$ 。

Step 4. 因此每个复合事件唯一写成

$$((y, z), (x, y)),$$

它与三元组 (x, y, z) 一一对应。

Step 5. 复合源端点来自前一步事件 (x, y) 的源端点，即 x ；复合目标端点来自后一步事件 (y, z) 的目标端点，即 z 。

Step 6. 权重按路径合成为

$$w_B(y, z) \otimes w_A(x, y) = B_{zy} \otimes A_{yx}.$$

□

现在可以看出： $\iota(B) \circ \iota(A)$ 通常不是 $\iota(B \odot A)$ 。前者有 $X \times Y \times Z$ 个规范路径事件，后者只有 $X \times Z$ 个规范端点事件。二者的矩阵影子相同，但事件结构不同。

例 6.1: 两条路径与一个矩阵条目

取 $X = \{x\}$, $Y = \{y_1, y_2\}$, $Z = \{z\}$, 在自然数半环 $\mathbb{N}$ 上令所有相关矩阵条目都等于 1。则

$$(B \odot A)_{zx} = 1 \cdot 1 + 1 \cdot 1 = 2.$$

坐标对应复合 $\iota(B) \circ \iota(A)$ 有两条从 x 到 z 的路径事件, 权重分别为 1 与 1。而 $\iota(B \odot A)$ 只有一条从 x 到 z 的规范事件, 权重为 2。

边界

这里不能说 $\iota(B) \circ \iota(A) = \iota(B \odot A)$ 。正确说法是: 它们有相同矩阵影子。矩阵乘法等于“先做路径复合, 再按端点聚合”。如果忽略这一点, 就会把对应语言中最重要的事件层结构删掉。

6.4 矩阵化正规化算子

上一节说明, 原始对应复合保留路径, 而矩阵乘法把路径按端点聚合。为了精确恢复矩阵语言, 我们引入一个正规化算子: 它把任意对应替换成具有同样矩阵影子的规范坐标对应。

定义 6.4: 矩阵化正规化

设 $\mathcal{A} : X \rightsquigarrow Y$ 是一个加权对应。定义它的矩阵化正规化为

$$N(\mathcal{A}) := \iota(M(\mathcal{A})).$$

也就是说, 先取矩阵影子 $M(\mathcal{A})$, 再把这个矩阵重新写成完整坐标对应。

展开来看, $N(\mathcal{A})$ 的事件空间是 $X \times Y$, 且从 x 到 y 的规范事件权重为

$$M(\mathcal{A})_{yx} = \bigoplus_{e: s(e)=x, t(e)=y} w(e).$$

因此, N 的作用就是: 把所有具有相同端点的事件聚合为一个规范事件。

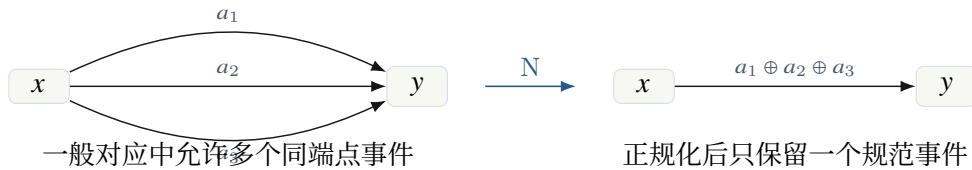


图 6.3 矩阵化正规化把同一端点纤维中的事件权重聚合成一个矩阵条目。

命题 6.3: 正规化保持矩阵影子

任意加权对应 $\mathcal{A} : X \rightsquigarrow Y$ 满足

$$M(N(\mathcal{A})) = M(\mathcal{A}).$$

证明.

Step 1. 由定义, $N(\mathcal{A}) = \iota(M(\mathcal{A}))$ 。

Step 2. 对完整坐标对应, 已经证明 $M(\iota(A)) = A$ 。

Step 3. 令 $A = M(\mathcal{A})$, 得到

$$M(N(\mathcal{A})) = M(\iota(M(\mathcal{A}))) = M(\mathcal{A}).$$

□

命题 6.4: 正规化的幂等性

矩阵化正规化算子满足

$$N(N(\mathcal{A})) = N(\mathcal{A})$$

在规范坐标对应意义下成立。

证明.

Step 1. 由正规化定义,

$$N(N(\mathcal{A})) = \iota(M(N(\mathcal{A}))).$$

Step 2. 由命题 6.3,

$$M(N(\mathcal{A})) = M(\mathcal{A}).$$

Step 3. 因此

$$N(N(\mathcal{A})) = \iota(M(\mathcal{A})) = N(\mathcal{A}).$$

□

定义 6.5: 矩阵化对应

称加权对应 $\mathcal{A} : X \rightsquigarrow Y$ 是矩阵化的, 如果存在某个矩阵 $A : X \rightarrow Y$ 使得

$$\mathcal{A} = \iota(A).$$

等价地, $\mathcal{A}$ 的事件空间为 $X \times Y$, 并且每个端点对 (x, y) 恰有一个规范事件。

命题 6.5: 正规化固定矩阵化对应

若 $\mathcal{A}$ 是矩阵化对应, 则

$$N(\mathcal{A}) = \mathcal{A}.$$

证明.

Step 1. 由于 $\mathcal{A}$ 是矩阵化对应, 存在矩阵 A 使得 $\mathcal{A} = \iota(A)$ 。

Step 2. 于是

$$N(\mathcal{A}) = \iota(M(\iota(A))).$$

Step 3. 由引理 6.3, $M(\iota(A)) = A$ 。

Step 4. 因此

$$N(\mathcal{A}) = \iota(A) = \mathcal{A}.$$

□

6.5 正规化复合与矩阵乘法

现在可以精确说出矩阵乘法在对应语言中的含义。它不是单纯的原始对应复合，而是原始复合之后再正规化。

定义 6.6: 正规化复合

设

$$\mathcal{A} : X \rightsquigarrow Y, \quad \mathcal{B} : Y \rightsquigarrow Z$$

是两个加权对应。定义它们的正规化复合为

$$\mathcal{B} \star \mathcal{A} := N(\mathcal{B} \circ \mathcal{A}).$$

这里 $\circ$ 是原始纤维复合， N 是矩阵化正规化。

也就是说， $\star$ 复合分三步：

先做纤维拉回，得到路径事件；

再合成路径权重；

最后按端点纤维聚合，得到规范矩阵化对应。

定理 6.1: 坐标嵌入与正规化复合相容

设 $A : X \rightarrow Y$ 与 $B : Y \rightarrow Z$ 是两个矩阵。则

$$\iota(B) \star \iota(A) = \iota(B \odot A).$$

证明.

Step 1. 根据正规化复合的定义，

$$\iota(B) \star \iota(A) = N(\iota(B) \circ \iota(A)).$$

Step 2. 根据正规化定义，右边等于

$$\iota(M(\iota(B) \circ \iota(A))).$$

Step 3. 由矩阵影子与对应复合相容定理 5.1，

$$M(\iota(B) \circ \iota(A)) = M(\iota(B)) \odot M(\iota(A)).$$

Step 4. 由引理 6.3，

$$M(\iota(B)) = B, \quad M(\iota(A)) = A.$$

Step 5. 所以

$$M(\iota(B) \circ \iota(A)) = B \odot A.$$

Step 6. 代回第二步，得到

$$\iota(B) \star \iota(A) = \iota(B \odot A).$$

□

推论 6.1: 单位矩阵的正规化相容性

对任意有限集合 X , 有

$$\iota(I_X) = \text{id}_X$$

在规范对应意义下成立。并且对任意矩阵 $A : X \rightarrow Y$,

$$\iota(A) \star \iota(I_X) = \iota(A), \quad \iota(I_Y) \star \iota(A) = \iota(A).$$

证明.

Step 1. I_X 的条目只有在 $x' = x$ 时为 1, 否则为 0。完整坐标对应中虽然每个端点对都有事件, 但非对角事件权重为 0。

Step 2. 在支撑坐标对应中, $\iota_{\text{supp}}(I_X)$ 的事件空间正好是对角集合 $\{(x, x) : x \in X\}$, 与单位对应自然相同。

Step 3. 两个单位律由定理 6.1 和矩阵单位律直接得到。

□

定理 6.2: 矩阵语言的正规化嵌入

令 $\text{Corr}_{\mathbb{K}}^N$ 表示如下正规化对应语言:

1. 对象是有限集合;
2. 从 X 到 Y 的态射是矩阵化对应 $\iota(A)$;
3. 复合由正规化复合 $\star$ 给出;
4. 单位态射为 $\iota(I_X)$ 。

则

$$\iota : \text{Mat}_{\mathbb{K}} \longrightarrow \text{Corr}_{\mathbb{K}}^N$$

是一个保持对象、保持单位、保持复合且在每个 Hom 集上双射的嵌入。换言之, 矩阵语言等同于矩阵化对应语言。

证明.

Step 1. 对象层面, ι 把有限集合 X 送到同一个有限集合 X 。

Step 2. 态射层面, 任意矩阵 $A : X \rightarrow Y$ 被送到矩阵化对应 $\iota(A) : X \rightsquigarrow Y$ 。

Step 3. Hom 集上的单射性: 若 $\iota(A) = \iota(A')$, 则取矩阵影子得到

$$A = M(\iota(A)) = M(\iota(A')) = A'.$$

Step 4. Hom 集上的满射性: 任意矩阵化对应按定义都形如 $\iota(A)$, 所以来自某个矩阵。

Step 5. 单位保持由推论 6.1 给出。

Step 6. 复合保持由定理 6.1 给出:

$$\iota(B) \star \iota(A) = \iota(B \odot A).$$

Step 7. 因此矩阵语言被严格嵌入到正规化对应语言中, 并且该嵌入在矩阵化对应子语言上是等同。

□

说明

这个定理给出本讲义需要的精确表述：矩阵语言不是一般对应语言的全部，而是“每次复合后立即正规化”的子语言。普通线性代数可以看作一种极端策略：不保留路径事件，不保留生成历史，只保留端点聚合后的矩阵条目。

6.6 矩阵语言作为影子商

上一节给出了矩阵化子语言。还有另一个同样重要的观点：矩阵语言是一般对应语言在“同矩阵影子”等价关系下的商。

定义 6.7: 影子等价

设 $\mathcal{A}, \mathcal{A}' : X \rightsquigarrow Y$ 是两个加权对应。称它们影子等价，记为

$$\mathcal{A} \sim_M \mathcal{A}',$$

如果

$$M(\mathcal{A}) = M(\mathcal{A}').$$

命题 6.6: 影子等价是等价关系

在固定端点 X, Y 的所有加权对应上， $\sim_M$ 是一个等价关系。

证明.

Step 1. 自反性：任意 $\mathcal{A}$ 都满足 $M(\mathcal{A}) = M(\mathcal{A})$ ，所以 $\mathcal{A} \sim_M \mathcal{A}$ 。

Step 2. 对称性：若 $\mathcal{A} \sim_M \mathcal{A}'$ ，则 $M(\mathcal{A}) = M(\mathcal{A}')$ 。等号对称，所以 $M(\mathcal{A}') = M(\mathcal{A})$ ，即 $\mathcal{A}' \sim_M \mathcal{A}$ 。

Step 3. 传递性：若 $\mathcal{A} \sim_M \mathcal{A}'$ 且 $\mathcal{A}' \sim_M \mathcal{A}''$ ，则

$$M(\mathcal{A}) = M(\mathcal{A}') = M(\mathcal{A}''),$$

所以 $\mathcal{A} \sim_M \mathcal{A}''$ 。

□

如果要用影子等价做商，必须证明复合与等价关系相容。也就是说，更换某个对应为同影子对应，不应改变复合结果的影子。

命题 6.7: 影子等价与复合相容

设

$$\mathcal{A}, \mathcal{A}' : X \rightsquigarrow Y, \quad \mathcal{B}, \mathcal{B}' : Y \rightsquigarrow Z.$$

若

$$\mathcal{A} \sim_M \mathcal{A}', \quad \mathcal{B} \sim_M \mathcal{B}',$$

则

$$\mathcal{B} \circ \mathcal{A} \sim_M \mathcal{B}' \circ \mathcal{A}'.$$

证明.

Step 1. 由矩阵影子与复合相容定理,

$$M(\mathcal{B} \circ \mathcal{A}) = M(\mathcal{B}) \odot M(\mathcal{A}).$$

Step 2. 同理,

$$M(\mathcal{B}' \circ \mathcal{A}') = M(\mathcal{B}') \odot M(\mathcal{A}').$$

Step 3. 由假设, $M(\mathcal{A}) = M(\mathcal{A}')$ 且 $M(\mathcal{B}) = M(\mathcal{B}')$ 。

Step 4. 因此

$$M(\mathcal{B}) \odot M(\mathcal{A}) = M(\mathcal{B}') \odot M(\mathcal{A}').$$

Step 5. 所以

$$M(\mathcal{B} \circ \mathcal{A}) = M(\mathcal{B}' \circ \mathcal{A}'),$$

即二者影子等价。

□

定义 6.8: 影子商语言

定义影子商语言 $\text{Corr}_{\mathbb{K}}/\sim_M$ 如下:

1. 对象是有限集合;
2. 从 X 到 Y 的态射是加权对应的影子等价类 $[\mathcal{A}]_M$;
3. 复合定义为

$$[\mathcal{B}]_M \circ [\mathcal{A}]_M := [\mathcal{B} \circ \mathcal{A}]_M;$$

4. 单位态射为单位对应的等价类。

命题 6.8: 影子商复合良定义

影子商语言中的复合不依赖代表元选择。

证明.

Step 1. 假设 $[\mathcal{A}]_M = [\mathcal{A}']_M$ 且 $[\mathcal{B}]_M = [\mathcal{B}']_M$ 。

Step 2. 这等价于 $\mathcal{A} \sim_M \mathcal{A}'$ 且 $\mathcal{B} \sim_M \mathcal{B}'$ 。

Step 3. 由命题 6.7,

$$\mathcal{B} \circ \mathcal{A} \sim_M \mathcal{B}' \circ \mathcal{A}'.$$

Step 4. 所以

$$[\mathcal{B} \circ \mathcal{A}]_M = [\mathcal{B}' \circ \mathcal{A}']_M.$$

Step 5. 因此商语言中的复合与代表元选择无关。

□

定理 6.3: 矩阵语言等同于影子商

矩阵影子诱导一个同构

$$\overline{M} : \text{Corr}_{\mathbb{K}}/\sim_M \longrightarrow \text{Mat}_{\mathbb{K}}.$$

具体地,

$$\overline{M}([\mathcal{A}]_M) = M(\mathcal{A}).$$

该映射保持对象、保持单位、保持复合, 并且在每个 Hom 集上双射。

证明.

Step 1. 首先证明良定义。若 $[\mathcal{A}]_M = [\mathcal{A}']_M$, 则 $\mathcal{A} \sim_M \mathcal{A}'$, 即 $M(\mathcal{A}) = M(\mathcal{A}')$ 。所以 $\overline{M}$ 不依赖代表元。

Step 2. Hom 集上的单射性: 若

$$\overline{M}([\mathcal{A}]_M) = \overline{M}([\mathcal{A}']_M),$$

则 $M(\mathcal{A}) = M(\mathcal{A}')$, 所以 $\mathcal{A} \sim_M \mathcal{A}'$, 即 $[\mathcal{A}]_M = [\mathcal{A}']_M$ 。

Step 3. Hom 集上的满射性: 给定任意矩阵 $A : X \rightarrow Y$, 取对应 $\iota(A)$ 。则

$$\overline{M}([\iota(A)]_M) = M(\iota(A)) = A.$$

Step 4. 保持单位: 单位对应的矩阵影子是单位矩阵, 这由单位对应定义直接得到。

Step 5. 保持复合:

$$\begin{aligned} \overline{M}([\mathcal{B}]_M \circ [\mathcal{A}]_M) &= \overline{M}([\mathcal{B} \circ \mathcal{A}]_M) \\ &= M(\mathcal{B} \circ \mathcal{A}) \\ &= M(\mathcal{B}) \odot M(\mathcal{A}) \\ &= \overline{M}([\mathcal{B}]_M) \odot \overline{M}([\mathcal{A}]_M). \end{aligned}$$

Step 6. 所以 $\overline{M}$ 是从影子商语言到矩阵语言的同构。

□



矩阵语言可以从两个方向得到: 要么选择矩阵化规范代表, 要么把所有同影子事件结构取商。

图 6.4 矩阵语言与对应语言的关系: 正规化子语言与影子商语言。

6.7 为什么嵌入定理不是换皮

到这里为止, 我们已经说明矩阵语言可以严格嵌入对应语言, 也可以作为影子商恢复。接下来必须说明: 这不是把同一件事换了名字。真正的差异在于, 矩阵语言只保留端点聚合值, 而对应语言保留事件空间、路径来源、生成规则和中间结构。

命题 6.9: 矩阵影子遗忘事件重数

存在不同的加权对应 $\mathcal{A}, \mathcal{A}' : X \rightsquigarrow Y$ ，它们具有相同矩阵影子，但事件空间基数不同。

证明.

Step 1. 取 $X = \{x\}$, $Y = \{y\}$ ，权重半环取自然数半环 $\mathbb{N}$ 。

Step 2. 定义 $\mathcal{A}$ 有一个事件 $e : x \rightarrow y$ ，权重为 2。

Step 3. 定义 $\mathcal{A}'$ 有两个事件 $e_1, e_2 : x \rightarrow y$ ，权重都为 1。

Step 4. 则

$$M(\mathcal{A})_{yx} = 2, \quad M(\mathcal{A}')_{yx} = 1 + 1 = 2.$$

Step 5. 因此 $M(\mathcal{A}) = M(\mathcal{A}')$ 。

Step 6. 但 $|E_{\mathcal{A}}| = 1$, $|E_{\mathcal{A}'}| = 2$ ，事件空间不同，故它们不是同一个对应结构。

□

命题 6.10: 原始路径空间比矩阵条目更细

设 $A : X \rightarrow Y$ 与 $B : Y \rightarrow Z$ 。若存在 $x \in X, z \in Z$ 以及至少两个不同的 $y_1, y_2 \in Y$ ，使得

$$A_{y_1 x}, B_{zy_1}, A_{y_2 x}, B_{zy_2}$$

均非零，则从 x 到 z 的复合路径空间至少含有两个不同事件，而矩阵乘积 $(B \odot A)_{zx}$ 只记录它们的聚合值。

证明.

Step 1. 在坐标对应复合 $\iota(B) \circ \iota(A)$ 中，从 x 到 z 的路径事件由中间点 $y \in Y$ 标记。

Step 2. 对 y_1 ，存在路径事件 (x, y_1, z) ，权重为 $B_{zy_1} \otimes A_{y_1 x}$ 。

Step 3. 对 y_2 ，存在路径事件 (x, y_2, z) ，权重为 $B_{zy_2} \otimes A_{y_2 x}$ 。

Step 4. 因为 $y_1 \neq y_2$ ，两条路径事件不同。

Step 5. 但矩阵乘积条目为

$$(B \odot A)_{zx} = \bigoplus_{y \in Y} B_{zy} \otimes A_{yx},$$

它只保留所有路径权重的聚合结果，无法恢复具体经过哪些中间点。

□

注记

这正是对应语言可能带来结构优势的地方。若后续计算还需要区分路径来源、事件类型、空间局部性、层级关系或硬件路由位置，那么过早矩阵化会丢掉有用信息。矩阵语言的默认策略是“立即聚合”；对应语言允许“延迟聚合”或“选择性聚合”。

6.8 硬件解释：矩阵乘法的隐藏流水线

从芯片设计角度看，本章的数学结论可以翻译成一句话：矩阵乘法并不是单一原语，它隐藏了三个操作。

$$\boxed{\text{fiber join} \longrightarrow \text{weight compose} \longrightarrow \text{endpoint reduce}}$$

传统矩阵乘法把这三步融合成一个 GEMM 或稀疏矩阵乘法内核。对应处理器则把这三步显式化。

硬件解释

若 $A : X \rightarrow Y$ 与 $B : Y \rightarrow Z$ ，那么计算 $B \odot A$ 可以写成以下硬件语义：

1. **Fiber join**: 找到所有满足 $t_A(e_A) = s_B(e_B)$ 的事件对。
2. **Weight compose**: 对每个匹配事件对计算 $w_B(e_B) \otimes w_A(e_A)$ 。
3. **Endpoint reduce**: 对相同外部端点 (x, z) 的路径权重执行 $\oplus$ 聚合。

矩阵芯片默认总是执行完整的 endpoint reduce；CoPU 可以选择保留路径事件，也可以选择局部聚合、分层聚合或延迟聚合。



图 6.5 矩阵乘法的隐藏流水线。对应语言把 join、compose、reduce 三个阶段分开，使硬件可以在不同阶段利用结构。

这个解释对后续 CoPU 架构非常重要。若一个任务的事件结构稀疏，或者中间纤维高度不均匀，或者很多路径在后续步骤中会被再次筛选，那么立即把所有路径聚合成矩阵条目未必是最优策略。对应语言允许我们把“何时聚合”作为算法和硬件共同决定的自由度。

6.9 本章小结

本章完成了矩阵语言与对应语言之间的精确连接。

首先，矩阵可以通过完整坐标嵌入 ι 写成规范对应。其次，原始对应复合产生路径事件空间，而不是直接产生矩阵乘积。第三，矩阵化正规化 $N = \iota \circ M$ 把一般对应按端点聚合为规范对应。第四，在正规化复合

$$B \star A = N(B \circ A)$$

下，坐标嵌入严格保持矩阵乘法：

$$\iota(B) \star \iota(A) = \iota(B \odot A).$$

最后，一般对应取影子商之后也正好恢复矩阵语言：

$$\text{Corr}_{\mathbb{K}} / \sim_M \cong \text{Mat}_{\mathbb{K}}.$$

因此，矩阵语言既是对应语言中的正规化子语言，也是对应语言的影子商。这个结论使后续章节可以安全地进行两件事：一方面继承矩阵代数中的迹、谱、秩；另一方面研究对应语言中矩阵影子看不到事件结构、表达复杂度和硬件优势。

练习 6.1: 检查支撑版本

设 $\mathbb{K}$ 是可以判定零元素的半环。证明支撑坐标对应 $\iota_{\text{supp}}(A)$ 与完整坐标对应 $\iota(A)$ 具有相同矩阵影子。进一步说明，在布尔半环上，支撑坐标对应正好是矩阵非零模式对应的二部图。

练习 6.2: 正规化前后的事件数

设 $|X| = m, |Y| = n, |Z| = p$ ，且 $A : X \rightarrow Y, B : Y \rightarrow Z$ 都是完整 dense 矩阵。计算 $\iota(B) \circ \iota(A)$ 的事件数，并与 $\iota(B \odot A)$ 的事件数比较。解释为什么矩阵乘法会隐藏一个从 mnp 个路径事件到 mp 个端点事件的聚合过程。

迹、闭合路径谱与对应秩

前面几章已经建立了对应的加法、反向、恒等与复合，并证明了矩阵乘法可以看作“先形成路径，再按端点聚合”的影子。本章继续处理矩阵理论中更深的一层结构：迹、幂、谱与秩。

这些概念在矩阵语言中有十分成熟的形式。迹是对角线条目的和， A^k 记录 k 次迭代，谱控制迭代增长，秩刻画一个线性变换是否能通过小的中间空间分解。若对应语言只是把矩阵乘法换一种写法，却不能解释这些概念，那么它就不能真正替代矩阵语言。反过来，如果这些概念都能在事件空间与纤维复合中自然出现，那么我们就获得了一条可靠路线：先在对应层保留结构，再在需要时取矩阵影子。

安排

本章按如下顺序展开。

1. 先定义端点相同的对应，即自对应，并解释闭合事件为何对应矩阵对角线。
2. 再定义对应的幂，把 $\mathcal{A}^k$ 解释为长度为 k 的路径空间。
3. 然后用闭合路径定义迹幂矩 $\text{Tr}(\mathcal{A}^k)$ ，并证明它与矩阵影子的迹幂矩一致。
4. 接着引入路径 zeta 函数，说明在有限复矩阵情形它退化为 $\det(I - uA)^{-1}$ 。
5. 最后讨论对应秩。矩阵秩是“通过小中间空间分解”的影子；对应秩则把中间空间提升为真实的隐藏对象空间。

本章默认对象空间有限。关于谱的部分需要普通加法、乘法、幂级数与行列式，因此在那里默认权重取在 $\mathbb{C}$ ，或至少取在含有 $\mathbb{Q}$ 的交换代数中。关于路径计数与秩的部分仍可在一般交换半环上讨论。

7.1 自对应与闭合事件

矩阵迹的定义看起来很坐标化：把矩阵对角线上的元素相加，得到

$$\text{Tr}(A) = \sum_{x \in X} A_{xx}.$$

但对角线并不是任意选择出来的一组位置。若把 A_{yx} 看成从 x 到 y 的权重，那么对角线条目 A_{xx} 表示“从 x 出发又回到 x ”的权重。换言之，迹不是单纯的数组操作，而是对闭合一步路径的聚合。

为了把这个想法从矩阵中抽出来，首先需要让源空间与目标空间相同。

定义 7.1: 自对应

设 X 是有限对象空间。一个从 X 到自身的加权对应

$$\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} X, w_A)$$

称为 X 上的自对应。

事件 $e \in E_A$ 可以记作

$$s_A(e) \xrightarrow{e} t_A(e).$$

若 $s_A(e) = t_A(e)$, 则称 e 是一个闭合事件, 或长度为 1 的闭合路径。

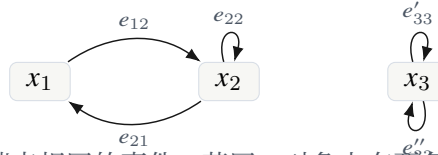
定义 7.2: 对应迹

设 $\mathcal{A} : X \rightsquigarrow X$ 是 X 上的加权自对应。定义它的迹为

$$\text{Tr}(\mathcal{A}) := \bigoplus_{\substack{e \in E_A \\ s_A(e) = t_A(e)}} w_A(e).$$

若不存在闭合事件, 则该聚合为空, 因此 $\text{Tr}(\mathcal{A}) = 0$ 。

这个定义只使用端点映射与权重函数, 不需要坐标表。它保留了一个重要差异: 若同一点 x 上有多条不同类型的闭合事件, 它们在对应该层是不同事件; 取迹时才把这些事件的权重聚合。



闭合事件是端点相同的事件。若同一对象上有两条闭合事件, 它们在事件空间中仍是两条事件, 迹只在最后聚合它们的权重。

图 7.1 自对应中的闭合事件。

例 7.1: 三个半环上的迹

设 $\mathcal{A} : X \rightsquigarrow X$ 。

1. 在布尔半环 $\{0, 1\}$ 上, $\text{Tr}(\mathcal{A}) = 1$ 当且仅当存在至少一个闭合事件。
2. 在计数半环 $\mathbb{N}$ 上, 若所有事件权重为 1, 则 $\text{Tr}(\mathcal{A})$ 是闭合事件的数量。
3. 在实数半环上, $\text{Tr}(\mathcal{A})$ 是所有闭合事件权重的普通求和。

命题 7.1: 对应迹等于矩阵影子迹

设 $\mathcal{A} : X \rightsquigarrow X$ 是一个加权自对应, $M(\mathcal{A})$ 是它的矩阵影子, 则

$$\text{Tr}(\mathcal{A}) = \bigoplus_{x \in X} M(\mathcal{A})_{xx}.$$

特别地, 若 $\mathcal{A} = \iota(A)$ 是矩阵 A 的完整坐标对应, 则

$$\text{Tr}(\mathcal{A}) = \text{Tr}(A).$$

证明.

Step 1. 根据矩阵影子的定义, 对每个 $x \in X$, 有

$$M(\mathcal{A})_{xx} = \bigoplus_{e \in E_A(x,x)} w_A(e),$$

其中

$$E_A(x,x) = \{e \in E_A : s_A(e) = x, t_A(e) = x\}.$$

Step 2. 因此

$$\bigoplus_{x \in X} M(\mathcal{A})_{xx} = \bigoplus_{x \in X} \bigoplus_{e \in E_A(x,x)} w_A(e).$$

Step 3. 集合 $E_A(x,x)$ 随 x 两两不交, 并且它们的并正好是所有闭合事件:

$$\bigsqcup_{x \in X} E_A(x,x) = \{e \in E_A : s_A(e) = t_A(e)\}.$$

有限聚合可以按这组不交分解重排。

Step 4. 于是

$$\bigoplus_{x \in X} M(\mathcal{A})_{xx} = \bigoplus_{\substack{e \in E_A \\ s_A(e)=t_A(e)}} w_A(e) = \text{Tr}(\mathcal{A}).$$

Step 5. 若 $\mathcal{A} = \iota(A)$, 由第六章的 $M(\iota(A)) = A$, 得到

$$\text{Tr}(\mathcal{A}) = \bigoplus_{x \in X} A_{xx},$$

即普通矩阵迹。

□

注记

矩阵语言中的迹把“闭合事件”预先压成对角线条目。对应语言把这一步拆开: 先看哪些事件闭合, 再沿闭合事件集合聚合。这个拆分在后面研究闭合路径、路径谱与硬件 trace unit 时很重要。

7.2 迹的循环性

矩阵迹有一个基本性质: 若 $A : X \rightarrow Y$, $B : Y \rightarrow X$, 则

$$\text{Tr}(BA) = \text{Tr}(AB).$$

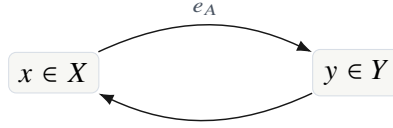
在线性代数中, 这通常通过坐标展开证明。在对应语言中, 这个性质的原因更简单: BA 的闭合二步路径和 AB 的闭合二步路径是同一批循环, 只是选择的起点不同。

命题 7.2: 二步迹的循环性

设 $\mathbb{K}$ 是交换半环, $\mathcal{A} : X \rightsquigarrow Y$, $\mathcal{B} : Y \rightsquigarrow X$ 是两个加权对应。则

$$\text{Tr}(\mathcal{B} \circ \mathcal{A}) = \text{Tr}(\mathcal{A} \circ \mathcal{B}).$$

证明.



同一个二步闭合循环可以从 x 开始看成 $\mathcal{B} \circ \mathcal{A}$ 的闭合路径，也可以从 y 开始看成 $\mathcal{A} \circ \mathcal{B}$ 的闭合路径。

图 7.2 迹循环性的路径解释。

Step 1. 复合 $\mathcal{B} \circ \mathcal{A} : X \rightsquigarrow X$ 的事件是二元组 (e_B, e_A) ，其中

$$s_B(e_B) = t_A(e_A).$$

它是闭合事件当且仅当

$$t_B(e_B) = s_A(e_A).$$

因此闭合事件集合为

$$C_{BA} = \{(e_B, e_A) : s_B(e_B) = t_A(e_A), t_B(e_B) = s_A(e_A)\}.$$

Step 2. 同理， $\mathcal{A} \circ \mathcal{B} : Y \rightsquigarrow Y$ 的闭合事件集合为

$$C_{AB} = \{(e_A, e_B) : s_A(e_A) = t_B(e_B), t_A(e_A) = s_B(e_B)\}.$$

Step 3. 定义映射

$$\Phi : C_{BA} \longrightarrow C_{AB}, \quad \Phi(e_B, e_A) = (e_A, e_B).$$

两个闭合条件只是顺序互换，因此 Φ 确实落在 C_{AB} 中。

Step 4. 映射 Φ 的逆映射就是再次交换两个分量，所以 Φ 是双射。

Step 5. 对应的权重分别为

$$w_{BA}(e_B, e_A) = w_B(e_B)w_A(e_A),$$

和

$$w_{AB}(e_A, e_B) = w_A(e_A)w_B(e_B).$$

由于 $\mathbb{K}$ 交换，二者相等。

Step 6. 因而沿双射 Φ ，闭合路径权重逐项相等。对所有闭合路径聚合，得到

$$\text{Tr}(\mathcal{B} \circ \mathcal{A}) = \text{Tr}(\mathcal{A} \circ \mathcal{B}).$$

□

边界

若权重乘法不交换，则上面的最后一步可能失败。非交换权重下仍可定义迹，但循环性通常需要额外结构，例如代数上的 trace functional 满足 $\tau(ab) = \tau(ba)$ 。本讲义当前主要处理交换半环，因此暂不展开非交换情形。

推论 7.1: 多因子迹的循环性

设 $\mathbb{K}$ 是交换半环, 并设

$$\mathcal{A}_1 : X_0 \rightsquigarrow X_1, \mathcal{A}_2 : X_1 \rightsquigarrow X_2, \dots, \mathcal{A}_m : X_{m-1} \rightsquigarrow X_0$$

可首尾相接。则复合的迹在循环置换下不变。例如

$$\text{Tr}(\mathcal{A}_m \circ \dots \circ \mathcal{A}_1) = \text{Tr}(\mathcal{A}_1 \circ \mathcal{A}_m \circ \dots \circ \mathcal{A}_2).$$

证明. 一个闭合 m 步路径由事件序列 $(e_m, \dots, e_1)$ 给出, 并满足首尾端点闭合。循环置换只是改变这条闭合循环的起点, 不改变事件的循环顺序。由于权重乘法交换, 循环置换后的权重乘积相同。于是闭合路径集合之间存在保权双射, 聚合后迹相等。 $\square$

7.3 对应幂与长度为 k 的路径

矩阵幂 A^k 有两个解释。在线性代数中, 它表示线性变换迭代 k 次; 在图论中, 它记录长度为 k 的路径。对应语言中, 第二个解释是基本的, 第一个解释是取矩阵影子之后得到的。

定义 7.3: 长度为 k 的路径空间

设 $\mathcal{A} : X \rightsquigarrow X$ 是自对应。对 $k \geq 1$, 定义长度为 k 的路径空间

$$P_k(\mathcal{A}) = \{(e_k, \dots, e_1) \in E_A^k : s_A(e_{i+1}) = t_A(e_i), i = 1, \dots, k-1\}.$$

其中路径

$$e_1, e_2, \dots, e_k$$

表示从 $s_A(e_1)$ 出发, 经过 e_1 到达 $t_A(e_1) = s_A(e_2)$, 依次前进, 最后到达 $t_A(e_k)$ 。
定义路径的源、目标与权重为

$$s_k(e_k, \dots, e_1) = s_A(e_1), \quad t_k(e_k, \dots, e_1) = t_A(e_k),$$

$$w_k(e_k, \dots, e_1) = w_A(e_k) \cdots w_A(e_1).$$

注意这里的书写顺序与矩阵乘法保持一致: 先走 e_1 , 再走 e_2 , 最后走 e_k ; 但复合符号中后走的事件写在左边。因此路径事件写作 $(e_k, \dots, e_1)$ 。

定义 7.4: 对应幂

设 $\mathcal{A} : X \rightsquigarrow X$ 。定义

$$\mathcal{A}^1 = \mathcal{A}, \quad \mathcal{A}^{k+1} = \mathcal{A} \circ \mathcal{A}^k.$$

同时定义 $\mathcal{A}^0$ 为恒等对应 id_X 。

定理 7.1: 对应幂的路径解释

对每个 $k \geq 1$, 对应 $\mathcal{A}^k$ 的事件空间自然同构于 $P_k(\mathcal{A})$ 。在这个同构下, 源端点、目标端点与权重分别为上一定义中的 s_k, t_k, w_k 。

证明.

Step 1. 当 $k = 1$ 时, $P_1(\mathcal{A}) = E_A$, 路径源、目标和权重正好就是 s_A, t_A, w_A 。因此结论成立。

Step 2. 假设结论对某个 $k \geq 1$ 成立, 即 $\mathcal{A}^k$ 的事件可看成长度为 k 的路径

$$p = (e_k, \dots, e_1),$$

其目标为 $t_k(p) = t_A(e_k)$ 。

Step 3. 由复合定义, $\mathcal{A}^{k+1} = \mathcal{A} \circ \mathcal{A}^k$ 的事件是二元组 (e_{k+1}, p) , 其中

$$s_A(e_{k+1}) = t_k(p) = t_A(e_k).$$

这正说明 e_{k+1} 可以接在路径 p 后面。

Step 4. 因此 (e_{k+1}, p) 等价于序列

$$(e_{k+1}, e_k, \dots, e_1)$$

并满足

$$s_A(e_{i+1}) = t_A(e_i), \quad i = 1, \dots, k.$$

也就是说, 它正是 $P_{k+1}(\mathcal{A})$ 的一个元素。

Step 5. 这个对应显然可逆: 给定长度 $k+1$ 的路径 $(e_{k+1}, \dots, e_1)$, 取前面的 e_{k+1} 和后面的长度 k 路径 $(e_k, \dots, e_1)$ 即得复合事件。

Step 6. 源端点为路径第一步的源:

$$s_{k+1}(e_{k+1}, \dots, e_1) = s_A(e_1),$$

目标端点为最后一步的目标:

$$t_{k+1}(e_{k+1}, \dots, e_1) = t_A(e_{k+1}).$$

这与复合端点定义一致。

Step 7. 权重由复合定义给出:

$$w_{k+1}(e_{k+1}, p) = w_A(e_{k+1})w_k(p) = w_A(e_{k+1})w_A(e_k) \cdots w_A(e_1).$$

归纳完成。

□



$\mathcal{A}^3$ 的一个事件就是一条长度为 3 的可复合路径 (e_3, e_2, e_1)

图 7.3 对应幂的事件空间是路径空间。

推论 7.2: 矩阵影子的幂相容性

对任意自对应 $\mathcal{A} : X \rightsquigarrow X$ 与任意 $k \geq 0$, 有

$$M(\mathcal{A}^k) = M(\mathcal{A})^k.$$

右边是半环矩阵乘法意义下的矩阵幂。

证明.

Step 1. 当 $k = 0$ 时, $\mathcal{A}^0 = \text{id}_X$, 其矩阵影子是恒等矩阵, 故等式成立。

Step 2. 当 $k = 1$ 时, 等式显然成立。

Step 3. 假设对 k 成立。由复合与矩阵影子相容定理,

$$M(\mathcal{A}^{k+1}) = M(\mathcal{A} \circ \mathcal{A}^k) = M(\mathcal{A})M(\mathcal{A}^k).$$

Step 4. 代入归纳假设 $M(\mathcal{A}^k) = M(\mathcal{A})^k$, 得到

$$M(\mathcal{A}^{k+1}) = M(\mathcal{A})M(\mathcal{A})^k = M(\mathcal{A})^{k+1}.$$

归纳完成。

□

7.4 闭合 k 路径与迹幂矩

矩阵迹 $\text{Tr}(A)$ 只看一步闭合路径。矩阵幂迹 $\text{Tr}(A^k)$ 则看长度为 k 的闭合路径。许多谱信息正是通过这一串数

$$\text{Tr}(A), \text{Tr}(A^2), \text{Tr}(A^3), \dots$$

体现出来的。对应语言中的相应对象是闭合路径权重的聚合。

定义 7.5: 闭合 k 路径与迹幂矩

设 $\mathcal{A} : X \rightsquigarrow X$ 。长度为 k 的路径

$$p = (e_k, \dots, e_1) \in P_k(\mathcal{A})$$

称为闭合的, 如果

$$s_k(p) = t_k(p),$$

即

$$s_A(e_1) = t_A(e_k).$$

定义第 k 个迹幂矩为

$$p_k(\mathcal{A}) := \text{Tr}(\mathcal{A}^k) = \bigoplus_{\substack{p \in P_k(\mathcal{A}) \\ s_k(p) = t_k(p)}} w_k(p).$$

这里 $p_k(\mathcal{A})$ 中的下标 p 只是 moment 的记号, 不应与路径 p 混淆。若担心歧义, 也可写成

$$\tau_k(\mathcal{A}) = \text{Tr}(\mathcal{A}^k).$$

定理 7.2: 迹幂矩与矩阵影子一致

设 $\mathcal{A} : X \rightsquigarrow X$ 是有限自对应。则对所有 $k \geq 0$,

$$\text{Tr}(\mathcal{A}^k) = \text{Tr}(M(\mathcal{A})^k).$$

证明.

Step 1. 由上一节推论,

$$M(\mathcal{A}^k) = M(\mathcal{A})^k.$$

Step 2. 对任意自对应 $C : X \rightsquigarrow X$, 已经证明

$$\text{Tr}(C) = \text{Tr}(M(C)).$$

这里右边表示矩阵影子的对角线聚合。

Step 3. 令 $C = \mathcal{A}^k$, 得到

$$\text{Tr}(\mathcal{A}^k) = \text{Tr}(M(\mathcal{A}^k)).$$

Step 4. 代入第一步, 得

$$\text{Tr}(\mathcal{A}^k) = \text{Tr}(M(\mathcal{A})^k).$$

□

例 7.2: 无权有向图

令 $G = (V, E)$ 是有限有向图, 把每条边看成权重为 1 的事件, 得到计数半环上的自对应

$$\mathcal{A}_G : V \rightsquigarrow V.$$

则

$$\text{Tr}(\mathcal{A}_G^k)$$

等于 G 中长度为 k 的闭合有向游走数量。若取布尔半环, 则它只记录是否存在长度为 k 的闭合游走。

例 7.3: 带权闭合路径

若 $\mathbb{K} = \mathbb{R}$, 且每条事件 e 有权重 $w(e)$, 则

$$\text{Tr}(\mathcal{A}^k)$$

是所有闭合 k 步路径的权重乘积之和:

$$\sum_{x_0=x_k} \sum_{x_0 \xrightarrow{e_1} x_1 \xrightarrow{e_2} \dots \xrightarrow{e_k} x_k} w(e_k) \cdots w(e_1).$$

如果同一端点之间有多条平行事件, 它们在内层求和中分别出现。

硬件解释

闭合路径迹对应一种硬件操作: 枚举或采样返回起点的路径, 并聚合它们的权重。普通矩阵硬件通常通过反复矩阵乘法得到 A^k , 再取对角线; CoPU 可以在事件层维护路径端点和起点标签, 直接检测闭合条件。这并不总是更快, 但它避免了不必要的完整矩阵中间物化。

7.5 路径 zeta 函数

矩阵谱可以从特征值定义，也可以从迹幂矩定义。若 A 的特征值为 $\lambda_1, \dots, \lambda_n$ ，则在复数域上

$$\mathrm{Tr}(A^k) = \lambda_1^k + \dots + \lambda_n^k.$$

因此序列 $\mathrm{Tr}(A^k)$ 是特征值的幂和。对应语言中一般没有预设的线性空间基底，也不应首先谈特征向量；更自然的起点是闭合路径的生成函数。

定义 7.6: 路径 zeta 函数

设 $\mathcal{A} : X \rightsquigarrow X$ 是有限复加权自对应。定义它的路径 zeta 函数为形式幂级数

$$\zeta_{\mathcal{A}}(u) := \exp \left(\sum_{k \geq 1} \frac{u^k}{k} \mathrm{Tr}(\mathcal{A}^k) \right).$$

其中 u 是形式变量。

为什么要在指数前面放 $1/k$ ？原因来自闭合循环的起点选择。一个长度为 k 的周期若没有更小周期，沿它有 k 个可能起点；迹 $\mathrm{Tr}(A^k)$ 会把这些起点都计入。对数 zeta 中的 $1/k$ 正是为了把“带起点的闭合路径”转化为“循环轨道”的自然权重。即使不使用这个几何解释，下面的行列式恒等式也会强迫出现这个系数。

定理 7.3: 任意有限对应的 zeta 行列式公式

设 $\mathcal{A} : X \rightsquigarrow X$ 是有限复加权自对应，并令

$$A = M(\mathcal{A})$$

为它的矩阵影子。则在形式幂级数意义下，

$$\zeta_{\mathcal{A}}(u) = \frac{1}{\det(I - uA)}.$$

证明.

Step 1. 由迹幂矩与矩阵影子一致定理，对所有 $k \geq 1$ ，有

$$\mathrm{Tr}(\mathcal{A}^k) = \mathrm{Tr}(A^k).$$

Step 2. 因此

$$\zeta_{\mathcal{A}}(u) = \exp \left(\sum_{k \geq 1} \frac{u^k}{k} \mathrm{Tr}(A^k) \right).$$

Step 3. 现在证明矩阵恒等式

$$\exp \left(\sum_{k \geq 1} \frac{u^k}{k} \mathrm{Tr}(A^k) \right) = \det(I - uA)^{-1}.$$

为了看清来源，先假设 $|u|$ 足够小，使得级数收敛。形式幂级数情形可由同样代数展开得到。

Step 4. 矩阵对数满足

$$\log(I - uA) = - \sum_{k \geq 1} \frac{u^k}{k} A^k.$$

这是标量公式 $\log(1 - z) = - \sum_{k \geq 1} z^k / k$ 在矩阵代数中的展开。

Step 5. 对两边取迹，利用迹的线性性，得到

$$\text{Tr} \log(I - uA) = - \sum_{k \geq 1} \frac{u^k}{k} \text{Tr}(A^k).$$

Step 6. 对任意可逆矩阵 B ，有

$$\text{Tr} \log B = \log \det B.$$

在可对角化情形，这是因为特征值取对数后求和等于行列式取对数；一般情形可通过 Jordan 标准形或解析延拓得到。令 $B = I - uA$ ，得

$$\log \det(I - uA) = - \sum_{k \geq 1} \frac{u^k}{k} \text{Tr}(A^k).$$

Step 7. 移项并取指数，得到

$$\exp \left(\sum_{k \geq 1} \frac{u^k}{k} \text{Tr}(A^k) \right) = \det(I - uA)^{-1}.$$

Step 8. 结合第二步，即得

$$\zeta_{\mathcal{A}}(u) = \det(I - uM(\mathcal{A}))^{-1}.$$

□

注记

这个定理有两层含义。第一，路径 zeta 没有背离矩阵谱理论；当取矩阵影子时，它正好给出 $\det(I - uA)$ 。第二，zeta 的定义本身只依赖闭合路径，因此它可以在比矩阵更细的事件层被解释。矩阵行列式是它的坐标影子，而闭合路径生成函数是它的结构来源。

推论 7.3: 坐标对应情形

若 A 是有限复矩阵， $\mathcal{A} = \iota(A)$ ，则

$$\zeta_{\mathcal{A}}(u) = \frac{1}{\det(I - uA)}.$$

证明. 由 $M(\iota(A)) = A$ 直接代入上一定理。

□

7.6 路径谱与闭合路径增长

严格说，zeta 函数给出的不是单个数，而是一整套闭合路径信息。若 A 是复矩阵， $\det(I - uA)$ 的零点是 $u = \lambda^{-1}$ ，其中 λ 遍历 A 的非零特征值。因此路径 zeta 在矩阵情形确实包含普通谱。

对应语言中可以从两个层面谈“谱”。第一个层面是矩阵影子谱：取 $M(\mathcal{A})$ 后使用普通矩阵谱。第二个层面是路径谱：直接研究闭合路径迹幂矩与 zeta 函数。两者由上节定理相容，但关注点不同。

定义 7.7: 矩阵影子谱

设 $\mathcal{A} : X \rightsquigarrow X$ 是有限复加权自对应。定义它的矩阵影子谱为

$$\text{Spec}_M(\mathcal{A}) := \text{Spec}(M(\mathcal{A})).$$

也就是说, $\text{Spec}_M(\mathcal{A})$ 是矩阵影子 $M(\mathcal{A})$ 的普通特征值集合, 按代数重数计。

定义 7.8: 闭合路径增长率

设 $\mathcal{A} : X \rightsquigarrow X$ 是有限复加权自对应。若迹幂矩不是最终全零, 定义迹增长率为

$$\rho_{\text{Tr}}(\mathcal{A}) := \limsup_{k \rightarrow \infty} |\text{Tr}(\mathcal{A}^k)|^{1/k}.$$

若 $\text{Tr}(\mathcal{A}^k) = 0$ 对所有充分大的 k 成立, 则定义 $\rho_{\text{Tr}}(\mathcal{A}) = 0$ 。

命题 7.3: 迹增长率受谱半径控制

设 $\mathcal{A} : X \rightsquigarrow X$ 是有限复加权自对应, 令

$$A = M(\mathcal{A}), \quad \rho(A) = \max\{|\lambda| : \lambda \in \text{Spec}(A)\}.$$

则

$$\rho_{\text{Tr}}(\mathcal{A}) \leq \rho(A).$$

证明.

Step 1. 由迹幂矩与矩阵影子一致,

$$\text{Tr}(\mathcal{A}^k) = \text{Tr}(A^k).$$

Step 2. 设 A 的特征值为 $\lambda_1, \dots, \lambda_n$, 按代数重数计。无论 A 是否可对角化, 都有

$$\text{Tr}(A^k) = \lambda_1^k + \dots + \lambda_n^k.$$

这是因为迹等于特征值之和, 应用于矩阵 A^k 即可。

Step 3. 因此

$$|\text{Tr}(A^k)| \leq \sum_{i=1}^n |\lambda_i|^k \leq n\rho(A)^k.$$

Step 4. 两边取 $1/k$ 次方并令 $k \rightarrow \infty$ 的上极限, 得到

$$\rho_{\text{Tr}}(\mathcal{A}) \leq \rho(A).$$

□

边界

上式一般不能改成等号。若不同特征值的幂和发生抵消, $\text{Tr}(A^k)$ 的增长率可能低于谱半径。例如特征值 1 与 -1 的贡献在奇偶 k 上有抵消。因此闭合路径增长率只是一种 trace-visible 的谱量, 不等同于完整谱半径。

命题 7.4: 非负不可约情形的直觉

设 A 是非负矩阵，并且底层有向图强连通且有某个闭合路径长度集合的最大公因数为 1。在这种常见情形下， $\text{Tr}(A^k)$ 的指数增长率等于 Perron-Frobenius 特征值。

证明. 这是 Perron-Frobenius 理论的标准结论之一。直观上，强连通和非周期性排除了主特征值之间的周期性抵消；非负性又排除了符号抵消。因此闭合路径总权重的增长由最大正特征值支配。本讲义在这里只记录这个事实的用法，完整证明属于非负矩阵理论。 $\square$

这一节的目的不是重新发展全部谱理论，而是说明谱在对应语言中的位置：闭合路径是基本对象，矩阵特征值是闭合路径生成函数在坐标化之后的解析表达。

7.7 对应秩：通过隐藏对象空间分解

矩阵秩的常见定义是列空间维数或行空间维数。但从分解观点看，秩有一个更适合本讲义的解释：矩阵 $A : X \rightarrow Y$ 的秩不超过 r ，当且仅当它可以分解为

$$\mathbb{K}^X \longrightarrow \mathbb{K}^H \longrightarrow \mathbb{K}^Y, \quad |H| = r.$$

换成矩阵就是

$$A = BC,$$

其中 C 有 r 行， B 有 r 列。也就是说，秩衡量一个变换能否通过一个小的隐藏坐标集合 H 中转。

对应语言保留这个思想，但不把 H 看作抽象坐标维数，而看作真实的隐藏对象空间。

定义 7.9: 强对应秩

设 $\mathcal{A} : X \rightsquigarrow Y$ 是加权对应。定义它的强对应秩 $\text{crank}_s(\mathcal{A})$ 为所有有限集合 H 的最小基数 $|H|$ ，使得存在对应

$$C : X \rightsquigarrow H, \quad \mathcal{B} : H \rightsquigarrow Y,$$

并且有对应同构

$$\mathcal{A} \cong \mathcal{B} \circ C.$$

若不存在这样的有限 H ，则令 $\text{crank}_s(\mathcal{A}) = \infty$ 。

强对应秩要求的不只是矩阵影子相等，而是事件空间本身可由二步路径空间给出。因此它对事件结构非常敏感。

定义 7.10: 弱对应秩

设 $\mathcal{A} : X \rightsquigarrow Y$ 是加权对应。定义它的弱对应秩 $\text{crank}_M(\mathcal{A})$ 为所有有限集合 H 的最小基数 $|H|$ ，使得存在对应

$$C : X \rightsquigarrow H, \quad \mathcal{B} : H \rightsquigarrow Y,$$

满足矩阵影子等式

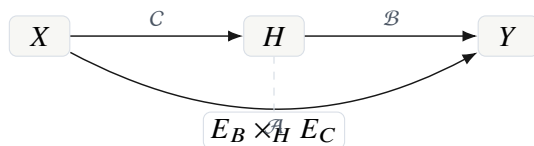
$$M(\mathcal{A}) = M(\mathcal{B} \circ C).$$

等价地，由复合与矩阵影子相容，

$$M(\mathcal{A}) = M(\mathcal{B})M(C).$$

注记

强对应秩问的是：事件程序本身能否通过小的隐藏对象空间生成。弱对应秩问的是：聚合后的矩阵影子能否通过小的隐藏对象空间分解。前者属于事件层；后者属于矩阵影子层。传统低秩矩阵分解对应的是弱对应秩。



对应秩把“低秩分解”解释为通过小隐藏对象空间 H 的二步路径生成。

图 7.4 对应秩的分解观点。

命题 7.5: 弱秩不超过强秩

若 $\text{crank}_s(\mathcal{A}) < \infty$, 则

$$\text{crank}_M(\mathcal{A}) \leq \text{crank}_s(\mathcal{A}).$$

证明.

Step 1. 设 $\text{crank}_s(\mathcal{A}) = r$, 则存在 $|H| = r$ 以及对应 $C : X \rightsquigarrow H$ 、 $B : H \rightsquigarrow Y$, 使得

$$\mathcal{A} \cong B \circ C.$$

Step 2. 对应同构保持矩阵影子, 因此

$$M(\mathcal{A}) = M(B \circ C).$$

Step 3. 这正是弱对应秩定义中允许的分解。于是

$$\text{crank}_M(\mathcal{A}) \leq |H| = r.$$

□

定理 7.4: 域上弱对应秩等于普通矩阵秩

设 $\mathbb{K}$ 是域, $A \in \mathbb{K}^{Y \times X}$ 是有限矩阵, 并令 $\mathcal{A} = \iota(A)$ 。则

$$\text{crank}_M(\mathcal{A}) = \text{rank}(A).$$

证明.

Step 1. 记 $r = \text{rank}(A)$ 。先证明 $\text{crank}_M(\mathcal{A}) \leq r$ 。

Step 2. 由普通线性代数, 存在矩阵

$$C \in \mathbb{K}^{H \times X}, \quad B \in \mathbb{K}^{Y \times H}, \quad |H| = r,$$

使得

$$A = BC.$$

例如可以取 $H = \{1, \dots, r\}$, 让 C 给出 A 的 r 个独立行坐标, 让 B 给出列空间中的重构系数; 也可以用秩分解 $A = UV$ 。

Step 3. 取坐标对应 $C = \iota(C) : X \rightsquigarrow H$ 与 $\mathcal{B} = \iota(B) : H \rightsquigarrow Y$ 。由矩阵影子相容,

$$M(\mathcal{B} \circ C) = M(\mathcal{B})M(C) = BC = A = M(\mathcal{A}).$$

因此 $\text{crank}_M(\mathcal{A}) \leq r$ 。

Step 4. 反过来, 设 $\text{crank}_M(\mathcal{A}) = m$ 。则存在 $|H| = m$ 与对应 $C : X \rightsquigarrow H$ 、 $\mathcal{B} : H \rightsquigarrow Y$, 使得

$$M(\mathcal{A}) = M(\mathcal{B})M(C).$$

Step 5. 记

$$C' = M(C) \in \mathbb{K}^{H \times X}, \quad B' = M(\mathcal{B}) \in \mathbb{K}^{Y \times H}.$$

于是

$$A = M(\mathcal{A}) = B'C'.$$

Step 6. 矩阵 $B'C'$ 的秩不超过中间维数 $|H| = m$, 因此

$$\text{rank}(A) \leq m = \text{crank}_M(\mathcal{A}).$$

Step 7. 两个方向合并, 得到

$$\text{crank}_M(\iota(A)) = \text{rank}(A).$$

□

例 7.4: 秩一矩阵的对应解释

设 $A_{yx} = b_y c_x$ 。则 A 可以通过单点隐藏空间 $H = \{h\}$ 分解:

$$x \xrightarrow{c_x} h \xrightarrow{b_y} y.$$

对应复合后, 从 x 到 y 的路径权重为

$$b_y c_x = A_{yx}.$$

因此 $\text{crank}_M(\iota(A)) = 1$, 除非 $A = 0$ 时弱秩为 0。这正是普通秩一矩阵的对应版本。

7.8 强秩、弱秩与硬件含义

弱秩与普通矩阵秩完全相容, 因此它是矩阵语言在对应框架中的继承物。强秩则更接近硬件程序: 它要求事件空间本身由隐藏对象生成, 而不是只要求最终数值相等。

为了看出差别, 考虑一个从 X 到 Y 的对应。若它通过隐藏对象 $h \in H$ 分解, 则每条复合事件来自一对事件

$$x \xrightarrow{c} h, \quad h \xrightarrow{b} y.$$

因此同一个隐藏对象会把进入它的源事件与离开它的目标事件做笛卡尔式配对。强分解是否成立, 取决于原事件空间是否真的具有这种“中间对象纤维积”结构。

例 7.5: 匹配型对应需要多个隐藏对象

设

$$X = \{x_1, x_2\}, \quad Y = \{y_1, y_2\},$$

并考虑只有两条事件的无权对应:

$$x_1 \rightarrow y_1, \quad x_2 \rightarrow y_2.$$

若试图通过单个隐藏对象 h 强分解, 并且 x_1, x_2 都能进入 h , h 又能到达 y_1, y_2 , 则复合会额外产生路径

$$x_1 \rightarrow h \rightarrow y_2, \quad x_2 \rightarrow h \rightarrow y_1.$$

这些事件在原对应中不存在。因此在不允许零权事件伪装的支撑事件模型下, 强分解需要至少两个隐藏对象, 分别服务两条匹配边。

边界

强对应秩对“是否保留零权事件”很敏感。如果允许任意零权事件出现在事件空间中, 那么一些本应不存在的路径可以用零权重掩盖, 但事件结构上仍然存在。这会破坏强秩作为事件结构复杂度的意义。因此讨论强秩时, 通常应使用支撑对应: 只存储非零或有效事件。

硬件解释

弱秩对应数值压缩: 只要最终矩阵影子能低秩分解, 硬件可以少存权重。强秩对应程序压缩: 事件图本身能通过小隐藏对象空间生成, 硬件可以少存事件、少做路由、少做匹配。对 CoPU 来说, 强秩通常比弱秩更直接关联事件存储与 fiber join 成本。

7.9 本章小结

本章把矩阵理论中四个核心概念提升到了对应语言中。

首先, 迹被解释为闭合事件权重的聚合。矩阵对角线只是闭合事件在坐标表中的影子。其次, 对应幂的事件空间是长度为 k 的路径空间, 迹幂矩就是闭合 k 路径的权重聚合。再次, 路径 zeta 函数用这些闭合路径迹幂矩构造, 在有限复矩阵影子下满足

$$\zeta_{\mathcal{A}}(u) = \det(I - uM(\mathcal{A}))^{-1}.$$

这说明矩阵谱理论可以从闭合路径生成函数中恢复。最后, 对应秩把矩阵低秩分解解释为通过隐藏对象空间的二步分解; 弱对应秩在域上等于普通矩阵秩, 强对应秩则保留了事件程序的结构约束。

下一章将转向表达复杂度。那里要回答的问题是: 如果矩阵只是对应的影子, 那么对应语言是否真的能更短、更结构化地表达同一个计算? 这需从卷积、局部规则、群作用、层级事件空间等例子中给出严格的表达优势。

练习 7.1: 闭合路径计数

令 G 是有向三角形

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 1.$$

把每条边看成计数半环上的权重 1 事件。计算 $\text{Tr}(\mathcal{A}_G^k)$, 并说明它在 k 被 3 整除和不被 3 整除时的区别。

练习 7.2: 二步迹循环性

设 $\mathcal{A} : X \rightsquigarrow Y$, $\mathcal{B} : Y \rightsquigarrow X$ 。直接写出 $\mathcal{B} \circ \mathcal{A}$ 和 $\mathcal{A} \circ \mathcal{B}$ 的闭合路径集合, 并构造二者之间的保权双射。

练习 7.3: 弱秩与矩阵秩

设

$$A = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}.$$

写出一个单点隐藏空间 $H = \{h\}$ 上的分解 $A = BC$, 并把它翻译成对应分解。

表达复杂度：为什么不展开矩阵

前几章建立了对应语言的基本代数结构。本章开始讨论一个直接面向计算的问题：既然每个有限加权对应都有矩阵影子，为什么不始终把它写成矩阵？

这个问题不能只用“矩阵太大”回答。许多矩阵本来就是稀疏的，稀疏矩阵格式已经避免了部分零条目的存储；许多矩阵乘法也可以通过分块、低秩、快速算法和硬件阵列得到很好的实现。因此，对应语言若要真正有意义，就必须指出一种更细的差别：

矩阵语言记录端点对， 对应语言记录生成端点对的事件与规则。

二者在没有结构的情况下相差不大；在结构丰富的情况下，差别可能非常大。本章的目标，是把这种差别写成可以检查的数学陈述。

说明

本章不把“表达复杂度”理解为一个完全形式化的复杂性理论，而把它作为一种数学建模尺度：同一个算子，如果用矩阵条目、事件空间、生成规则、商空间来表示，所需信息量和实际计算代价分别是什么？这种尺度足以解释为什么对应处理器不应以矩阵展开作为内部语言。

8.1 从条目到事件再到规则

先考虑最基本的情形。设 X, Y 是有限集合，一个从 X 到 Y 的矩阵影子是函数

$$A : Y \times X \longrightarrow \mathbb{K}, \quad (y, x) \longmapsto A_{yx}.$$

如果逐项存储这个函数，就要给每一对 (y, x) 留一个位置。即使多数位置为零，矩阵表示仍然先把全部坐标格子 $Y \times X$ 放在背景中。

对应表示则不同。一个加权对应是

$$\mathcal{A} = (X \xleftarrow{s} E \xrightarrow{t} Y, w),$$

其中 E 是事件空间。事件 $e \in E$ 不是坐标格子里的一个位置，而是一条带来源的关系：

$$s(e) = x, \quad t(e) = y, \quad w(e) = a.$$

同一端点对 (x, y) 可以有多个事件；不同事件也可以有不同类型、时间、生成方式或硬件位置。矩阵影子只看它们的聚合：

$$M(\mathcal{A})_{yx} = \bigoplus_{e \in s^{-1}(x) \cap t^{-1}(y)} w(e).$$

定义 8.1: 三种基本表示复杂度

设 $\mathcal{A}: X \rightsquigarrow Y$ 是有限加权对应。我们区分以下三种基本表示复杂度。

1. 矩阵展开复杂度: 显式存储全部端点对 $(y, x) \in Y \times X$ 的矩阵条目所需的位置数, 记为

$$S_{\text{mat}}(X, Y) = |Y| |X|.$$

2. 显式事件复杂度: 显式存储事件集合 E 及其端点和权重所需的事件数, 记为

$$S_{\text{ev}}(\mathcal{A}) = |E|.$$

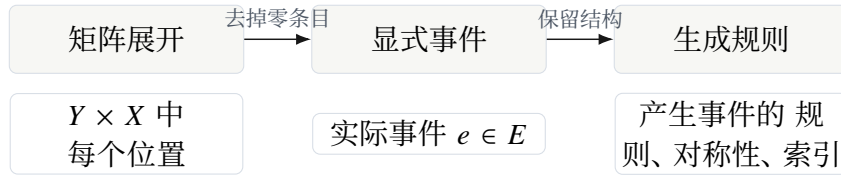
若考虑端点编码和权重编码, 则还要乘上每个事件的字长; 本章先只记录事件数量。

3. 规则复杂度: 若 E, s, t, w 由某个有限规则或算法生成, 则记录该生成规则所需的信息量, 记为

$$S_{\text{rule}}(\mathcal{A}).$$

本讲义中它只作为粗粒度尺度使用, 例如 $O(r + \log n)$ 、 $O(\log n)$ 或 $O(|\Theta|)$ 。

这三个尺度对应三种不同层级的表示。矩阵展开把所有端点对物化。显式事件表示只物化真正存在的事件。规则表示甚至不必事先物化事件, 而是在需要时生成事件。

**注记**

稀疏矩阵格式主要利用 S_{ev} , 因为它把非零条目作为事件存储。对应语言还要进一步利用 S_{rule} : 某些非零事件本身也不应全部存储, 而应由平移、局部邻域、树结构、群作用或候选生成器在线产生。

8.2 物化成本与表示成本

表示复杂度和计算复杂度之间有一层常被忽略的中介: 物化成本。所谓物化, 是指把一个本来可由规则生成的结构显式写成数组、矩阵或事件表。

定义 8.2: 物化

设一个对应 $\mathcal{A}$ 由规则 R 生成。若计算系统先生成并存储完整的矩阵影子 $M(\mathcal{A})$, 称为矩阵物化; 若先生成并存储完整事件表 E , 称为事件物化; 若只在计算过程中按需产生端点纤维或事件, 称为按需生成。

物化本身不是错误的。对于小矩阵, 矩阵物化最简单; 对于反复使用的固定稀疏结构, 事件物化也很合理。但在许多结构化算子中, 矩阵物化会制造出大量没有信息含量的中间对象。

例 8.1: 循环移位

令 $X = Y = \mathbb{Z}/n\mathbb{Z}$ 。循环右移算子 T 定义为

$$(Tf)(i) = f(i - 1).$$

矩阵语言把它写成 $n \times n$ 的置换矩阵：

$$T_{ij} = 1 \iff j = i - 1 \pmod{n}.$$

显式事件表示有 n 个事件

$$e_i : i - 1 \longrightarrow i.$$

规则表示只需要记录“模 n 加一”这条规则，因此大小约为 $O(\log n)$ ，加上一个单位权重。

这个例子很小，但它揭示了本章的基本问题。矩阵展开复杂度是 n^2 ；显式事件复杂度是 n ；规则复杂度近似为 $\log n$ 。同一个线性算子，在三种语言中的表达长度完全不同。

命题 8.1: 循环移位的三层复杂度

循环移位算子 T 的矩阵展开复杂度、显式事件复杂度和规则复杂度分别满足

$$S_{\text{mat}}(T) = n^2, \quad S_{\text{ev}}(T) = n, \quad S_{\text{rule}}(T) = O(\log n).$$

证明.

Step 1. 作为从 n 个输入坐标到 n 个输出坐标的矩阵， T 的矩阵坐标格子有 n^2 个位置。若采用完整矩阵展开，需要 n^2 个条目位置。

Step 2. 每个输出 i 只从唯一输入 $i - 1$ 接收值，因此事件空间可以取

$$E = \{e_i : i \in \mathbb{Z}/n\mathbb{Z}\},$$

故 $|E| = n$ 。

Step 3. 生成所有事件只需知道集合大小 n 和规则

$$s(e_i) = i - 1, \quad t(e_i) = i, \quad w(e_i) = 1.$$

记录 n 需要 $O(\log n)$ 位；规则本身长度不随 n 增长。因此规则复杂度为 $O(\log n)$ 。

□

硬件解释

若芯片内部执行循环移位，最差的做法是读取一个 $n \times n$ 置换矩阵；较好的做法是存储 n 条连接；更好的做法是用地址发生器计算 $i \mapsto i + 1$ 。对应语言把这三种实现区分为矩阵物化、事件物化和规则生成。

8.3 卷积：从 Toeplitz 矩阵回到局部规则

卷积是最典型的例子。在线性代数里，卷积可以写成 Toeplitz 矩阵或循环矩阵；在信号处理中，卷积本来就是局部平移不变规则。矩阵写法正确，但常常不是最自然的表示。

设 $X = Y = \mathbb{Z}/n\mathbb{Z}$ ，核半径为 r ，偏移集合为

$$R = \{-r, -r + 1, \dots, r\}.$$

给定核权重 $k_a \in \mathbb{K}$ ，循环卷积定义为

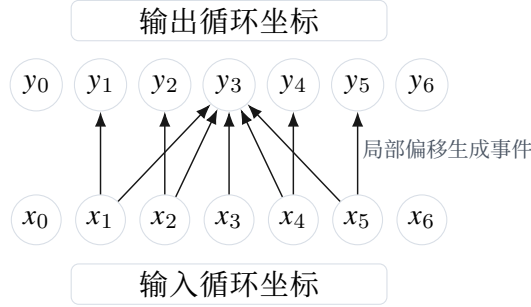
$$(Kf)(i) = \bigoplus_{a \in R} k_a \otimes f(i - a).$$

在对应语言中，事件空间取

$$E = Y \times R.$$

对事件 (i, a) ，定义

$$s(i, a) = i - a, \quad t(i, a) = i, \quad w(i, a) = k_a.$$



命题 8.2: 循环卷积的表达复杂度

长度为 n 、核半径为 r 的循环一维卷积满足

$$S_{\text{mat}} = n^2, \quad S_{\text{ev}} = n(2r + 1), \quad S_{\text{rule}} = O(r + \log n).$$

证明.

Step 1. 矩阵展开需要给每个端点对 $(i, j) \in (\mathbb{Z}/n\mathbb{Z})^2$ 一个条目位置，因此有 n^2 个位置。

Step 2. 对应事件由输出位置 i 和偏移 $a \in R$ 决定。每个输出位置对应 $2r + 1$ 个偏移，因此事件数为

$$|E| = |Y| |R| = n(2r + 1).$$

Step 3. 规则表示只需记录集合大小 n 、偏移集合 R 、端点规则

$$(i, a) \mapsto i - a, \quad (i, a) \mapsto i,$$

以及 $2r + 1$ 个核权重 k_a 。因此除固定规则外，只需 $O(\log n)$ 的大小参数和 $O(r)$ 个权重。

Step 4. 所以规则复杂度为 $O(r + \log n)$ 。

□

上面的命题说明，卷积矩阵的 n^2 个位置大多不是独立信息。它们由局部偏移规则和少量核权重共同生成。对应语言直接保存这个事实。

推论 8.1: 固定半径卷积的严格分离

若核半径 r 固定而 $n \rightarrow \infty$ ，则循环卷积的三种复杂度满足

$$S_{\text{mat}} = \Theta(n^2), \quad S_{\text{ev}} = \Theta(n), \quad S_{\text{rule}} = O(\log n).$$

因此矩阵展开、显式事件和规则生成之间存在二次、线性与对数级别的分离。

证明. 把 r 视为常数，命题 8.2 中的 $n(2r + 1)$ 等于 $\Theta(n)$ ，而 $O(r + \log n) = O(\log n)$ 。

□

8.4 稀疏性不等于结构

上一节的卷积不仅稀疏，而且有强规则结构。必须区分这两件事。

定义 8.3: 稀疏表示与结构表示

设 $\mathcal{A} : X \rightsquigarrow Y$ 。

1. 若 $|E| \ll |X||Y|$ ，称 $\mathcal{A}$ 具有事件稀疏性。
2. 若 E, s, t, w 可以由远小于 $|E|$ 的规则生成，称 $\mathcal{A}$ 具有规则结构。

稀疏性回答“有多少事件”；结构回答“这些事件从哪里来”。一个随机稀疏图可能事件数很少，但没有短规则生成；一个卷积图不仅事件数少，而且由偏移规则生成。

例 8.2: 随机稀疏图没有明显规则

令 G 是 n 个点、 dn 条边的随机有向图。邻接矩阵有 n^2 个位置，边事件只有 dn 个，因此它是稀疏的。但若这些边没有额外规律，记录全部边仍然需要 $\Theta(dn \log n)$ 位左右的信息。对应语言能避免矩阵展开，却不能凭空把随机边压缩成 $O(\log n)$ 规则。

边界

对应语言不是万能压缩算法。它只把已经存在的结构保留下来。若一个算子本身没有低复杂度生成结构，则对应语言至多避免矩阵零条目的浪费，不能把一般稠密矩阵变成短公式。

8.5 一般矩阵不能免费压缩

为了避免把对应语言误解成“所有矩阵都能压缩”，我们给出一个简单的计数论证。这个论证不依赖深的复杂性理论，只说明大多数矩阵没有短描述。

定理 8.1: 一般矩阵的计数下界

设 $\mathbb{K}$ 是含 $q \geq 2$ 个元素的有限权重集合，考虑所有 $n \times n$ 的 $\mathbb{K}$ -矩阵。若一种编码长度不超过 L 比特，则它最多能表示 2^L 个矩阵。因此，当

$$L < n^2 \log_2 q$$

时，不可能用长度不超过 L 的编码表示所有 $n \times n$ 的 $\mathbb{K}$ -矩阵。特别地，至少存在某些矩阵需要 $\Omega(n^2)$ 比特描述。

证明.

Step 1. 一个 $n \times n$ 的 $\mathbb{K}$ -矩阵有 n^2 个条目，每个条目有 q 种取值。因此矩阵总数为

$$q^{n^2}.$$

Step 2. 长度不超过 L 比特的二进制字符串最多有

$$1 + 2 + \cdots + 2^L < 2^{L+1}$$

个。若只考虑长度正好为 L 的编码，则最多为 2^L 个；常数差别不影响结论。

Step 3. 若所有矩阵都能由长度不超过 L 的编码表示，则编码数必须不少于矩阵数，即

$$2^L \geq q^{n^2}.$$

取二进制对数，得到

$$L \geq n^2 \log_2 q.$$

Step 4. 因此当 $L < n^2 \log_2 q$ 时，必有矩阵不能被这种长度的编码表示。

□

注记

这个定理的意义不是讨论具体压缩算法，而是划定边界：对应语言的优势不能来自对任意矩阵的神秘压缩，只能来自算子本身具有规则、稀疏、对称、局部或层级结构。

8.6 图消息传递：边空间优先于邻接矩阵

图神经网络常被写成

$$H' = \sigma(AHW),$$

其中 A 是邻接矩阵。这种写法方便使用线性代数工具，但图的原始数据不是邻接矩阵，而是边空间。

定义 8.4: 图对应

给定有向图 $G = (V, E)$ 。其图对应为

$$\mathcal{G} = (V \xleftarrow{s} E \xrightarrow{t} V, w),$$

其中 $s(e)$ 与 $t(e)$ 分别为边的源点与目标点。若图无权，则可取 $w(e) = 1$ 。

图消息传递可以写成

$$h'_v = \bigoplus_{e \in t^{-1}(v)} w(e) \otimes h_{s(e)}.$$

这正是“拉回-加权-推前”的三步形式：先沿 s 取源点特征，再乘以边权，最后沿 t 的目标纤维聚合。

由于本讲义没有为章节加全局标签，上面一句只应理解为对第四章结构的回顾。为了避免引用歧义，我们把公式重新写清楚。给定 $h: V \rightarrow \mathbb{K}^d$ ，对应作用为

$$h \xrightarrow{s^*} h \circ s \xrightarrow{\text{乘以 } w} (e \mapsto w(e) \otimes h(s(e))) \xrightarrow{t_!} h'.$$

其中

$$h'(v) = \bigoplus_{e \in t^{-1}(v)} w(e) \otimes h(s(e)).$$

命题 8.3: 有界度图的事件复杂度

若有向图 $G = (V, E)$ 有 $|V| = n$ ，且每个顶点入度至多 d ，则图消息传递的矩阵展开复杂度为 n^2 ，显式事件复杂度至多为 dn 。

证明.

Step 1. 邻接矩阵是 $n \times n$ 矩阵，因此完整展开有 n^2 个端点位置。

Step 2. 每个顶点入度至多 d ，所以以该顶点为目标边至多 d 条。对所有目标顶点求和，得到

$$|E| = \sum_{v \in V} |t^{-1}(v)| \leq dn.$$

Step 3. 显式事件表示只需要存储边事件，因此事件复杂度至多为 dn 。

□



8.7 复合成本由中间纤维控制

对应复合的关键成本不是矩阵维数的三重乘积，而是中间对象上纤维负载的乘积求和。这个公式在第二章已经作为纤维积基数公式出现；这里把它解释为计算成本。

定理 8.2: 对应复合成本公式

设

$$\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, w_A), \quad \mathcal{B} = (Y \xleftarrow{s_B} E_B \xrightarrow{t_B} Z, w_B).$$

若显式生成所有二步路径，则路径数为

$$|E_B \times_Y E_A| = \sum_{y \in Y} |t_A^{-1}(y)| |s_B^{-1}(y)|.$$

证明.

Step 1. 纤维积定义为

$$E_B \times_Y E_A = \{(e_B, e_A) : s_B(e_B) = t_A(e_A)\}.$$

Step 2. 按中间点 $y \in Y$ 分桶。经过 y 的二步路径正好是

$$s_B^{-1}(y) \times t_A^{-1}(y).$$

Step 3. 该桶内路径数为

$$|s_B^{-1}(y)| |t_A^{-1}(y)|.$$

Step 4. 不同 y 的桶互不相交，并且它们的并就是整个纤维积。因此总路径数为

$$\sum_{y \in Y} |t_A^{-1}(y)| |s_B^{-1}(y)|.$$

□

推论 8.2: 有界中间度的复合成本

若对所有 $y \in Y$ 都有

$$|t_A^{-1}(y)| \leq d_A, \quad |s_B^{-1}(y)| \leq d_B,$$

则

$$|E_B \times_Y E_A| \leq |Y|d_Ad_B.$$

证明. 由定理 8.2,

$$|E_B \times_Y E_A| = \sum_{y \in Y} |t_A^{-1}(y)| |s_B^{-1}(y)| \leq \sum_{y \in Y} d_Ad_B = |Y|d_Ad_B.$$

□

推论 8.3: 矩阵乘法是满纤维特例

若 $\mathcal{A}$ 与 $\mathcal{B}$ 是完整矩阵坐标对应, 且

$$|X| = n, \quad |Y| = m, \quad |Z| = p,$$

则二步路径数为

$$pmn.$$

证明.

Step 1. 完整矩阵坐标对应 $\mathcal{A} : X \rightarrow Y$ 对每个 (y, x) 有一个事件。因此对固定 y , 目标为 y 的事件有 n 个:

$$|t_A^{-1}(y)| = n.$$

Step 2. 完整矩阵坐标对应 $\mathcal{B} : Y \rightarrow Z$ 对每个 (z, y) 有一个事件。因此对固定 y , 源为 y 的事件有 p 个:

$$|s_B^{-1}(y)| = p.$$

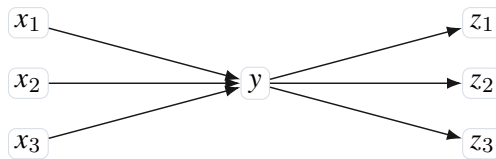
Step 3. 代入复合成本公式, 得到

$$|E_B \times_Y E_A| = \sum_{y \in Y} np = mnp.$$

这正是普通矩阵乘法中的三重循环规模。

□

这个推论说明, 矩阵乘法不是被对应语言排斥, 而是被识别为“中间纤维完全饱和”的特例。对应复合把三重循环中的实际负载暴露出来。



经过同一中间纤维的路径数为 $3 \cdot 3$

8.8 商压缩：把重复事件放回对称性

有些结构不仅稀疏，而且具有对称性。卷积的平移不变性就是最简单的对称性：同一个局部模式在每个位置重复出现。矩阵语言会把这些重复模式展开；对应语言可以把重复模式写成群作用下的轨道。

定义 8.5: 带群作用的对应

设群 G 作用在 X, Y, E 上。若

$$s(ge) = gs(e), \quad t(ge) = gt(e), \quad w(ge) = w(e)$$

对所有 $g \in G, e \in E$ 成立，则称 $\mathcal{A} = (X \xleftarrow{s} E \xrightarrow{t} Y, w)$ 为一个 G -等变对应。

在 G -等变对应中，许多事件由同一个原型通过群作用生成。若只记录原型和群作用规则，就可以避免重复存储全部事件。

命题 8.4: 自由作用下的事件轨道分解

若有限群 G 自由作用在有限事件空间 E 上，则

$$|E| = |G| |E/G|,$$

其中 E/G 是事件轨道集合。

证明.

Step 1. 自由作用意味着若 $ge = e$ ，则 g 必为单位元。

Step 2. 因此每个事件 e 的轨道

$$Ge = \{ge : g \in G\}$$

含有 $|G|$ 个不同元素。

Step 3. 不同轨道互不相交，并且所有轨道的并为 E 。

Step 4. 所以事件空间由 $|E/G|$ 个轨道组成，每个轨道有 $|G|$ 个事件，故

$$|E| = |G| |E/G|.$$

□

例 8.3: 卷积的轨道解释

循环卷积中，群 $G = \mathbb{Z}/n\mathbb{Z}$ 作用在位置上。固定一个偏移 a ，所有事件

$$i - a \longrightarrow i$$

构成一个平移轨道。因此半径为 r 的卷积有 $2r + 1$ 个事件原型，每个原型在 n 个位置上重复。

硬件解释

商压缩在硬件里对应“原型存储 + 地址生成”。芯片不需要存储每个位置上的重复边，只需存储局部模板和群作用规则，然后由地址发生器生成具体端点。

8.9 张量积结构与乘积对应

矩阵语言中，高维结构常被压平成一个长向量，再写成大矩阵。对应语言可以把乘积结构保留在对象和事件空间中。

设

$$\mathcal{A} : X \rightsquigarrow Y, \quad \mathcal{B} : X' \rightsquigarrow Y'.$$

它们的乘积对应是

$$\mathcal{A} \boxtimes \mathcal{B} : X \times X' \rightsquigarrow Y \times Y',$$

事件空间为 $E_A \times E_B$ ，端点映射为

$$s(e_A, e_B) = (s_A(e_A), s_B(e_B)), \quad t(e_A, e_B) = (t_A(e_A), t_B(e_B)),$$

权重为

$$w(e_A, e_B) = w_A(e_A) \otimes w_B(e_B).$$

命题 8.5: 乘积对应的矩阵影子

若 $\mathbb{K}$ 是交换半环，则

$$M(\mathcal{A} \boxtimes \mathcal{B}) = M(\mathcal{A}) \otimes_{\text{Kr}} M(\mathcal{B}),$$

其中右边是矩阵的 Kronecker 积，按照对象乘积的字典序排列坐标。

证明.

Step 1. 固定输入端点 (x, x') 和输出端点 (y, y') 。

Step 2. 乘积对应中从 (x, x') 到 (y, y') 的事件是所有满足

$$s_A(e_A) = x, \quad t_A(e_A) = y, \quad s_B(e_B) = x', \quad t_B(e_B) = y'$$

的对 (e_A, e_B) 。

Step 3. 因此影子条目为

$$\begin{aligned} M(\mathcal{A} \boxtimes \mathcal{B})_{(y, y'), (x, x')} &= \bigoplus_{e_A, e_B} w_A(e_A) \otimes w_B(e_B) \\ &= \left(\bigoplus_{e_A : s_A(e_A) = x, t_A(e_A) = y} w_A(e_A) \right) \otimes \left(\bigoplus_{e_B : s_B(e_B) = x', t_B(e_B) = y'} w_B(e_B) \right) \\ &= M(\mathcal{A})_{yx} \otimes M(\mathcal{B})_{y'x'}. \end{aligned}$$

第二个等号使用半环乘法对加法的分配律和有限求和换序。

Step 4. 这正是 Kronecker 积的条目公式。

□

注记

Kronecker 积矩阵的维度会乘起来，但乘积对应可以保留因子结构。若两个因子都有短规则，则乘积结构常常也有短规则。这里的优势不是改变数学算子，而是避免过早把乘积结构压平成单一矩阵。

8.10 Attention 的动态对应解释

标准 attention 常写成

$$\text{Attn}(Q, K, V) = \text{softmax}(QK^\top) V.$$

矩阵语言首先形成 $n \times n$ 的分数矩阵。对应语言则可以把 attention 看成由输入动态生成的对应。

设 token 集合为 $I = \{1, \dots, n\}$ 。对每个输入样本，query 和 key 给出分数函数

$$c_x(i, j) = \langle q_i, k_j \rangle.$$

门控规则 Γ 从每个 i 选择一批候选 j ，例如局部窗口、近邻搜索、阈值规则或哈希桶。于是动态事件空间为

$$E_x = \{(i, j) : j \in \Gamma_x(i)\}.$$

权重可取为局部归一化后的

$$w_x(i, j) = \frac{\exp(c_x(i, j))}{\sum_{j' \in \Gamma_x(i)} \exp(c_x(i, j'))}.$$

定义 8.6: 动态对应

一个动态对应不是固定的 $X \leftarrow E \rightarrow Y$ ，而是对每个输入状态 x 给出一个对应

$$\mathcal{A}(x) = (X \xleftarrow{s_x} E_x \xrightarrow{t_x} Y, w_x).$$

若 E_x 由候选生成规则产生，则称它为规则生成的动态对应。

命题 8.6: 有界候选 attention 的事件复杂度

若每个 query 位置 i 至多选择 k 个 key 位置，即

$$|\Gamma_x(i)| \leq k$$

对所有 i 成立，则动态 attention 对应的事件数满足

$$|E_x| \leq nk.$$

而完整 attention 分数矩阵有 n^2 个位置。

证明. 由定义，事件空间为

$$E_x = \{(i, j) : j \in \Gamma_x(i)\}.$$

对每个 i ，可选的 j 至多 k 个，因此

$$|E_x| = \sum_{i=1}^n |\Gamma_x(i)| \leq \sum_{i=1}^n k = nk.$$

完整分数矩阵则包含所有 (i, j) ，共有 n^2 个位置。 □

边界

这个命题只说明事件数下降，不说明近似误差一定小。要证明稀疏 attention 保持精度，还需要关于 score 函数、数据几何、核衰减或任务损失的额外假设。对应语言先提供正确的结构化表达，误差理论需要在具体模型中另行建立。

8.11 表达复杂度与硬件代价

现在把前面的数学尺度翻译成硬件尺度。矩阵展开对应大块连续存储和规则的乘加阵列；事件表示对应边表、纤维索引和局部聚合；规则表示对应地址发生器、模式发生器和按需计算。

定义 8.7: 对应实现的三类代价

对于一个对应程序，我们区分三类硬件代价：

1. 存储代价：需要存储多少矩阵条目、事件或规则参数；
2. 访存代价：计算时需要从存储层级搬运多少条目或事件；
3. 匹配聚合代价：为了复合对应，需要完成多少纤维匹配、权重合成和端点聚合。

对 CoPU 来说，核心问题不是如何把 GEMM 做得更快，而是如何避免把本来可以由纤维生成的结构展开成 GEMM。典型流水线为

规则生成 → 纤维索引 → 事件匹配 → 权重合成 → 推前聚合。

**命题 8.7: 不物化矩阵的访存节省**

若一个算子的矩阵展开复杂度为 S_{mat} ，显式事件复杂度为 S_{ev} ，并且一次应用该算子只需读取每个事件常数次，则事件实现相对矩阵展开的访存条目数比例至多为

$$\frac{S_{\text{ev}}}{S_{\text{mat}}} \sim \frac{|E|}{|X||Y|}.$$

特别地，当 $|E| \ll |X||Y|$ 时，事件表示在访存规模上有直接优势。

证明. 矩阵展开实现至少需要访问与非零判定或条目读取相关的矩阵位置；若按完整矩阵扫描，则规模为 $|X||Y|$ 。事件实现只扫描事件表 E ，规模为 $|E|$ 。在每个事件只被常数次读取的假设下，访存规模比为

$$O(|E|)/O(|X||Y|),$$

即命题中的比例。这里省略常数，因为不同硬件的数据字长、缓存策略和流水线会改变常数。□

公式中的符号“ $\sim$ ”不是标准数学符号，容易误导。严格写法应为

$$\text{访存比例的主导项为 } \frac{|E|}{|X||Y|}.$$

因此上面的命题只是一条粗粒度硬件估算，不是精确能耗定理。

8.12 本章小结

本章建立了对应语言的第一个计算优势框架。矩阵展开、显式事件和规则生成是三个不同层级：

$$Y \times X \longrightarrow E \longrightarrow \text{生成} E \text{ 的规则.}$$

矩阵语言默认在第一个层级工作；稀疏矩阵主要利用第二个层级；对应语言试图系统利用第二、第三个层级以及商结构。

这一章最重要的结论可以概括为三点。第一，对应语言不会压缩一般无结构矩阵，计数下界已经说明这一点。第二，当算子具有局部、稀疏、群对称、乘积或动态候选结构时，对应表示可能显著短于矩阵展开。第三，复合成本由中间纤维负载控制，而不是只由外部矩阵尺寸控制。

下一章将转向硬件结构。到目前为止，我们已经有了对象、事件、权重、复合、正规化、迹谱秩和表达复杂度。接下来要问：什么样的处理器可以把这些数学操作作为原生指令执行？这正是 CoPU 架构的出发点。

对应处理器 CoPU：从代数原语到硬件架构

前面几章已经给出对应计算的数学语言：对象空间、事件空间、权重半环、纤维复合、矩阵影子、正规化、迹与表达复杂度。本章开始把这些概念翻译成处理器结构。这里的目标不是先追求某个工艺节点上的性能数字，而是回答一个更基本的问题：如果一种机器把“对应复合”而不是“矩阵乘法”作为原生操作，那么它必须保存什么状态，必须提供哪些指令，必须怎样组织存储、匹配、合成与聚合。

本章的写法仍然从最小情形开始。我们先说明为什么 CoPU 不是稀疏矩阵加速器的简单改名；然后定义对应程序的机器表示；接着构造纤维索引、纤维匹配、权重合成和推前聚合这四个硬件原语；最后给出一个可综合原型的模块划分、代价模型和验证条件。

9.1 设计出发点：不要把结构先展开

矩阵加速器的典型路线是

模型 $\rightarrow$ 张量程序 $\rightarrow$ 矩阵或块矩阵核 $\rightarrow$ MAC 阵列.

这条路线非常成功，因为大量数值计算确实可以归约为高吞吐的乘加操作。但是从第八章的表达复杂度分析可见，许多结构化算子在矩阵语言中会先被展开为一个大表，再由硬件去计算这个大表的作用。展开以后，卷积的平移结构、图的边空间结构、局部 attention 的候选生成结构、群对称产生的等价事件结构都变得不显式。

CoPU 的路线不同：

模型 $\rightarrow$ 对应程序 $\rightarrow$ 对象表、事件空间与纤维索引 $\rightarrow$ join-compose-reduce 流水线.

也就是说，机器直接面对的是

$$X \xleftarrow{s} E \xrightarrow{t} Y, \quad w : E \rightarrow \mathbb{K},$$

而不是首先面对一个已经展开的矩阵

$$A \in \mathbb{K}^{Y \times X}.$$

这种区别不是术语上的，而是机器状态上的：矩阵处理器主要保存二维坐标条目，CoPU 必须保存事件本身以及事件在源端点、目标端点上的纤维结构。

说明

本章反复使用一个原则：矩阵乘法是 *join-compose-reduce* 的正规化影子。其中 *join* 是在中间对象上做纤维匹配，*compose* 是沿路径合成权重，*reduce* 是把同一端点对上的路径权重聚合。CoPU 要做的事情，就是把这个过程作为原生流水线实现。

定义 9.1: CoPU 的设计不变量

一个对应处理器的最小设计应满足以下不变量。

1. 对象不变量：机器状态中保留对象空间的元素标识，而不只保留线性地址。
2. 事件不变量：机器状态中保留事件记录 e ，包括源端点、目标端点、权重和附加标签。
3. 纤维不变量：机器能按 $s(e)$ 或 $t(e)$ 访问事件纤维。
4. 路径不变量：两个对应复合时，机器枚举的是纤维积

$$E_B \times_Y E_A = \{(e_B, e_A) : s_B(e_B) = t_A(e_A)\}.$$

5. 正规化不变量：若需要输出矩阵影子，则机器必须把同一端点对 (x, z) 上的路径贡献按 $\oplus$ 聚合。

这些不变量决定了 CoPU 不是普通稀疏矩阵处理器。稀疏矩阵格式通常保存的是非零条目 (i, j, A_{ij}) ，其目标是跳过零。CoPU 保存的是事件空间 E ，其目标是保留生成结构、端点纤维、事件 multiplicity、事件类型以及商压缩关系。即使两个不同事件在矩阵影子中落到同一个条目，它们在 CoPU 中仍然可以保持不同身份，直到某条指令显式要求正规化。

9.2 对应程序的机器表示

要把对应写进硬件，首先需要确定机器看到的最小数据结构。设 X, Y 为有限对象空间。一个加权对应

$$\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, w_A)$$

在机器中分为三类表：对象表、事件表和索引表。

定义 9.2: 对象记录

对象表中的一条对象记录记为

$$o = (\text{id}, \text{type}, \text{coord}, \text{tag}).$$

其中 id 是对象标识， type 说明对象属于哪一个空间， coord 是可选坐标或局部编号， tag 是类型、层级、批次或商类信息。对象记录的数学作用是把有限集合的元素变成可寻址的机器元素。

定义 9.3: 事件记录

事件表中的一条事件记录记为

$$e = (\text{src}, \text{dst}, \text{weight}, \text{etype}, \text{flag}).$$

其中 $\text{src} = s(e)$ ， $\text{dst} = t(e)$ ， $\text{weight} = w(e)$ ， etype 记录事件类型， flag 记录是否有效、是否已被商合并、是否为边界事件等信息。

硬件解释

在第一版 FPGA 原型中，可以把事件记录简化为固定宽度字。例如

[valid : 1] [type : 3] [src : 14] [dst : 14] [weight : 16].

这只是一个工程选择，不影响数学定义。若对象空间很小，源端点和目标端点位宽可以降低；若权重半环是布尔半环，则 weight 字段甚至可以退化为一位。

为了执行纤维复合，仅有事件表还不够。机器还必须能快速访问所有满足 $t_A(e_A) = y$ 或 $s_B(e_B) = y$ 的事件。这就是纤维索引。

定义 9.4: 源索引与目标索引

设 $\mathcal{A} : X \rightsquigarrow Y$ 的事件空间为 E_A 。源索引是按照源端点分桶的事件列表

$$\text{SrcIdx}_A(x) = s_A^{-1}(x), \quad x \in X.$$

目标索引是按照目标端点分桶的事件列表

$$\text{DstIdx}_A(y) = t_A^{-1}(y), \quad y \in Y.$$

在硬件中，这两个索引可以用 CSR/CSC 类似格式保存：一个指针数组和一个事件编号数组。

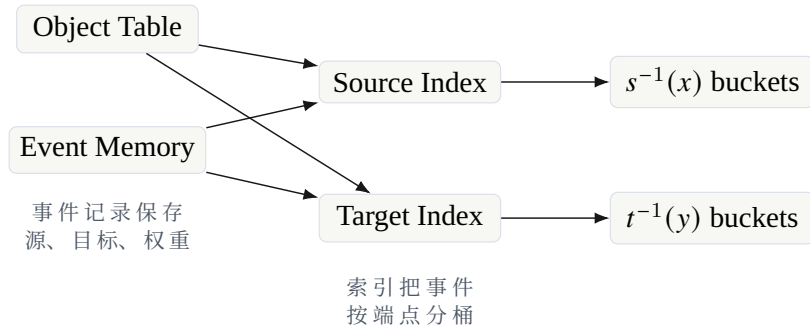


图 9.1 对象表、事件表与纤维索引。事件表保存对应本身；索引表保存沿源端点或目标端点的访问方式。

9.3 纤维索引的构造

本节说明一个基本事实：纤维索引不是额外数学结构，而是端点映射的机器化表示。对每个事件 e ，源端点 $s(e)$ 和目标端点 $t(e)$ 已经确定。建立索引就是把事件按照端点值重新排列或建立指针。

设事件编号为

$$E = \{0, 1, \dots, m-1\},$$

目标对象空间为

$$Y = \{0, 1, \dots, n_Y - 1\}.$$

目标索引可以用两个数组表示:

$$\text{ptr}_t[0 : n_Y], \quad \text{list}_t[0 : m - 1].$$

其中属于 $t^{-1}(y)$ 的事件编号被连续放在

$$\text{list}_t[\text{ptr}_t[y]], \dots, \text{list}_t[\text{ptr}_t[y + 1] - 1].$$

命题 9.1: CSR 型纤维索引的正确性

若数组 $\text{ptr}_t, \text{list}_t$ 由端点映射 $t : E \rightarrow Y$ 按端点分桶构造, 则对每个 $y \in Y$, 区间

$$[\text{ptr}_t[y], \text{ptr}_t[y + 1])$$

中列出的事件编号恰好构成目标纤维 $t^{-1}(y)$ 。

证明.

Step 1. 构造时先统计每个端点 y 的出现次数

$$c_y = |\{e \in E : t(e) = y\}|.$$

然后令

$$\text{ptr}_t[0] = 0, \quad \text{ptr}_t[y + 1] = \text{ptr}_t[y] + c_y.$$

因此第 y 个桶的长度为 c_y 。

Step 2. 构造列表时, 每个事件 e 被写入由 $t(e)$ 决定的桶。于是若 e 被写入第 y 个桶, 则必有 $t(e) = y$ 。

Step 3. 反过来, 若 $t(e) = y$, 构造规则会把 e 写入第 y 个桶。每个事件只写入一次, 因此没有遗漏。

Step 4. 前两步说明第 y 个桶中事件集合既包含于 $t^{-1}(y)$, 又包含 $t^{-1}(y)$, 故二者相等。

□

命题 9.2: 索引构造代价

若对象编号已经是整数端点编号, 则一次源索引或目标索引构造可以在

$$O(|E| + |Y|)$$

时间和

$$O(|E| + |Y|)$$

额外空间内完成。这里 Y 表示所索引的端点空间。

证明. 构造分为三步。第一步扫描事件表, 统计每个端点的事件数, 代价 $O(|E|)$ 。第二步对计数数组做前缀和, 代价 $O(|Y|)$ 。第三步再次扫描事件表, 把事件编号写入对应桶, 代价 $O(|E|)$ 。总时间为 $O(|E| + |Y|)$ 。数组长度分别为 $|Y| + 1$ 和 $|E|$, 因此空间也是 $O(|E| + |Y|)$ 。 □

注记

在数学叙述中，纤维 $t^{-1}(y)$ 是一个集合；在硬件叙述中，它是一段连续地址。这是本章最重要的翻译之一：集合论中的分解

$$E = \bigsqcup_{y \in Y} t^{-1}(y)$$

变成了存储中的分桶布局。

9.4 纤维匹配：从纤维积到 join 单元

设

$$\mathcal{A} = (X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, w_A), \quad \mathcal{B} = (Y \xleftarrow{s_B} E_B \xrightarrow{t_B} Z, w_B).$$

复合对应的事件空间是纤维积

$$E_B \times_Y E_A = \{(e_B, e_A) : s_B(e_B) = t_A(e_A)\}.$$

这句话在硬件中读作：对每个中间对象 $y \in Y$ ，把 A 中目标为 y 的事件桶与 B 中源为 y 的事件桶做笛卡尔积。

定义 9.5: 纤维 join 单元

给定 DstIdx_A 与 SrcIdx_B ，纤维 join 单元输出事件对流

$$\text{Join}(A, B) = \{(e_B, e_A) : s_B(e_B) = t_A(e_A)\}.$$

其基本循环为

```
for each  $y \in Y$  :
  for each  $e_A \in t_A^{-1}(y)$  :
    for each  $e_B \in s_B^{-1}(y)$  :
      emit  $(e_B, e_A)$ .
```

定理 9.1: fiber join 枚举纤维积

若目标索引 DstIdx_A 与源索引 SrcIdx_B 正确，则纤维 join 单元输出的事件对集合恰好等于纤维积 $E_B \times_Y E_A$ 。

证明.

Step 1. 先证输出包含于纤维积。若 join 单元输出 (e_B, e_A) ，则它们是在同一个中间桶 y 中被枚举的。因此

$$e_A \in t_A^{-1}(y), \quad e_B \in s_B^{-1}(y).$$

于是 $t_A(e_A) = y = s_B(e_B)$ ，所以 $(e_B, e_A) \in E_B \times_Y E_A$ 。

Step 2. 再证纤维积包含于输出。若 $(e_B, e_A) \in E_B \times_Y E_A$ ，令

$$y = t_A(e_A) = s_B(e_B).$$

由索引正确性， e_A 会出现在目标桶 $t_A^{-1}(y)$ 中， e_B 会出现在源桶 $s_B^{-1}(y)$ 中。join 单元遍历这两个桶的笛卡尔积，所以必输出 (e_B, e_A) 。

Step 3. 两个包含关系合起来说明输出集合与纤维积相等。

□

命题 9.3: join 数量公式

记

$$L_y = |t_A^{-1}(y)|, \quad R_y = |s_B^{-1}(y)|.$$

则 join 单元输出事件对的数量为

$$J(A, B) = \sum_{y \in Y} L_y R_y.$$

证明. 对固定的 y , join 单元枚举两个桶

$$t_A^{-1}(y), \quad s_B^{-1}(y)$$

的笛卡尔积, 输出数量为

$$|t_A^{-1}(y)| |s_B^{-1}(y)| = L_y R_y.$$

不同的 y 对应不相交的中间端点条件, 因此总输出数量为各桶输出数量之和。

□

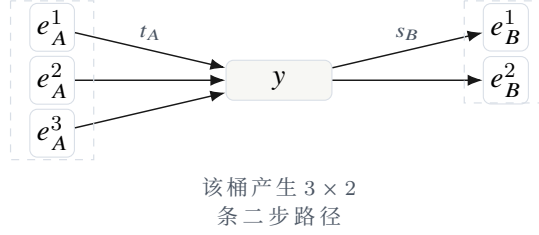


图 9.2 固定中间端点 y 时, 纤维 join 枚举左桶与右桶的笛卡尔积。

9.5 权重合成与半环数据通路

join 单元只产生二步路径的事件对。要得到复合对应, 还必须计算这条路径的权重。若

$$(e_B, e_A) \in E_B \times_Y E_A,$$

则复合事件的权重定义为

$$w_{B \circ A}(e_B, e_A) = w_B(e_B) \otimes w_A(e_A).$$

这里 $\otimes$ 来自权重半环 $\mathbb{K}$ 。

定义 9.6: 权重合成单元

权重合成单元是一个以半环乘法 $\otimes$ 为语义的组合逻辑或流水线逻辑。输入为两个权重字 $u, v \in \mathbb{K}$, 输出为

$$\text{Compose}(u, v) = u \otimes v.$$

不同半环对应不同数据通路。

半环	$\oplus$	$\otimes$	硬件含义
布尔半环	$\vee$	$\wedge$	可达性路径
$\mathbb{N}$	$+$	$\times$	路径计数
$\mathbb{R}$	$+$	$\times$	线性权重传播
max-plus	max	$+$	最长路径或最大得分
min-plus	min	$+$	最短路或代价传播

注记

CoPU 的一个重要特点是它把“乘法器”抽象为 $\otimes$ ，把“加法器”抽象为 $\oplus$ 。普通矩阵处理器通常默认 $\oplus = +$ 、 $\otimes = \times$ 。对应处理器则允许同一个 join-reduce 结构服务于布尔可达性、路径计数、最短路、图消息传递和线性传播。

命题 9.4: 合成单元的局部正确性

若合成单元实现的运算正是半环乘法 $\otimes$ ，则它对每个 join 输出事件对 (e_B, e_A) 产生的权重等于复合对应定义中的路径权重。

证明. 复合对应的定义规定

$$w_{B \circ A}(e_B, e_A) = w_B(e_B) \otimes w_A(e_A).$$

合成单元的输入为 $w_B(e_B)$ 与 $w_A(e_A)$ ，输出按定义为二者的 $\otimes$ 。因此输出权重与数学定义一致。 $\square$

9.6 推前聚合与正规化

原始复合对应保留路径事件

$$(e_B, e_A).$$

它的源端点和目标端点为

$$s_{B \circ A}(e_B, e_A) = s_A(e_A), \quad t_{B \circ A}(e_B, e_A) = t_B(e_B).$$

如果机器只需要输出原始路径事件流，那么 join 与 compose 已经足够。如果机器需要输出矩阵影子或正规化对应，则必须对相同端点对 (x, z) 上的路径权重做聚合。

定义 9.7: 推前聚合单元

推前聚合单元接收带端点对的路径事件流

$$(x, z, q), \quad q \in \mathbb{K},$$

并对每个端点对 $(x, z) \in X \times Z$ 输出

$$Q_{zx} = \bigoplus_{\text{路径 } p: x \rightarrow z} q(p).$$

这里 $\oplus$ 是半环加法。

定理 9.2: join-compose-reduce 实现正规化复合

设 $\mathcal{A} : X \rightsquigarrow Y$, $\mathcal{B} : Y \rightsquigarrow Z$ 为有限加权对应。若 join 单元枚举 $E_B \times_Y E_A$, compose 单元实现 $\otimes$, reduce 单元按端点对 (x, z) 实现 $\oplus$ 聚合, 则整个流水线输出的矩阵条目为

$$Q_{zx} = M(\mathcal{B} \circ \mathcal{A})_{zx}.$$

因此, 若 $A = M(\mathcal{A})$ 且 $B = M(\mathcal{B})$, 则

$$Q = BA.$$

证明.

Step 1. 对固定端点对 (x, z) , 流水线输出的所有贡献来自满足

$$s_A(e_A) = x, \quad t_B(e_B) = z, \quad t_A(e_A) = s_B(e_B)$$

的事件对 (e_B, e_A) 。

Step 2. join 正确性保证这些事件对恰好是复合事件空间 $E_B \times_Y E_A$ 中从 x 到 z 的事件。

Step 3. 对每个这样的事件对, compose 单元输出

$$w_B(e_B) \otimes w_A(e_A),$$

这正是复合对应的路径权重。

Step 4. reduce 单元按 (x, z) 聚合所有上述贡献, 因此

$$Q_{zx} = \bigoplus_{\substack{(e_B, e_A) \in E_B \times_Y E_A \\ s_A(e_A) = x, t_B(e_B) = z}} w_B(e_B) \otimes w_A(e_A).$$

Step 5. 右边正是矩阵影子 $M(\mathcal{B} \circ \mathcal{A})_{zx}$ 的定义。因此第一条结论成立。

Step 6. 由第六章的影子相容性,

$$M(\mathcal{B} \circ \mathcal{A}) = M(\mathcal{B})M(\mathcal{A}) = BA.$$

所以若输入对应的矩阵影子为 A, B , 流水线的正规化输出就是矩阵乘积 BA 。

□



图 9.3 join-compose-reduce 流水线。若最后执行 reduce, 则输出正规化矩阵影子; 若跳过 reduce, 则输出原始路径事件流。

9.7 CoPU 的最小机器状态

前面的数学操作可以落实为一个最小机器状态。为避免一开始陷入复杂处理器细节，我们先定义单核 CoPU 的抽象状态。

定义 9.8: CoPU 单核状态

一个单核 CoPU 的状态由以下部分组成：

$$S = (O, E, I_s, I_t, R, C).$$

其中：

1. O 是对象表；
2. E 是事件存储；
3. I_s 是源纤维索引；
4. I_t 是目标纤维索引；
5. R 是 reduce 缓冲区或输出表；
6. C 是控制寄存器，记录当前半环、对象空间、事件段、索引段和指令状态。

硬件解释

实际实现中， E 通常不是一个单一数组，而是若干事件段：输入对应 E_A 、输入对应 E_B 、中间路径事件流、输出事件段。 R 可以是哈希表、排序缓冲、按端点对寻址的稠密小表，或者分块 sparse accumulator。

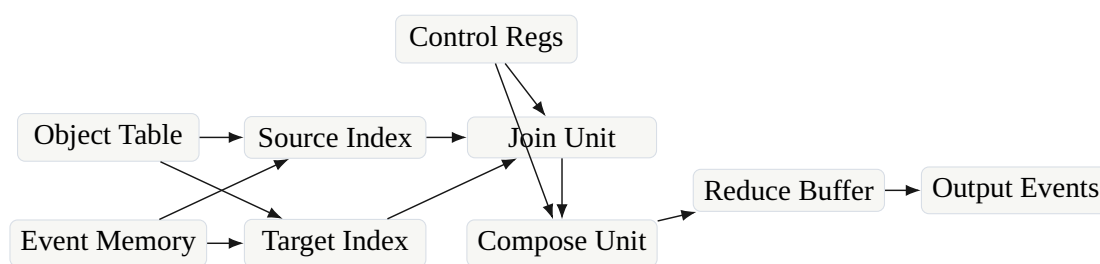


图 9.4 单核 CoPU 的抽象状态与基本数据通路。

9.8 指令语义

为了把 CoPU 变成可编程原型，需要定义一组最小指令。这里的指令不是最终 ISA，只是数学语义清晰的原型接口。

定义 9.9: CoPU 原型指令集

CoPU v0 的核心指令如下。

指令	语义
LOAD_OBJ	载入对象空间的一段对象记录。
LOAD_EVENT	载入事件记录，包括源端点、目标端点和权重。
BUILD_SRC	对事件段建立源纤维索引 $s^{-1}(x)$ 。
BUILD_DST	对事件段建立目标纤维索引 $t^{-1}(y)$ 。
FIBER_JOIN	对两个事件段按共同中间对象执行纤维 join。
WEIGHT_COMPOSE	对 join 产生的事件对执行半环乘法 $\otimes$ 。
PUSH_REDUCE	对相同输出端点对执行半环加法 $\oplus$ 。
EMIT_PATHS	不正规化，直接输出路径事件流。
NORMALIZE	把路径事件流正规化为坐标对应或矩阵影子。
QUOTIENT	按给定等价关系合并事件或对象。
TRACE_LOOP	只聚合闭合端点路径，用于迹和闭合路径统计。

每条指令应带有前置条件和后置条件。比如 FIBER_JOIN 的前置条件是两个输入事件段已经分别建立了需要的索引；后置条件是输出流中的每一对事件满足中间端点相等。

命题 9.5: FIBER_JOIN 的局部语义

若 FIBER_JOIN 的输入为 A 的目标索引和 B 的源索引，则它的输出满足如下后置条件：

$$(e_B, e_A) \text{ 被输出} \iff s_B(e_B) = t_A(e_A).$$

证明. 这正是定理 9.1 的指令化表述。指令按照中间端点 y 遍历两个桶的笛卡尔积，所以输出一定满足中间端点相等；反过来，任何中间端点相等的事件对都会在相应桶中同时出现，并被枚举。□

命题 9.6: PUSH_REDUCE 的局部语义

若输入路径流为

$$(x_i, z_i, q_i)_{i \in I},$$

则 PUSH_REDUCE 对每个端点对 (x, z) 输出

$$Q_{zx} = \bigoplus_{i \in I: x_i = x, z_i = z} q_i.$$

证明. PUSH_REDUCE 的定义就是按键 (x, z) 分组，再对每组权重执行半环加法 $\oplus$ 。由于 $\oplus$ 交换结合，组内聚合结果与事件到达顺序无关。□

9.9 微架构：单 tile 与多 tile

单核 CoPU 可以直接执行小规模对应复合。为了处理更大事件空间，需要把事件和对象分布到多个 tile。每个 tile 管理对象空间的一部分或事件空间的一部分，并通过片上网络交换路径事件。

定义 9.10: tile 局部状态

一个 CoPU tile 的局部状态包括

$$S_i = (O_i, E_i, I_{s,i}, I_{t,i}, R_i, Q_i),$$

其中 Q_i 是输入输出事件队列。tile 的局部计算仍然是 join-compose-reduce，但 join 所需的两个事件桶可能位于不同 tile 上。

多 tile 设计有两种基本分解方式。

第一种是按中间对象 Y 分解。每个 tile 负责一部分中间端点 Y_i 。这样 join 代价天然局部化，因为第 i 个 tile 只处理

$$\bigsqcup_{y \in Y_i} t_A^{-1}(y) \times s_B^{-1}(y).$$

第二种是按输出端点对 (x, z) 分解。这样 reduce 更局部，但 join 阶段可能需要更多事件搬运。

命题 9.7: 按中间对象分区的 join 局部性

若 Y 被划分为互不相交的子集

$$Y = Y_1 \sqcup \cdots \sqcup Y_P,$$

并且第 i 个 tile 持有所有满足 $t_A(e_A) \in Y_i$ 的 A 事件和所有满足 $s_B(e_B) \in Y_i$ 的 B 事件，则纤维 join 阶段不需要跨 tile 读取事件。

证明. 任意可复合事件对 (e_B, e_A) 满足

$$y = t_A(e_A) = s_B(e_B).$$

该 y 属于唯一的分区 Y_i 。根据分配假设， e_A 与 e_B 均保存在第 i 个 tile。因此该事件对的 join 可以在第 i 个 tile 内完成，不需要跨 tile 读取输入事件。 $\square$

边界

按中间对象分区可以局部化 join，但不一定局部化 reduce。因为路径事件的输出端点对 (x, z) 可能分布在许多 tile 的输出缓冲中。若最终必须得到正规化矩阵影子，仍然需要一次按 (x, z) 的归并或路由。

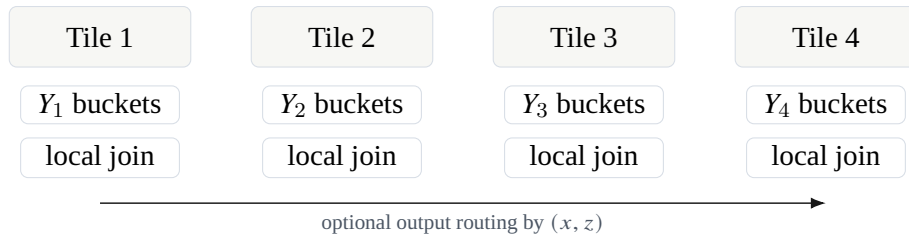


图 9.5 按中间对象分区。每个 tile 负责一组中间纤维桶，因此 join 可以局部完成；正规化输出可能仍需跨 tile reduce。

9.10 商压缩单元

第八章说明, 当事件空间带有群作用或等价结构时, 可以通过商压缩降低表示复杂度。硬件上这对应 QUOTIENT 指令或 quotient engine。它的作用不是简单删除事件, 而是把等价事件合并, 并正确聚合权重。

定义 9.11: 可合并等价关系

设 $\mathcal{A} = (X \xleftarrow{s} E \xrightarrow{t} Y, w)$ 。事件空间上的等价关系 $\sim$ 称为端点相容的, 若

$$e \sim e' \implies s(e) = s(e'), \quad t(e) = t(e').$$

在这种情况下, 可以定义商事件空间 $E/\sim$, 并令商权重为

$$\bar{w}([e]) = \bigoplus_{e' \in [e]} w(e').$$

命题 9.8: 端点相容商不改变矩阵影子

若 $\sim$ 是端点相容等价关系, 则商对应

$$\bar{\mathcal{A}} = (X \xleftarrow{\bar{s}} E/\sim \xrightarrow{\bar{t}} Y, \bar{w})$$

满足

$$M(\bar{\mathcal{A}}) = M(\mathcal{A}).$$

证明.

Step 1. 由于等价关系端点相容, 若 $e' \in [e]$, 则 $s(e') = s(e)$ 且 $t(e') = t(e)$ 。因此可以定义

$$\bar{s}([e]) = s(e), \quad \bar{t}([e]) = t(e),$$

且定义与代表元无关。

Step 2. 对固定端点对 (x, y) , 商对应的矩阵影子条目为

$$M(\bar{\mathcal{A}})_{yx} = \bigoplus_{[e]: \bar{s}([e])=x, \bar{t}([e])=y} \bar{w}([e]).$$

Step 3. 代入商权重定义, 得到

$$M(\bar{\mathcal{A}})_{yx} = \bigoplus_{[e]: \bar{s}([e])=x, \bar{t}([e])=y} \bigoplus_{e' \in [e]} w(e').$$

Step 4. 端点相容性保证这些等价类正好把所有满足 $s(e') = x, t(e') = y$ 的事件划分成不交块。因此上式等于

$$\bigoplus_{e': s(e')=x, t(e')=y} w(e') = M(\mathcal{A})_{yx}.$$

□

硬件解释

quotient engine 的最小实现可以是 “sort by key + reduce”。key 是等价类标签或端点对加类型标签。若等价类由规则生成，例如卷积中的相同偏移 r ，则 quotient engine 不必真的比较所有事件；它只需在生成阶段复用同一规则编号。

9.11 迹单元与闭合路径

第七章定义了自对应的迹与路径 zeta。硬件中对应的最小操作是闭合路径检测：只保留源端点等于目标端点的事件或路径。

定义 9.12: 迹循环单元

对于自对应 $\mathcal{A} : X \rightsquigarrow X$ ，迹循环单元接收事件流 $(s(e), t(e), w(e))$ ，只对满足

$$s(e) = t(e)$$

的事件执行聚合，输出

$$\text{Trace}(\mathcal{A}) = \bigoplus_{e: s(e)=t(e)} w(e).$$

命题 9.9: 迹单元正确性

迹循环单元的输出等于对应迹 $\text{Tr}(\mathcal{A})$ 。

证明. 对应迹在第七章定义为所有闭合事件权重的 $\oplus$ 聚合。迹循环单元正是过滤条件 $s(e) = t(e)$ 后对权重执行同一聚合，因此输出等于定义中的 $\text{Tr}(\mathcal{A})$ 。□

对于 $\text{Tr}(\mathcal{A}^k)$ ，机器可以重复使用 join-compose 流水线生成长度为 k 的路径，然后由迹单元过滤闭合路径。第一版原型不必高效实现所有 k ，但应当支持 $k = 1, 2$ ，因为这已经可以验证闭合路径语言与矩阵迹幂之间的联系。

9.12 代价模型

一颗原型芯片必须有可解释的性能指标。CoPU 的主要代价不应只用 TOPS 描述，而应使用事件数、纤维负载、join 数、reduce 冲突和物化规模。

定义 9.13: CoPU 复合的基本代价量

设 $\mathcal{A} : X \rightsquigarrow Y$ ， $\mathcal{B} : Y \rightsquigarrow Z$ 。定义：

$$\begin{aligned} m_A &= |E_A|, & m_B &= |E_B|, \\ L_y &= |t_A^{-1}(y)|, & R_y &= |s_B^{-1}(y)|, \\ J &= \sum_{y \in Y} L_y R_y. \end{aligned}$$

其中 J 是路径事件数，也就是 join 输出数。进一步，令

$$O_{xz} = |\{(e_B, e_A) : s_A(e_A) = x, t_B(e_B) = z, t_A(e_A) = s_B(e_B)\}|.$$

这是 reduce 阶段端点对 (x, z) 上的冲突负载。

命题 9.10: 单核顺序执行的主导周期数

在以下简化假设下: 一次事件读取、一次权重合成、一次 reduce 更新均为常数周期, 并且不考虑缓存未命中, 则一次正规化复合的主导周期数满足

$$T_{\text{seq}} = O(m_A + m_B + |Y| + J + N_{\text{red}}),$$

其中 N_{red} 是 reduce 阶段实际执行的聚合更新次数, 通常满足 $N_{\text{red}} \leq J$ 。

证明. 建立两个索引需要扫描输入事件并处理对象桶, 代价为 $O(m_A + m_B + |Y|)$ 。join 阶段输出 J 个事件对, 逐个处理的代价为 $O(J)$ 。compose 阶段对每个 join 输出执行一次半环乘法, 仍为 $O(J)$, 可并入 $O(J)$ 。reduce 阶段对路径事件执行聚合更新, 次数为 N_{red} 。相加即得所述估计。□

注记

若 reduce 对每条路径都更新一次累加器, 则 $N_{\text{red}} = J$ 。若路径流先在局部缓冲中做预聚合, 再写回全局输出, 则全局写回次数可能接近非零输出端点对数, 而局部更新次数仍为 J 。因此在实验中需要分别报告 local reduction 与 global writeback。

定理 9.3: 按中间对象并行的理想负载

设 Y 被分配到 P 个 tile, $Y = Y_1 \sqcup \cdots \sqcup Y_P$ 。若第 i 个 tile 处理所有 $y \in Y_i$ 的 join, 则第 i 个 tile 的 join 负载为

$$J_i = \sum_{y \in Y_i} L_y R_y.$$

在理想流水线和无跨 tile 输入搬运的假设下, join 阶段的并行时间由

$$\max_{1 \leq i \leq P} J_i$$

主导。

证明. 第 i 个 tile 需要枚举的事件对正是所有 $y \in Y_i$ 的桶笛卡尔积, 其数量为 J_i 。在每个 tile 每周处理常数个事件对的理想模型下, 该 tile 的处理时间与 J_i 成正比。所有 tile 并行执行, 因此整体 join 阶段需要等待最慢 tile 完成, 主导项为 $\max_i J_i$ 。□

这个定理说明, CoPU 的负载均衡问题不是简单均分事件数, 而是均分乘积负载 $L_y R_y$ 。若某个中间对象 y 的两个纤维都很大, 它会形成 join 热点。

边界

实验报告若只给平均纤维大小是不够的。join 代价由 $L_y R_y$ 控制, 少数大纤维会支配总成本。因此必须报告纤维负载分布, 例如最大值、分位数和 $\sum_y L_y R_y$ 。

9.13 与 SpMV、Graph Accelerator 和 GPU 的区别

CoPU 很容易被误解为稀疏矩阵加速器或图处理器。本节把区别说清楚。

稀疏矩阵加速器通常输入已经正规化的条目

$$(i, j, A_{ij}).$$

它的核心操作是把这些条目应用到向量或做稀疏乘法。CoPU 输入的是事件

$$e = (s(e), t(e), w(e), \text{type}, \text{tag}),$$

并且允许多个事件拥有同一端点对。它可以选择保留路径事件，也可以选择正规化成矩阵影子。

图处理器通常围绕顶点和边执行遍历、消息传递或图算法。CoPU 与图处理器有交集，因为一个无权对应就是一张二部图。但 CoPU 的基础对象是可复合对应，而不只是单层图。它关心的是

$$E_B \times_Y E_A,$$

也就是路径事件空间，以及这个路径空间的权重半环语义。

GPU 强在高吞吐、规则内存访问和大规模 SIMD/SIMT。CoPU 的目标不是在 dense GEMM 上战胜 GPU，而是在不展开矩阵的结构化对应程序上减少物化、访存与无意义匹配。

命题 9.11: CoPU 优势的必要条件

若一个算子的对应表示没有比矩阵展开更短，纤维负载没有结构性稀疏或可均衡性，并且最终输出必须完整正规化为稠密矩阵，则 CoPU 相对 dense 矩阵处理器没有原则性优势。

证明. 在这些条件下，CoPU 不能通过事件表示减少输入规模，不能通过纤维结构减少 join 规模，也不能通过跳过正规化减少输出规模。此时它仍需完成与矩阵乘法同阶的路径枚举和聚合，还承担额外的索引与事件调度成本。因此没有原则性优势。□

这条命题很重要，因为它防止本体系被夸大。CoPU 的意义只在真实结构存在时出现：局部性、稀疏性、生成规则、低秩中间对象、商对称、动态候选或不需要完全正规化的路径语义。

9.14 编译器接口：从对应程序到事件流

硬件不能直接读数学定义。需要一个中间表示，把高层对应程序降到对象表、事件表和指令序列。我们称之为 Correspondence IR，简称 C-IR。

定义 9.14: C-IR 程序

一个最小 C-IR 程序包含：

1. 对象声明：定义对象空间 X, Y, Z 及其编号；
2. 半环声明：指定 $\oplus, \otimes, 0, 1$ ；
3. 对应声明：给出事件生成规则或显式事件表；
4. 索引声明：指定需要构造的源索引或目标索引；
5. 复合声明：指定哪些对应要复合，是否正规化；
6. 输出声明：指定输出路径事件流、正规化对应、矩阵影子或迹量。

一个典型 C-IR 片段可以抽象写成：

```
semiring real_plus_times;
object X size n;
object Y size m;
```



```

object Z size p;
correspondence A: X -> Y from events EA;
correspondence B: Y -> Z from events EB;
build_dst A;
build_src B;
C = compose B A normalize;
emit C;

```

命题 9.12: C-IR 到 CoPU 指令的保语义条件

若 C-IR 中每个对象声明被编译为对象表, 每个显式事件或规则生成事件被编译为事件表, 每个索引声明调用正确的纤维索引构造, 每个复合声明调用 join-compose-reduce 流水线, 则编译后的 CoPU 程序输出与 C-IR 的对应语义一致。

证明. 对象声明只负责建立有限集合元素与机器编号之间的双射, 因此不改变数学对象。事件声明把每个事件的源端点、目标端点和权重写入事件表, 保留了对应定义。索引声明由命题 9.1 保证正确。复合声明由定理 9.2 保证与对应复合或正规化复合一致。因此逐条指令组合后, 输出保持 C-IR 的数学语义。□

9.15 FPGA 原型的最小闭环

第一版原型不应试图一次性实现全部语言。一个合理的闭环是: 输入两个小型对应, 构造索引, 执行 join-compose-reduce, 输出正规化矩阵影子, 并与软件参考结果逐项比较。

硬件解释

建议 RTL 文件结构如下:

```

rtl/
  copu_top.sv
  object_table.sv
  event_memory.sv
  fiber_index_builder.sv
  fiber_bucket_reader.sv
  fiber_join.sv
  weight_compose.sv
  push_reduce.sv
  quotient_engine.sv
  trace_loop.sv
  semiring_alu.sv
  stream_fifo.sv
sim/
  tb_fiber_index.py
  tb_fiber_join.py
  tb_weight_compose.py
  tb_push_reduce.py
  tb_copu_compose.py
compiler/

```



```

cir.py
lower_to_events.py
emit_vectors.py
reference_semantics.py

```

定义 9.15: 原型正确性测试

一次 CoPU 原型测试包含以下数据:

1. 输入对象空间 X, Y, Z ;
2. 输入事件表 E_A, E_B 与权重;
3. 软件参考输出

$$Q_{zx} = \bigoplus_{y \in Y} \bigoplus_{\substack{e_A: x \rightarrow y \\ e_B: y \rightarrow z}} w_B(e_B) \otimes w_A(e_A);$$

4. 硬件输出 $\hat{Q}$;
5. 校验条件 $\hat{Q} = Q$ 。

命题 9.13: 逐项比较足以验证有限测试实例

对固定有限输入实例, 若硬件输出 $\hat{Q}$ 与软件参考 Q 在所有端点对 (x, z) 上相等, 则该实例上的正规化复合计算正确。

证明. 固定实例的数学输出就是端点对集合 $X \times Z$ 上的函数 $Q: X \times Z \rightarrow \mathbb{K}$ 。若对每个 (x, z) 都有 $\hat{Q}_{zx} = Q_{zx}$, 则两个函数完全相同。因此硬件在该实例上的输出等于数学定义的输出。□

9.16 实验指标

CoPU 原型必须报告能够反映对应语言优势的指标, 而不是只报告峰值吞吐。至少应包括:

1. 输入事件数 m_A, m_B ;
2. 中间对象数 $|Y|$;
3. 纤维负载分布 L_y, R_y ;
4. join 数 $J = \sum_y L_y R_y$;
5. 输出端点对数与 reduce 冲突分布 O_{xz} ;
6. 矩阵展开比

$$\rho_{\text{mat}} = \frac{|Y||X|}{|E_A|} \quad \text{或} \quad \frac{|Z||Y|}{|E_B|};$$

7. 中间物化比

$$\rho_{\text{path}} = \frac{|E_B \times_Y E_A|}{|Z||X|};$$

8. FPGA 资源: LUT、FF、BRAM、DSP;
9. 时钟频率、周期数、延迟、吞吐;
10. 事件读取次数、输出写回次数、索引构造开销。

边界

如果实验只展示“稀疏矩阵很稀疏，所以事件数少”，说服力不足。必须展示至少一种矩阵展开无法自然表达的结构：例如由规则生成的卷积对应、动态候选 attention 对应、带商压缩的平移等价事件，或不需要完全正规化的路径语义。

9.17 本章小结

本章把纤维对应代数翻译成了处理器结构。对象表保存对象空间，事件表保存对应，纤维索引保存端点分解。纤维 join 枚举纤维积，权重合成实现半环乘法，推前聚合实现半环加法。三者合起来给出 join-compose-reduce 流水线，并严格实现正规化复合。

这一章最重要的结论是定理 9.2：CoPU 的基本流水线不是矩阵乘法的经验近似，而是对应复合的直接硬件实现；当最后进行正规化时，它的矩阵影子正好等于矩阵乘积。由此，矩阵乘法被解释为一种特殊输出模式，而不是机器必须采用的内部语言。

接下来一章将回到整体研究路线：如何从这套讲义走向可验证原型、毕业设计、论文和更长期的芯片项目。

研究路线：从形式语言到原型系统

前面九章已经建立了对应计算的基本语法：对象空间、事件空间、半环权重、加权对应、纤维复合、正规化影子、迹与路径谱、表达复杂度以及 CoPU 的最小硬件结构。本章的任务不是作一般性的展望，而是把这些概念组织成一个可执行的研究计划。所谓可执行，至少包含四层含义：第一，数学语言必须继续闭合，不能留下依赖直觉的定义；第二，编译器中间表示必须有形式语义，而不是只作为工程格式；第三，硬件原型必须有明确的输入、输出、状态和正确性条件；第四，论文主张必须能被实验与定理共同支撑。

本章仍然采用讲义式写法。我们先说明为什么研究路线本身也需要数学化；随后把项目拆成若干层次：语言层、IR 层、解释器层、硬件层、实验层和论文层。每一层都给出最小对象、待证明命题、失败边界与交付物。这样做的目的，是避免把“替代矩阵语言”变成一个过大的口号。真正可以推进的路线应当是：每向前一步，都能被定义、定理、代码或电路图检验。

10.1 为什么研究路线也要形式化

一个新的计算语言如果只停留在概念层面，很容易产生两种误解。第一种误解是把它理解成新的术语系统：把矩阵称为对应，把稀疏矩阵称为事件空间，把矩阵乘法称为纤维复合。这样没有改变计算对象，只是改名。第二种误解是把它直接推到宏大的系统叙事：下一代芯片、新型智能计算机、非冯诺依曼架构。这样虽然方向很大，但缺少中间的验证环节。

对应计算要避免这两种误解，必须采用一种更严格的路线：

数学对象 $\rightarrow$ 形式语义 $\rightarrow$ 参考实现 $\rightarrow$ 硬件实现 $\rightarrow$ 实验验证.

这里每一个箭头都应当有一个明确的正确性陈述。例如，从数学对象到形式语义，需要证明 IR 中的语句确实解释为对应；从形式语义到参考实现，需要证明解释器输出的事件表与数学定义一致；从参考实现到硬件实现，需要证明硬件状态转移实现同一个 join-compose-reduce 过程；从硬件实现到实验验证，需要说明测量指标与前面的复杂度模型一致。

说明

本章的基本原则是：路线不是宣传，而是证明义务的排列顺序。若某个目标不能转化为定义、引理、程序测试、RTL 模块或实验指标，它暂时不应作为主论文的核心贡献。

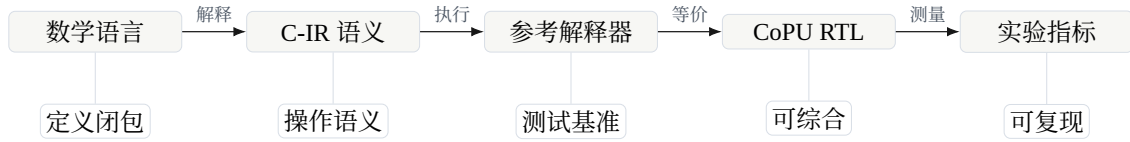


图 10.1 对应计算项目的验证链条。每一层都需要有独立的正确性条件。

10.2 第一个闭环：语言层

语言层的目标不是继续增加术语，而是使前面引入的术语形成闭合体系。所谓闭合，是指每一个常用操作都能在体系内部定义，并且与矩阵语言在相应情形下相容。对于当前讲义，语言层至少包括以下对象：

层次	基本对象
集合层	$X, Y, Z, f : X \rightarrow Y, f^{-1}(y)$
半环层	$(\mathbb{K}, \oplus, \otimes, 0, 1)$
对应层	$X \xleftarrow{s} E \xrightarrow{t} Y, w : E \rightarrow \mathbb{K}$
复合层	$E_B \times_Y E_A, \mathcal{B} \circ \mathcal{A}$
正规化层	$M(\mathcal{A}), N(\mathcal{A}), \mathcal{B} \star \mathcal{A}$
谱层	$\text{Tr}(\mathcal{A}^k), \zeta_{\mathcal{A}}(u)$
复杂度层	$ E , \sum_y t_A^{-1}(y) s_B^{-1}(y) , \text{size}_{\text{rule}}$

这些对象在前面章节中已经出现，但后续仍要继续整理。尤其需要注意：语言层不应把所有内容都归约成矩阵影子。矩阵影子是检查相容性的工具，不是最终对象。

定义 10.1: 语言闭包

称一组对应计算概念在给定范围内是闭合的，如果满足以下条件。

1. 对任意两个可复合对象，复合仍然在同一语言中可定义。
2. 单位、结合律、反向、并合、正规化等基本操作均有内部定义。
3. 每个操作都有矩阵影子相容性命题，说明在矩阵化后得到传统矩阵语言中的对应操作。
4. 每个复杂度陈述均说明它度量的是事件表示、规则表示还是矩阵展开表示。

这个定义看起来形式化，但它有实际作用。若一个新操作不能通过这四条检查，它通常还不适合进入主体系。例如，只说“商对应可以压缩结构”是不够的；必须说明商空间是什么、权重如何下推、复合是否与取商相容、矩阵影子是否保持所需性质、压缩率怎样计量。

命题 10.1: 语言闭包的检查顺序

设某个新构造 $\mathcal{T}$ 被加入对应计算语言。若 $\mathcal{T}$ 旨在成为基本构造，则应先证明存在性与良定义性，再证明矩阵影子相容性，最后讨论复杂度优势。

证明.

Step 1. 存在性与良定义性说明 $\mathcal{T}$ 在给定输入下确实产生语言内部的对象。例如，若 $\mathcal{T}$ 是复合，则输出必须仍是某个加权对应。

Step 2. 矩阵影子相容性说明 $\mathcal{T}$ 没有背离传统矩阵思想。例如，复合的矩阵影子应当等于矩阵影子的乘积。

Step 3. 复杂度优势只有在前两步成立后才有意义。否则所谓“更高效”可能只是因为输出对象已经变了。

□

安排

后续讲义扩写时，每引入一个新概念，都应使用以下四问：它的输入是什么？输出是什么？它的矩阵影子是什么？它的计算成本由哪个纤维或规则控制？这四问可以防止概念漂移。

10.3 第二个闭环：C-IR 中间语言

数学讲义给出的是对象与定理，但硬件原型不能直接读取数学表达式。中间必须有一个 IR，即 intermediate representation。为了避免和 C 语言混淆，本讲义暂称它为 C-IR，其中 C 表示 correspondence。C-IR 的目的不是做一个完整编程语言，而是提供足够少、足够精确的语句，使对象、事件、纤维索引和复合操作可以被解析、执行和综合。

10.3.1 C-IR 的最小语法

C-IR 的第一版不需要循环、类型推断和复杂优化。它只需要表达以下几类信息：对象空间声明、事件空间声明、端点映射、权重、复合、正规化、取商和迹计算。

定义 10.2: C-IR 程序的抽象语法

一个最小 C-IR 程序由若干语句组成，其抽象语法可写为

$$\begin{aligned} S ::= & \text{object } X[n] \\ & | \text{event } E[m] : X \rightarrow Y \\ & | \text{source } E[e] := x \\ & | \text{target } E[e] := y \\ & | \text{weight } E[e] := a \\ & | C := \text{compose}(B, A) \\ & | D := \text{normalize}(C) \\ & | Q := \text{quotient}(C, \sim) \\ & | r := \text{trace}(A, k). \end{aligned}$$

这里 X, Y 是对象空间， E 是事件空间， A, B, C, D, Q 是对应变数。

这个语法刻意很小。它只覆盖讲义前九章已经定义过的内容。以后当然可以加入规则生成、局部 attention、卷积模板、图导入等高级语句，但第一版应当先把显式事件复合做正确。

例 10.1: 一个二步路径程序

考虑两个对应

$$X \leftarrow E_A \rightarrow Y, \quad Y \leftarrow E_B \rightarrow Z.$$

C-IR 程序可以写成

```
object X[2], Y[3], Z[2];
event A[4] : X → Y;
event B[5] : Y → Z;
C := compose(B, A); D := normalize(C).
```

数学语义是先形成路径事件空间 $E_B \times_Y E_A$ ，然后把同一端点对 (x, z) 上的路径贡献聚合。

10.3.2 C-IR 的解释函数

语法本身没有意义，必须给出解释函数。设 State 表示解释器状态，包含已声明的对象表、事件表、端点映射、权重表和对应变表。每条语句都是状态变换。

定义 10.3: C-IR 状态

一个 C-IR 状态记为

$$\Sigma = (O, \mathcal{E}, C),$$

其中 O 是对象空间表， $\mathcal{E}$ 是事件空间与事件记录表， C 是已经构造的对应变表。若 $A \in C$ ，则 $\Sigma(A)$ 是一个加权对应

$$X_A \xleftarrow{s_A} E_A \xrightarrow{t_A} Y_A, \quad w_A : E_A \rightarrow \mathbb{K}.$$

定义 10.4: compose 语句的数学语义

设状态 Σ 中已有

$$\Sigma(A) = X \xleftarrow{s_A} E_A \xrightarrow{t_A} Y, \quad \Sigma(B) = Y \xleftarrow{s_B} E_B \xrightarrow{t_B} Z.$$

语句

$$C := \text{compose}(B, A)$$

把状态更新为 Σ' ，其中

$$\Sigma'(C) = X \xleftarrow{s_C} E_C \xrightarrow{t_C} Z,$$

并且

$$E_C = E_B \times_Y E_A, \quad s_C(e_B, e_A) = s_A(e_A), \quad t_C(e_B, e_A) = t_B(e_B), \\ w_C(e_B, e_A) = w_B(e_B) \otimes w_A(e_A).$$

其他变量保持不变。

定义 10.5: normalize 语句的数学语义

若 $C = X \xleftarrow{s_C} E_C \xrightarrow{t_C} Z$ ，则

$$D := \text{normalize}(C)$$

产生一个矩阵化正规对应

$$D = X \leftarrow E_D \rightarrow Z,$$

其中

$$E_D = \{(x, z) : \bigoplus_{e \in E_C, s_C(e)=x, t_C(e)=z} w_C(e) \neq 0\},$$

端点为投影，权重为

$$w_D(x, z) = \bigoplus_{\substack{e \in E_C \\ s_C(e)=x, t_C(e)=z}} w_C(e).$$

若半环中没有合适的“非零筛选”语义，也可以取 $E_D = X \times Z$ ，允许零权重事件存在。

命题 10.2: compose-normalize 与正规化复合一致

设 $A : X \rightarrow Y$ 、 $B : Y \rightarrow Z$ 是 C-IR 状态中的两个对应。执行

$$C := \text{compose}(B, A), \quad D := \text{normalize}(C)$$

得到的 D 等于第六章中的正规化复合 $B \star A$ 。

证明.

Step 1. 按 compose 的语义， C 的事件是所有可连接事件对 (e_B, e_A) ，满足 $s_B(e_B) = t_A(e_A)$ 。

Step 2. 按 normalize 的语义， D 在端点对 (x, z) 上的权重是所有满足

$$s_A(e_A) = x, \quad s_B(e_B) = t_A(e_A), \quad t_B(e_B) = z$$

的路径权重之和。

Step 3. 因此

$$w_D(x, z) = \bigoplus_{y \in Y} \bigoplus_{\substack{e_A: x \rightarrow y \\ e_B: y \rightarrow z}} w_B(e_B) \otimes w_A(e_A),$$

这正是 $B \star A$ 的定义。

□

10.3.3 C-IR 的类型检查

C-IR 必须在执行前检查对象端点是否匹配。否则 compose 语句可能没有数学意义。

定义 10.6: 对应变量的类型

若 A 是从 X 到 Y 的对应，则记

$$A : X \Rightarrow Y.$$

这里的箭头 $\Rightarrow$ 表示对应类型，不表示函数。

命题 10.3: compose 类型规则

C-IR 中的复合语句

$$C := \text{compose}(B, A)$$

类型正确，当且仅当存在对象空间 X, Y, Z 使得

$$A : X \Rightarrow Y, \quad B : Y \Rightarrow Z.$$

此时 $C : X \Rightarrow Z$ 。

证明.

Step 1. 若 $A : X \Rightarrow Y$ 且 $B : Y \Rightarrow Z$, 则 $E_B \times_Y E_A$ 中的匹配条件 $s_B(e_B) = t_A(e_A)$ 有共同对象空间 Y , 纤维积良定义。

Step 2. 输出端点分别来自 $s_A : E_A \rightarrow X$ 与 $t_B : E_B \rightarrow Z$, 所以输出对应从 X 到 Z 。

Step 3. 反之, 如果两个对应的中间对象不同, 例如 $A : X \Rightarrow Y$ 、 $B : Y' \Rightarrow Z$ 且 $Y \neq Y'$, 则表达式 $s_B(e_B) = t_A(e_A)$ 没有共同的比较空间。除非额外给出 Y 与 Y' 的识别映射, 否则 compose 没有定义。

□

硬件解释

类型检查在硬件中对应端口协议。若一个事件流的目标对象编码与另一个事件流的源对象编码不是同一命名空间, fiber join 单元不能直接比较两者。C-IR 的类型系统应当在编译期消除这种错误, 而不是交给 RTL 在运行时处理。

10.4 第三个闭环：参考解释器

有了 C-IR 语义后, 下一步不是直接写 RTL, 而是先写一个参考解释器。参考解释器的职责是严格按数学定义执行 C-IR 程序, 生成期望输出。后续 RTL、FPGA 和 ASIC 仿真都应当以参考解释器为 oracle。

10.4.1 解释器的状态表示

参考解释器可以使用非常直接的数据结构。对象空间用整数区间表示, 事件表用数组表示, 纤维索引字典或 CSR 表示。设一个事件记录为

$$e = (s, t, w, \text{tag}).$$

对一个对应 $A : X \Rightarrow Y$, 解释器保存

$$A.E, \quad A.\text{src}[e], \quad A.\text{dst}[e], \quad A.w[e].$$

为了执行 compose , 还需要两个索引:

$$\text{byDst}_A(y) = \{e \in E_A : t_A(e) = y\},$$

$$\text{bySrc}_B(y) = \{e \in E_B : s_B(e) = y\}.$$

定义 10.7: 参考复合算法

给定 $A : X \Rightarrow Y$ 与 $B : Y \Rightarrow Z$ ，参考解释器的复合算法为：

```

C.events := [];
for y in Y :
    for eA in byDstA[y] :
        for eB in bySrcB[y] :
            append(src = eA.src, dst = eB.dst, w = eB.w ⊗ eA.w).

```

定理 10.1: 参考复合算法的正确性

参考复合算法输出的事件表与数学定义中的纤维积事件空间 $E_B \times_Y E_A$ 一一对应，并且端点与权重完全一致。

证明.

Step 1. 算法枚举所有 $y \in Y$ 。

Step 2. 在固定 y 下，内层枚举所有 $e_A \in t_A^{-1}(y)$ 和所有 $e_B \in s_B^{-1}(y)$ 。

Step 3. 因此每次 append 对应一个满足 $t_A(e_A) = y = s_B(e_B)$ 的事件对 (e_B, e_A) 。

Step 4. 反过来，任取 $(e_B, e_A) \in E_B \times_Y E_A$ ，令 $y = t_A(e_A) = s_B(e_B)$ 。算法在该 y 的循环中必然枚举到 e_A 与 e_B ，所以该路径事件被输出。

Step 5. 算法赋予的源端点为 $s_A(e_A)$ ，目标端点为 $t_B(e_B)$ ，权重为 $w_B(e_B) \otimes w_A(e_A)$ ，与对应复合定义一致。

□

定义 10.8: 参考正规化算法

给定 $C : X \Rightarrow Z$ ，参考解释器的正规化算法为：

```

T := empty map;
for e in C.events :
    key := (e.src, e.dst);    T[key] := T[key] ⊕ e.w;
return events encoded by T.

```

定理 10.2: 参考正规化算法的正确性

参考正规化算法输出的事件权重满足

$$w_D(x, z) = \bigoplus_{\substack{e \in E_C \\ s_C(e)=x, t_C(e)=z}} w_C(e).$$

因此它实现 $N(C)$ 。

证明.

Step 1. 算法按 key $(e.src, e.dst)$ 分桶。数学上，这个 key 正是端点对 $(s_C(e), t_C(e))$ 。

Step 2. 对固定 (x, z) ，所有落入同一桶的事件恰好是满足 $s_C(e) = x, t_C(e) = z$ 的事件。

Step 3. 桶内更新使用 $\oplus$ 聚合，因此最终值为这些事件权重的有限 $\oplus$ 和。

Step 4. 这与正规化定义逐项一致。

□

10.4.2 解释器测试的基本原则

参考解释器至少需要三类测试。第一类是定义测试，即小例子手算输出并与程序比较；第二类是相容性测试，即把对应转成矩阵后比较矩阵乘法结果；第三类是随机测试，即随机生成小事件表，检查 `compose-normalize` 与矩阵影子乘法一致。

安排

参考解释器不是为了高性能。它的任务是成为规范。只要解释器严格按定义实现，就可以用它生成 RTL testbench 的黄金输出。后续任何硬件优化都不应改变解释器语义。

10.5 第四个闭环：CoPU 原型规格

第九章已经给出 CoPU 的基本结构。本节把它收束成一个可实现的原型规格。这里的目标不是一次性做出完整芯片，而是定义一个最小闭环：输入两个有限事件空间，构建纤维索引，执行 `join-compose-reduce`，输出正规化后的事件空间或矩阵影子。

10.5.1 CoPU v0 的最小功能

定义 10.9: CoPU v0 功能集合

CoPU v0 至少支持以下功能。

1. 加载两个事件空间 $A : X \Rightarrow Y$ 与 $B : Y \Rightarrow Z$ 。
2. 按 t_A 构建 A 的目标纤维索引，按 s_B 构建 B 的源纤维索引。
3. 对每个 $y \in Y$ 枚举 $t_A^{-1}(y) \times s_B^{-1}(y)$ 。
4. 对每一对事件计算 $w_B \otimes w_A$ 。
5. 按端点对 (x, z) 执行 $\oplus$ 聚合。
6. 输出正规化事件表。

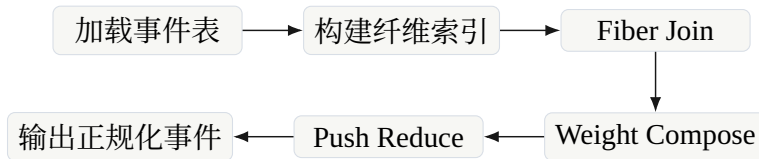


图 10.2 CoPU v0 的最小流水线。它只实现显式事件的正规化复合。

10.5.2 数据宽度与参数

第一版原型不应追求过大规模。较好的选择是把数据宽度固定，使 RTL 易于验证。

字段	建议宽度	含义
src	16 bit	源对象编号
dst	16 bit	目标对象编号
weight	16 或 32 bit	半环权重
tag	8 bit	事件类型或调试标记

若采用 16-bit 源端点、16-bit 目标端点、32-bit 权重与 8-bit tag，一个事件记录可打包到 72 bit，工程上可向上对齐为 80 或 96 bit。

硬件解释

第一版不要同时支持太多半环。建议只实现两个模式：整数加乘半环和 Boolean 半环。前者服务矩阵影子相容性测试，后者服务图可达性和关系复合测试。等 v0 通过后，再加入 min-plus 或 max-plus。

10.5.3 RTL 模块划分

CoPU v0 的 RTL 模块可以按定义直接划分，而不是按传统矩阵核划分：

模块	职责
event_memory	保存事件记录，支持顺序读写。
fiber_index	根据端点字段生成 CSR 型 offset 与 event id 列表。
fiber_join	读取同一个中间对象 y 上的两个纤维并枚举笛卡尔积。
weight_compose	按半环模式计算 $w_B \otimes w_A$ 。
push_reduce	以 (x, z) 为 key 聚合路径权重。
control_fsm	控制加载、索引构造、join、compose、reduce 与输出阶段。
copu_top	顶层模块，连接存储、流水线和外部接口。

命题 10.4: CoPU v0 的功能正确性目标

若 RTL 中的每个模块分别满足其局部规格，并且控制状态机按顺序执行加载、索引、join、compose、reduce，则 CoPU v0 输出等于参考解释器对同一输入执行 compose-normalize 的输出。

证明.

Step 1. event_memory 的正确性保证硬件读取到的事件表等于输入事件表。

Step 2. fiber_index 的正确性保证每个端点纤维被完整且不重复地列出。

Step 3. fiber_join 的正确性保证对每个 y 枚举 $t_A^{-1}(y) \times s_B^{-1}(y)$ 。

Step 4. weight_compose 的正确性保证每条路径贡献为 $w_B \otimes w_A$ 。

Step 5. push_reduce 的正确性保证同一端点对上的路径按 $\oplus$ 聚合。

Step 6. 这五步与参考解释器算法逐行对应，因此整体输出一致。

□

10.6 第五个闭环：实验设计

实验不是为了证明所有场景都优于 GPU，而是为了验证本讲义的核心主张：当计算有真实的对应结构时，不把结构展开成矩阵可以减少表示大小、中间物化和端点纤维复合成本。实验应当围绕这一主张设计。

10.6.1 实验对象的选择

第一组实验应选取四类最小但有代表性的结构化任务。

任务	对应表示	对比对象
循环移位	$E = \{x \mapsto x + 1\}$	$n \times n$ 矩阵展开
卷积	$E = \{(x, r) : r \in R\}$	Toeplitz 或 im2col
图消息传递	$E = \text{边空间}$	邻接矩阵 SpMV
局部 attention	$E = \text{候选 token 对}$	n^2 分数表

这四个任务的共同点是：它们都可以写成矩阵作用，但矩阵展开不是最自然的表示。

10.6.2 指标定义

实验指标必须和第八章的复杂度语言一致。至少需要记录：

指标	含义
$ X , Y , Z $	对象空间规模
$ E_A , E_B $	输入事件数
$ E_B \times_Y E_A $	路径事件数，即 join 产物规模
$\sum_y t_A^{-1}(y) s_B^{-1}(y) $	理论 join 成本
#reduce keys	正规化后的端点对数量
materialized matrix size	矩阵展开需要的条目数
cycles	RTL 或 FPGA 执行周期
BRAM/LUT/FF/DSP	FPGA 资源
power proxy	功耗估计或活动因子

边界

不要把实验主张写成“CoPU 总是比 GPU 快”。第一篇论文更合理的主张是：在某些结构化对应计算中，CoPU 避免了矩阵物化，并且硬件执行成本跟随纤维 join 规模，而不是跟随展开矩阵规模。

10.6.3 基线的设定

实验基线至少应包括三类。第一类是矩阵展开基线，即显式构造矩阵影子后做矩阵或稀疏矩阵计算；第二类是专门数据结构基线，例如图任务中的 CSR SpMV；第三类是参考解释器基线，用于确认输出正确。

安排

如果 CoPU v0 暂时不能在绝对速度上超过成熟库，这并不构成失败。v0 的核心指标是正确性、可综合性、表示规模、路径物化规模和纤维负载模型是否吻合。绝对性能留给 v1 之后的流水线优化、多 tile 并行和片上存储布局。

10.7 第六个闭环：论文结构

当数学、IR、解释器和硬件原型都形成闭环后，论文才有稳定结构。第一篇主论文不宜写成纯芯片论文，也不宜写成纯代数论文。更合理的结构是“数学语言 + 表达复杂度 + 原型硬件”的交叉论文。

10.7.1 主论文命题

主论文的中心命题可以写成：

Structured relational computation can be represented and executed directly as fibered correspondence composition, avoiding unnecessary matrix materialization while preserving the matrix shadow semantics.

中文意思是：结构化关系计算可以直接表示并执行为纤维对应复合，从而避免不必要的矩阵物化，同时保留矩阵影子语义。

这句话包含三个可检查部分。第一，“structured relational computation”对应实验任务和形式语言；第二，“fibered correspondence composition”对应数学定义与 C-IR；第三，“preserving the matrix shadow semantics”对应矩阵嵌入定理与 compose-normalize 正确性。

10.7.2 论文贡献的可审查表述

贡献	应当提供的证据
对应计算语言	定义、复合、单位、结合律、矩阵影子相容性
表达复杂度优势	卷积、循环移位、图、局部 attention 的分离例子与下界
C-IR 与参考解释器	操作语义、正确性定理、随机测试
CoPU 原型	RTL 模块、testbench、FPGA 资源与周期
实验验证	输出正确性、物化规模减少、纤维成本模型吻合

命题 10.5: 论文主张的闭合性

若一篇论文同时给出：对应复合的数学定义、矩阵影子相容性、C-IR 操作语义、参考解释器正确性、CoPU RTL 与实验指标，则其主张不依赖口号，而可以被定义、定理和实现共同审查。

证明。

Step 1. 数学定义说明论文讨论的对象是什么。

Step 2. 矩阵影子相容性说明新语言没有改变传统矩阵语义，而是保留其坐标化影子。

Step 3. C-IR 操作语义说明数学对象如何进入程序。

Step 4. 参考解释器正确性说明程序执行与数学定义一致。

Step 5. CoPU RTL 与实验指标说明这种程序可以被硬件执行，并且相关成本可以被测量。

Step 6. 因此论文主张可以逐层检查，而不是依赖整体叙事。

□

10.8 第七个闭环：版本规划

为了保持推进节奏，本项目可以分成三个版本。

10.8.1 v1：讲义与参考解释器

v1 的目标是让语言和软件闭合。交付物包括：

1. 当前讲义的完整第一版；
2. C-IR 的语法和操作语义；
3. Python 或 Rust 参考解释器；
4. 小规模随机测试；
5. 与矩阵影子的逐项一致性测试。

这一步完成后，项目不再只是设想，而有一个可运行的规范实现。

10.8.2 v2：CoPU RTL 与 FPGA

v2 的目标是让硬件闭合。交付物包括：

1. SystemVerilog 或 Chisel RTL；
2. Verilator/cocotb testbench；
3. 与参考解释器自动比对的测试流；
4. FPGA 综合资源、频率与周期统计；
5. 四类结构化任务的端到端 demo。

这一步完成后，项目拥有真正原型。

10.8.3 v3：ASIC flow 与论文投稿

v3 的目标是让论文闭合。交付物包括：

1. OpenROAD/OpenLane 的 ASIC flow；
2. 面积、时序、功耗估计；
3. 更系统的实验图表；
4. 主论文初稿；

5. 可公开复现实验仓库。

这一步完成后，项目可以从毕业设计进入国际投稿阶段。

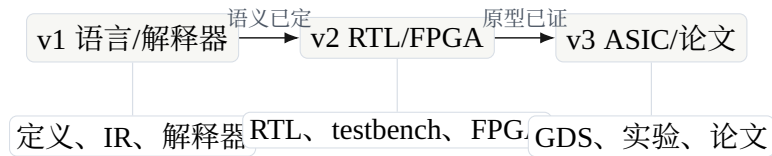


图 10.3 从讲义到原型再到论文的三阶段路线。

10.9 失败边界与修正方向

严肃项目必须写清楚失败边界。对应计算如果失败，通常不是因为概念不够宏大，而是因为某个闭环断裂。

边界

常见失败方式包括：

1. 定义不能闭合：某些操作依赖直觉，无法给出良定义的输出。
2. 矩阵影子不相容：新操作在矩阵化后不再对应传统计算。
3. 表达优势不可复现：只在手工例子中节省表示，而没有一般结构条件。
4. RTL 只实现了普通稀疏矩阵：事件空间、multiplicity、纤维索引和正规化没有真正保留。
5. 实验主张过大：试图证明全面超过 GPU，而不是证明结构化场景下避免矩阵物化。

这些失败方式并非坏事。它们给出了修正方向。例如，如果表达优势只在卷积中成立，那么第一篇论文就应当把范围限定为规则生成对应；如果 CoPU v0 性能不佳，就应当把论文重点放在语义正确与硬件可综合，而不是绝对速度；如果局部 attention 的误差分析尚未成熟，就先把它作为实验任务，不把它放进主定理。

10.10 本讲义后续需要补齐的章节

当前版本已经完成了基础语言与硬件路线，但仍有几类内容需要单独成章。

1. **商对应与群作用**。需要严格定义群作用在对象空间与事件空间上的相容条件，证明取商后的权重下推良定义，并研究商与复合的交换条件。
2. **对应秩的深化**。需要比较强对应秩、弱对应秩、矩阵秩、Boolean rank、nonnegative rank 与 communication complexity 中的矩形覆盖。
3. **局部 attention 的对应化**。需要说明候选生成、权重计算、softmax 正规化与稀疏复合怎样进入对应语言，并给出误差边界。
4. **通信下界**。需要把 fiber join 的数据移动成本形式化，证明某些中间纤维负载给出硬件通信下界。

5. **C-IR 编译器**。需要从数学语句到事件表、纤维索引、tile 映射和 RTL testbench 的完整编译流程。

这些章节不是附加装饰，而是决定项目能否从讲义变成论文与原型系统的关键环节。

10.11 结语：下一步的最小行动

对应计算的长期目标可以很大，但下一步必须很小。最小行动是完成一个可检验闭环：

两个小对应的 C-IR 程序 → 参考解释器输出 → RTL 输出 → 矩阵影子一致性检查.

只要这个闭环成立，后续的一切扩展都有根基。反过来，如果这个闭环不成立，再宏大的语言和芯片设想都没有意义。

安排

下一版最优先的技术工作不是继续扩大叙事，而是写出 C-IR 参考解释器，并用它生成第一个 CoPU v0 testbench。讲义、代码、RTL 和论文必须从这一点开始对齐。

记号索引

$\mathbb{K}$	权重半环或权重域。若无特别说明， $\mathbb{K}$ 是交换半环。
X, Y, Z	对象空间，通常是有限集合；在后续扩展中也可以是拓扑空间、可测空间或有限类型空间。
$X \xleftarrow{s} E \xrightarrow{t} Y$	一个从 X 到 Y 的对应，其中 E 是事件空间， s, t 是源端点与目标端点映射。
$w : E \rightarrow \mathbb{K}$	事件权重函数。
$E_B \times_Y E_A$	纤维积，表示可以首尾相接的事件对。
$\mathcal{B} \circ \mathcal{A}$	两个对应的复合。
$\text{Fib}_A(y)$	对应 $\mathcal{A}$ 在中间点 y 上的目标纤维。
$\zeta_{\mathcal{A}}(u)$	由闭合路径计数定义的路径 zeta 函数。